



INSTITUTO
UNIVERSITÁRIO
DE LISBOA

Aprendizagem por Reforço para Controlo de Enxames

Gonçalo Santos

Mestrado em Inteligência Artificial

Orientador:

Doutor Luís Nunes, Professor do Departamento de Ciências e
Tecnologias da Informação,
Iscte – Instituto Universitário de Lisboa

2026

[Página intencionalmente deixada em branco.]

Departamento de Ciências e Tecnologias da Informação

Aprendizagem por Reforço para Controlo de Enxames

Gonçalo Santos

Mestrado em Inteligência Artificial

Orientador:

Doutor Luís Nunes, Professor do Departamento de Ciências e
Tecnologias da Informação,
Iscte – Instituto Universitário de Lisboa

[Página intencionalmente deixada em branco.]

Aos meus pais e a todos que apoiaram este percurso.

[Página intencionalmente deixada em branco.]

Agradecimentos

Gostaria de expressar a minha gratidão aos meus familiares, por me terem apoiado ao longo desta jornada académica, e à minha namorada, por ter estado sempre ao meu lado nos bons e nos maus momentos. Agradeço também aos colegas e amigos que fiz ao longo do tempo, bem como aos professores, por terem dado o seu melhor para nos ensinar. Por fim, agradeço à TAISCTE por ter feito parte deste meu percurso de mestrado.

This work was partially supported by national funds through Fundação para a Ciência e a Tecnologia, I.P. (FCT), ISTAR-Iscte Pluriannual Projects UID/04466/2025 (<https://doi.org/10.54499/UID/04466/2025>).

[Página intencionalmente deixada em branco.]

Resumo

Esta dissertação apresenta um estudo comparativo rigoroso entre a Aprendizagem por Reforço Multiagente (MARL) e a Otimização Bio-inspirada para o controlo de enxames robóticos. Através de um simulador de alta fidelidade, implementou-se o *framework* MARL RS2C (com os algoritmos PPO e SAC, sob partilha de parâmetros) e um controlador bio-inspirado neuroevolutivo assente numa rede neuronal de grafos com atenção — responsável pela invariância à dimensão do enxame —, avaliando-se ambos em sete cenários de dificuldade crescente (7 execuções independentes por combinação) sob tratamento estatístico não-paramétrico. Os resultados mostram que, com o desenho adequado da função de *fitness* (*shaping* de *homing* geodésico), o controlador evolutivo é estatisticamente superior aos métodos de gradiente em três cenários de navegação e cooperação, indistinguível do melhor deles em mais dois, e é o único a permitir transferência *Zero-Shot* para enxames de dimensão não vista (N de 10 a 100, com 100% de sucesso) sem retreino; os métodos de gradiente preservam a fiabilidade em espaço aberto e uma vantagem clara de eficiência computacional. A hipótese de que a inteligência adaptativa supera a robustez estática confirma-se, assim, apenas parcialmente: a vantagem decisiva reside na *representação* (grafo com atenção) e no desenho do sinal de treino, e não no algoritmo de otimização.

PALAVRAS CHAVE: Robótica de Enxame, MARL, Otimização Bio-inspirada, GNNs.

[Página intencionalmente deixada em branco.]

Abstract

This dissertation presents a rigorous comparative study between Multi-Agent Reinforcement Learning (MARL) and Bio-inspired Optimization for the control of robotic swarms. Using a high-fidelity simulator, we implemented the MARL framework RS2C (with the PPO and SAC algorithms, under parameter sharing) and a neuroevolutionary bio-inspired controller built on a graph neural network with attention — responsible for invariance to swarm size —, evaluating both across seven scenarios of increasing difficulty (7 independent runs per combination) under non-parametric statistical treatment. The results show that, given an adequate fitness design (geodesic *homing* shaping), the evolutionary controller is statistically superior to the gradient-based methods in three navigation and cooperation scenarios, indistinguishable from the best of them in two more, and is the only one enabling Zero-Shot transfer to unseen swarm sizes (N from 10 to 100, at 100% success) without retraining; the gradient-based methods retain reliability in open space and a clear computational efficiency advantage. The hypothesis that adaptive intelligence surpasses static robustness is therefore only partially confirmed: the decisive advantage lies in the *representation* (graph with attention) and in the design of the training signal, rather than in the optimization algorithm.

KEYWORDS: Swarm Robotics, MARL, Bio-inspired Optimization, GNNs.

[Página intencionalmente deixada em branco.]

Índice

Agradecimentos	iii
Resumo	v
Abstract	vii
Lista de Figuras	xi
Lista de Acrónimos	xiii
Capítulo 1. Introdução	1
1.1. Enquadramento e Motivação	1
1.2. Definição do Problema e Justificação da Tese	2
1.3. Objetivos da Tese	3
1.4. Questões de Investigação	4
Capítulo 2. Fundamentos Teóricos e Tecnologias	7
2.1. Introdução ao Capítulo	7
2.2. Swarm Intelligence (SI) e Robótica de Enxame	8
2.3. Paradigmas de Controlo para Enxames	9
2.4. Abordagem 1: Otimização Bio-inspirada	9
2.5. Abordagem 2: Multi-Agent Reinforcement Learning (MARL)	11
Capítulo 3. Estado da Arte	17
3.1. Introdução ao Capítulo	17
3.2. Metodologia da Revisão Sistemática	17
3.3. Caracterização do Corpo de Literatura	21
3.4. Análise dos Melhores Resultados	22
3.5. O Desafio do <i>Deployment Gap</i> em Robótica de Enxame	24
3.6. Lacunas Identificadas e Oportunidade de Melhoria	25
3.7. Artigos-Chave e Ligação ao Problema	26
3.8. Conclusão Parcial	27
3.9. Síntese Comparativa da Literatura	28
Capítulo 4. Desenvolvimento do Ambiente de Simulação	31
4.1. Arquitetura de Software e Stack Tecnológica	31
4.2. O Ambiente de Simulação	32
4.3. Design da Função de Recompensa	38

4.4. Desafios de Engenharia e Otimização Computacional	40
4.5. Estrutura do Projeto e Extensibilidade	42
Capítulo 5. Arquitetura Algorítmica e Controladores	45
5.1. Algoritmo 1: Multi-Agent Reinforcement Learning (RS2C)	45
5.2. Algoritmo 2: Otimização Bio-inspirada (Benchmark)	46
5.3. Plano de Experiências e Tratamento Estatístico	48
Capítulo 6. Resultados Experimentais e Discussão	51
6.1. Metodologia de Avaliação e Notas de Leitura	51
6.2. Desempenho de Tarefa por Cenário (P_{task})	52
6.3. Análise Qualitativa da Navegação: Mapas de Calor	53
6.4. Convergência e Desempenho no Cenário Base (Sandbox)	54
6.5. Desempenho Comparativo por Cenário (Curvas de Aprendizagem)	55
6.6. Fiabilidade e Variância entre <i>Runs</i> (Boxplots)	58
6.7. Visão Global por Algoritmo	60
6.8. Avaliação Determinística (Tarefa Pura)	62
6.9. Análise de Cenários Complexos e Comportamento Emergente	63
6.10. Deceção e Procura por Novidade (<i>Novelty Search</i>)	64
6.11. Análise de Escalabilidade (Zero-Shot Transfer)	66
6.12. Resiliência e Robustez a Falhas de Agentes	68
6.13. Desempenho Computacional do Simulador	69
6.14. Discussão Global e Validação da Hipótese	70
Capítulo 7. Conclusões e Trabalhos Futuros	75
7.1. Conclusões	75
7.2. Resposta às Questões de Investigação	76
7.3. Limitações do Estudo	77
7.4. Contributos	78
7.5. Trabalhos Futuros	79
Referências	81
Apêndice A. Configurações e Hiperparâmetros	85
Apêndice B. Estudos Incluídos na Revisão Sistemática	89
Apêndice C. Excertos de Código Relevantes	99
C.1. Mecanismo de Atenção sobre Vizinhos (QKV)	99
C.2. Física de Deslizamento em Muros (<i>Wall-Sliding</i>)	99
C.3. Campo Geodésico para o <i>Reward Shaping</i>	100
C.4. Recompensa de Exploração <i>Count-Based</i>	100
Apêndice D. Recursos Adicionais	101

Lista de Figuras

Figura 3.1	Fluxograma PRISMA 2020 da revisão conduzida	21
Figura 4.1	Diagrama da arquitetura do simulador e fluxo de interação agente-ambiente.	32
Figura 4.2	Renderizações 3D dos sete cenários de teste, geradas a partir da geometria real do simulador (<code>scripts/render_maps.py</code>). Em cima, da esquerda para a direita: Sandbox, Beco Sem Saída (Muro em U) e Gargalo; ao centro: Quatro Salas, Porta Cooperativa (com a porta destacada a âmbar) e Percepção Cooperativa; em baixo: Porta Cooperativa com Alternativa (<i>bypass</i>). A esfera verde assinala o ninho (ou o alvo móvel, na Percepção Cooperativa).	36
Figura 4.3	Vista de topo (planta) dos sete cenários, evidenciando a geometria das paredes e a largura das passagens. Mesma ordem da Figura 4.2: em cima, Sandbox, Muro em U e Gargalo; ao centro, Quatro Salas, Porta Cooperativa e Percepção Cooperativa; em baixo, Porta Cooperativa com Alternativa.	37
Figura 6.1	Taxa de sucesso (P_{task}) por cenário, em avaliação determinística (7 <i>runs</i> $\times$ 20 episódios emparelhados por algoritmo).	53
Figura 6.2	Recolhas por episódio (média $\pm$ desvio padrão) por cenário. Mede a magnitude do desempenho, para além do sucesso binário.	53
Figura 6.3	Potencial de navegação no Muro U: euclidiano (esquerda) vs. geodésico (direita). As curvas de nível indicam a direção para a qual o termo de progresso atrai o agente; só o geodésico contorna a barra do U.	54
Figura 6.4	Densidade de ocupação dos robôs no Muro U (GNN, PPO e SAC, da esquerda para a direita). O fluxo que contorna a barra do U pelos lados evidencia a navegação resolvida pelo potencial geodésico.	54
Figura 6.5	Curvas de aprendizagem dos três algoritmos no cenário Sandbox. O eixo das abcissas está normalizado (0–100% do orçamento de treino); linha = média de 7 <i>runs</i> , banda = ± 1 desvio padrão.	55
Figura 6.6	Curvas de aprendizagem no cenário Beco Sem Saída (Muro em U).	56
Figura 6.7	Curvas de aprendizagem no cenário Gargalo.	56
Figura 6.8	Curvas de aprendizagem no cenário Quatro Salas (Labirinto).	57

Figura 6.9	Curvas de aprendizagem no cenário Porta Cooperativa.	57
Figura 6.10	Curvas de aprendizagem no cenário Percepção Cooperativa (Alvo Móvel).	57
Figura 6.11	Curvas de aprendizagem no cenário Porta Cooperativa com Alternativa (<i>deceptive</i>).	58
Figura 6.12	Recolhas/ep por <i>run</i> (7 <i>runs</i> , 20 ep cada): Sandbox (esq.) e Muro U (dir.).	59
Figura 6.13	Recolhas/ep por <i>run</i> : Gargalo (esq.) e Quatro Salas (dir.).	59
Figura 6.14	Recolhas/ep por <i>run</i> : Porta Cooperativa (esq.) e Percepção Cooperativa (dir.).	59
Figura 6.15	Recolhas/ep por <i>run</i> : Porta Cooperativa com Alternativa (<i>deceptive</i>).	60
Figura 6.16	Desempenho do GNN (evolutivo) nos sete cenários (média $\pm$ desvio padrão de 7 <i>runs</i>).	61
Figura 6.17	Desempenho do PPO nos sete cenários (média $\pm$ desvio padrão de 7 <i>runs</i>).	61
Figura 6.18	Desempenho do SAC nos sete cenários (média $\pm$ desvio padrão de 7 <i>runs</i>).	62
Figura 6.19	Resumo geral: recolhas por episódio (avaliação determinística) por algoritmo e cenário; barras de erro = desvio padrão sobre os 140 episódios.	62
Figura 6.20	Comportamento emergente no visualizador 3D: contorno do obstáculo em U (esq.) e navegação no labirinto de Quatro Salas (dir.).	64
Figura 6.21	Comportamento cooperativo: abertura sincronizada da porta (esq.) e cerco a 360° do alvo móvel (dir.).	64
Figura 6.22	Degradação (ou estabilidade) do desempenho com o aumento do número de agentes, sem retreino.	68
Figura 6.23	Resiliência do exame perante a perda súbita de 10% dos agentes a meio do episódio (R_{robust}).	69

Lista de Acrónimos

AE: Algoritmos Evolutivos

AI: Artificial Intelligence

CTDE: Centralized Training with Decentralized Execution

Dec-POMDP: Decentralized Partially Observable Markov Decision Process

GAE: Generalized Advantage Estimation

GNN: Graph Neural Network

MARL: Multi-Agent Reinforcement Learning

ML: Machine Learning

MLP: Multi-Layer Perceptron

PBRs: Potential-Based Reward Shaping

PPO: Proximal Policy Optimization

PSO: Particle Swarm Optimization

RL: Reinforcement Learning

SAC: Soft Actor-Critic

SI: Swarm Intelligence

SLR: Systematic Literature Review

SR: Swarm Robotics

:

[Página intencionalmente deixada em branco.]

Introdução

1.1. Enquadramento e Motivação

A robótica de enxame (*Swarm Robotics*), enquanto subcampo da inteligência artificial e dos sistemas multiagente, representa uma mudança de paradigma fundamental na engenharia contemporânea, transitando de arquiteturas monolíticas e robôs complexos para a coordenação de grandes grupos de agentes simples [1]. Esta transição fundamenta-se na premissa de que a inteligência não reside na complexidade individual de cada membro, mas sim na sofisticação das interações locais entre eles. Esta premissa ecoa a perspectiva de que a inteligência é, na sua essência, um fenómeno de processamento e organização de informação, em larga medida independente do substrato físico que a concretiza [2]. Desde as primeiras conceptualizações em meados da década de 1990 até às aplicações de ponta em 2025, a motivação central deste campo permanece constante: a emulação da inteligência coletiva observada na natureza — como em colónias de formigas ou cardumes — para atingir robustez, flexibilidade e escalabilidade superiores através de comportamento emergente [3], [4].

A urgência no desenvolvimento destes sistemas é impulsionada por aplicações críticas em ambientes de elevada incerteza e perigosidade, tais como a vigilância persistente de infraestruturas, a exploração espacial e a resposta a desastres naturais [5]. Nestes cenários, a redundância intrínseca do enxame é a sua maior vantagem: o sistema é desenhado para ser resiliente a falhas individuais, garantindo que a perda de um único agente não comprometa a integridade da missão global [6].

No entanto, a implementação prática desta visão descentralizada enfrenta barreiras técnicas significativas no que toca ao controlo. Métodos tradicionais, frequentemente dependentes de uma unidade central de processamento ou de modelos matemáticos globais, falham invariavelmente ao escalar o número de agentes (N). Este fenómeno, conhecido como a “explosão da complexidade de interações”, ocorre porque o espaço de estados e ações cresce exponencialmente com o aumento da população do enxame, tornando impossível a programação explícita de todas as contingências operacionais [7]. Conforme discutido por Hüttenrauch et al. (2019), a verdadeira escalabilidade exige que as políticas de controlo sejam independentes do número total de agentes, um desafio que as abordagens convencionais ainda têm dificuldade em endereçar de forma eficiente [1].

Neste contexto, a comunidade científica dividiu-se historicamente em duas abordagens principais para o desenho destes controladores descentralizados:

- (1) **Otimização Bio-inspirada:** Métodos como o *Particle Swarm Optimization* (PSO) e Algoritmos Evolutivos (AE) focam-se na robustez *offline* [4]. Tratam o desenho do controlador como um problema de otimização estocástica, procurando os pesos ótimos de uma rede neural ou parâmetros de regras heurísticas antes da implantação real [8], [9]. Embora robustos, estes métodos tendem a ser estáticos, podendo carecer de adaptabilidade perante mudanças dinâmicas e imprevistas no ambiente [10].
- (2) **Multi-Agent Reinforcement Learning (MARL):** Focado na adaptabilidade *online*, o MARL utiliza o paradigma de aprendizagem sequencial para que os agentes descubram políticas cooperativas através da interação direta com o ambiente [11]. A grande promessa do MARL reside na sua capacidade de ajuste em tempo real e na aprendizagem de políticas complexas formalizadas como um *Decentralized Partially Observable Markov Decision Process* (Dec-POMDP) [12].

A pertinência desta investigação sustenta-se na lacuna identificada por autores como Majid et al. (2024), que sublinham a escassez de estudos que comparem diretamente a eficiência e a robustez destas duas escolas em cenários operacionais dinâmicos [7]. Esta tese surge da necessidade urgente de validar qual destas estruturas melhor responde aos desafios de escalabilidade e adaptação exigidos pelas aplicações robóticas de 2025, propondo uma avaliação comparativa rigorosa entre a inteligência adaptativa do MARL e a robustez consolidada da otimização bio-inspirada [13].

1.2. Definição do Problema e Justificação da Tese

Apesar do crescimento exponencial no interesse por sistemas autónomos descentralizados, existe uma lacuna crítica na literatura científica: a escassez de uma comparação direta (*benchmarking*) e rigorosa do desempenho de paradigmas concorrentes em cenários de controlo de enxames que sejam, simultaneamente, complexos e dinâmicos [7], [13]. O estado da arte atual revela que, embora existam múltiplos *frameworks* de controlo, a maioria das investigações foca-se no refinamento de um algoritmo isolado, negligenciando a validação cruzada necessária para determinar a aplicabilidade industrial de cada método [7].

O *Multi-Agent Reinforcement Learning* (MARL), embora promissor devido à sua capacidade de aprender comportamentos emergentes complexos, enfrenta desafios teóricos profundos. Entre estes, destaca-se a **não-estacionaridade** do ambiente: como todos os agentes aprendem e alteram as suas políticas simultaneamente, a dinâmica do sistema do ponto de vista de um agente individual muda constantemente, violando a propriedade de Markov necessária para a convergência de algoritmos tradicionais [12]. Além disso, a explosão do espaço de ações conjuntas em enxames de grande escala dificulta a estabilidade do treino e a otimização de funções de recompensa esparsas.

Por outro lado, os métodos evolutivos e bio-inspirados (como o PSO) têm demonstrado uma robustez consolidada, tratando o controlo como a exploração de uma superfície de *fitness* global [4]. Contudo, como notado por Kegeleirs et al. (2025), estas abordagens

resultam frequentemente em controladores estáticos que carecem de adaptabilidade em tempo real perante mudanças estruturais imprevistas no ambiente [14]. Esta rigidez cria o que a literatura denomina como *Deployment Gap* (fosso de implantação), onde algoritmos otimizados em laboratório falham ao enfrentar o ruído e a incerteza do mundo real.

Portanto, a definição do problema desta tese é a **necessidade de uma avaliação comparativa rigorosa e sistemática**. Este estudo visa determinar qual o *framework* de controlo que melhor responde aos desafios práticos do controlo de enxames através de três eixos fundamentais:

- **Desempenho da Tarefa:** Avaliar a eficiência absoluta na concretização de objetivos globais (ex: taxa de cobertura de área ou taxa de recolha em *foraging*). Seguindo a linha de Majid et al. (2024), pretende-se quantificar se a convergência mais lenta do MARL resulta, de facto, numa política final superior em eficiência energética comparativamente aos métodos bio-inspirados [7].
- **Escalabilidade (Zero-Shot Transfer):** Analisar como a *performance* se degrada à medida que o número de agentes (N) aumenta para além do cenário de treino. É crucial validar se a representação por *Graph Neural Networks* (GNNs) — cujo mecanismo de invariância à cardinalidade é caracterizado por Chen et al. (2024) [15], ainda que no domínio das redes sem fios — permite uma escalabilidade superior à das políticas de dimensão de entrada fixa (MLP) usadas pelas *baselines* de gradiente. Note-se que, neste trabalho, é o controlador *evolutivo* que instancia a arquitetura de grafo: o eixo da escalabilidade opõe, portanto, representações (grafo *vs.* vetor fixo) e não paradigmas de otimização.
- **Robustez e Adaptação Dinâmica:** Testar a resiliência do sistema perante a perda súbita de agentes (falhas de *hardware*) ou a introdução de obstáculos móveis imprevistos. O objetivo é verificar se a capacidade de aprendizagem contínua do MARL compensa a sua complexidade de implementação face à estabilidade *offline* dos algoritmos evolutivos.

Esta tese justifica-se pela urgência de transformar o controlo de enxames de uma disciplina puramente teórica numa ferramenta de engenharia fiável. Através do uso de um simulador de alta-fidelidade, este trabalho pretende fornecer dados quantitativos que orientem a escolha do paradigma de controlo em função dos requisitos de missão e da disponibilidade de recursos computacionais.

1.3. Objetivos da Tese

Com base na definição do problema e na lacuna de investigação identificada na literatura recente, o objetivo geral desta dissertação é **desenvolver um *framework* de simulação de alta-fidelidade e realizar um *benchmarking* comparativo rigoroso entre o *Multi-Agent Reinforcement Learning* (MARL) e algoritmos de otimização bio-inspirados** (e.g., Algoritmos Evolutivos) aplicados ao controlo descentralizado de

enxames [14]. Pretende-se determinar, de forma quantitativa, em que condições a inteligência adaptativa e emergente do MARL supera a robustez estática das abordagens de otimização tradicionais.

Para atingir o objetivo geral, foram definidos os seguintes objetivos específicos:

- **Desenvolver um Ambiente de Simulação de Referência:** Projetar e implementar um simulador em ambiente Python capaz de modelar tarefas fundamentais de enxame, como o *foraging* (forrageamento) cooperativo dinâmico e a perseguição cooperativa (cerco de alvos móveis), em cenários de dificuldade de navegação crescente [13]. O ambiente deve contemplar modelos físicos realistas, observação local e parcial dos agentes e a simulação de falhas súbitas de agentes para testar a resiliência do sistema [6].
- **Implementar um Framework MARL Escalável (RS2C):** Desenvolver o *framework Robust and Scalable Swarm Control* (RS2C) assente em execução descentralizada com partilha de parâmetros [15]. Este objetivo inclui a exploração de **redes de grafos com mecanismos de atenção**, que garantem a invariância à permutação e permitem que um controlador treinado com N agentes seja aplicado a enxames de outra dimensão sem retreino (*Zero-Shot Transfer*).
- **Implementar uma *Baseline* de Otimização Bio-inspirada:** Desenvolver um controlador baseado em **Neuroevolução** (um Algoritmo Evolutivo, AE) para servir de termo de comparação de desempenho [8], [9]. Este controlador é otimizado *offline*, sem gradientes, para encontrar os pesos de uma rede neuronal de grafos com atenção, representando o paradigma de robustez pré-programada [4].
- **Realizar uma Análise Comparativa e Estatística:** Executar experiências controladas utilizando as métricas de performance da tarefa, escalabilidade e robustez a perturbações [7]. Os resultados deverão ser validados através de tratamento estatístico rigoroso — testes de hipótese não-paramétricos sobre episódios emparelhados, complementados por medidas de tamanho de efeito —, garantindo que as conclusões sobre a superioridade de um paradigma sobre o outro são cientificamente sustentadas.

1.4. Questões de Investigação

Decorrentes da definição do problema e dos três eixos de avaliação enunciados (desempenho, escalabilidade e robustez), esta dissertação propõe-se responder às seguintes questões de investigação, retomadas explicitamente nas Conclusões (Secção 7.2):

- QI1. Desempenho de tarefa:** Qual dos paradigmas de controlo descentralizado — MARL por gradiente (PPO/SAC) ou neuroevolução — atinge maior eficácia na tarefa de *foraging* cooperativo, em cenários de dificuldade de navegação crescente?
- QI2. Escalabilidade (*Zero-Shot Transfer*):** Uma política treinada com $N = 20$ agentes transfere-se, sem retreino, para enxames de dimensão distinta ($N = 10$ a

$N = 100$)? Em que medida essa capacidade depende da arquitetura da política (rede de grafos com atenção vs. MLP de dimensão fixa)?

- QI3. Robustez a falhas:** Como se degrada o desempenho coletivo de cada paradigma perante a falha súbita de uma fração dos agentes durante a execução da tarefa?
- QI4. Critério de escolha:** Em que condições operacionais (tipo de cenário, dimensão do exame, requisitos de fiabilidade) se justifica preferir um paradigma ao outro?
- QI5. Desenho da função de *fitness*:** Em que medida o desempenho do controlador neuroevolutivo depende do desenho da função de *fitness*? Em particular, pode um *shaping* de navegação (*homing*), que premeia a aproximação ao objetivo antes da primeira recolha, desbloquear cenários que uma *fitness* de tarefa pura não resolve?
- QI6. Deceção e procura por novidade:** Em cenários cujo gradiente de recompensa é enganador (*deceptive*), a hibridização da neuroevolução com procura por novidade (*Novelty Search*) melhora o resultado final face à otimização puramente objetiva?

[Página intencionalmente deixada em branco.]

Fundamentos Teóricos e Tecnologias

2.1. Introdução ao Capítulo

Este capítulo estabelece os fundamentos teóricos e os alicerces tecnológicos necessários para uma compreensão aprofundada do controlo descentralizado de enxames. A transição de sistemas robóticos isolados para coletivos coordenados exige uma base conceptual que interliga a biologia, a engenharia de controlo e a aprendizagem automática avançada. Assim, o presente capítulo visa dotar a dissertação do rigor matemático e técnico indispensável para suportar a investigação proposta.

Em primeiro lugar, serão definidos e contextualizados os conceitos de *Swarm Intelligence* (SI) e *Swarm Robotics* (SR). Esta análise inicial é fundamental para compreender como comportamentos globais complexos e auto-organizados podem emergir a partir de regras de interação locais extremamente simples. A robustez destes sistemas reside precisamente na sua natureza descentralizada, o que impõe desafios únicos ao desenho de controladores eficientes: como demonstram Rivière et al. (2020), sintetizar políticas que operem apenas sobre o estado relativo dos vizinhos próximos — e que ainda assim aproximem o desempenho de um planeador global — é, em si mesmo, um problema de investigação em aberto [16].

Posteriormente, o foco incidirá sobre as duas principais classes de paradigmas de controlo que constituem o núcleo do *benchmarking* desta tese:

- (1) **Otimização Bio-inspirada:** Serão explorados algoritmos como a Otimização por Enxame de Partículas (*Particle Swarm Optimization* - PSO) e os Algoritmos Evolutivos (AE). Estas técnicas tratam o controlo como um problema de busca num espaço de parâmetros global, focando-se na convergência para soluções estáveis e robustas em cenários estáticos.
- (2) **Multi-Agent Reinforcement Learning (MARL):** Será introduzida a formalização matemática do problema através do modelo de Processo de Decisão de Markov Parcialmente Observável Descentralizado (*Dec-POMDP*). Esta secção detalhará como a aprendizagem por reforço permite a emergência de políticas adaptativas através da interação *online*, abordando conceitos críticos como a arquitetura *Centralized Training with Decentralized Execution* (CTDE) e o papel das *Graph Neural Networks* (GNNs) na garantia da escalabilidade do enxame, tal como defendido por Chen et al. (2024).

Ao estabelecer estas definições, este capítulo permite identificar o “Deployment Gap” (fosso de implantação) discutido por Kegeleirs et al. (2025), justificando a necessidade

de uma metodologia rigorosa para validar a transição destes algoritmos do domínio da simulação para aplicações reais de alta fidelidade [14].

2.2. Swarm Intelligence (SI) e Robótica de Enxame

A Inteligência de Enxame (*Swarm Intelligence* - SI) refere-se ao comportamento coletivo e coordenado de sistemas descentralizados e auto-organizados, sejam eles naturais ou artificiais [3]. Este paradigma baseia-se na observação de sistemas biológicos — tais como colônias de formigas, enxames de abelhas ou cardumes de peixes — onde agentes individuais com capacidades cognitivas limitadas conseguem, através de interações sociais simples, resolver problemas de elevada complexidade que excederiam a capacidade de qualquer indivíduo isolado [4].

A Robótica de Enxame (*Swarm Robotics* - SR) surge como a aplicação direta dos princípios de SI ao domínio de sistemas multi-robô [14]. Ao contrário da robótica cooperativa tradicional, a SR foca-se na coordenação de um grande número de agentes, tipicamente homogêneos e de baixo custo, que operam sem uma infraestrutura de controlo centralizada ou um mapa global do ambiente. O controlo nestes sistemas é inerentemente distribuído, o que confere ao coletivo três propriedades fundamentais identificadas na literatura:

- **Robustez:** A redundância do sistema garante que a missão prossiga mesmo perante a falha de múltiplos agentes [1].
- **Escalabilidade:** O sistema mantém a sua eficiência independentemente da dimensão do enxame (N), uma vez que as interações são predominantemente locais.
- **Flexibilidade:** O enxame consegue adaptar-se a diferentes tarefas e ambientes dinâmicos sem necessidade de reprogramação externa.

O alicerce técnico destes sistemas reside no comportamento emergente. Conforme sistematizado por Bonabeau, Dorigo e Theraulaz (1999) no estudo dos insetos sociais, as ações globais coordenadas emergem de um ciclo de *feedback* contínuo entre regras de interação locais simples e o ambiente [17]. Um mecanismo crítico para esta emergência é a **estigmergia**, um método de coordenação indireta onde os agentes alteram o ambiente (por exemplo, através de trilhos de feromonas ou comunicação local ruidosa), influenciando as ações subsequentes de outros membros do grupo — o princípio que está na origem da própria Otimização por Colônias de Formigas [3].

No entanto, projetar manualmente estas regras locais para atingir um objetivo global específico é um desafio de engenharia não trivial. Como não existe um “conhecimento global” partilhado, o desenho do controlador deve garantir que a soma das micro-decisões individuais resulte na macro-estabilidade do enxame. Este desafio justifica a exploração de métodos de Aprendizagem Automática Avançada, onde, em vez de programar regras explícitas, o sistema “aprende” ou “evolui” as interações ótimas através de reforço ou otimização.

2.3. Paradigmas de Controle para Enxames

O controle de sistemas multi-robô em larga escala foca-se primordialmente no desenho das chamadas “regras locais” de interação. O desafio fundamental reside na resolução do problema inverso da robótica de enxame: como determinar comportamentos individuais simples que, quando executados em paralelo, resultem numa macro-estabilidade e no cumprimento de um objetivo global. Existem duas abordagens principais na literatura contemporânea que esta tese se propõe a analisar e comparar:

- (1) **Otimização Bio-inspirada (Offline):** Esta categoria abrange duas subfamílias principais frequentemente utilizadas em robótica:
 - **Algoritmos Evolutivos (AE):** Inspirados na seleção natural (Darwin), utilizam operadores como mutação e cruzamento para evoluir uma população de controladores ao longo de gerações.
 - **Inteligência de Enxame (SI):** Inspirados em comportamentos sociais (e.g., bandos de pássaros), como o *Particle Swarm Optimization* (PSO), onde a solução emerge do movimento cooperativo de partículas no espaço de busca.

Ambas as abordagens procuram *offline* o melhor conjunto de parâmetros fixos, maximizando uma função de *fitness*. Embora gerem comportamentos robustos, resultam frequentemente em controladores rígidos perante mudanças ambientais não previstas.

- (2) **Aprendizagem por Reforço (Online/Adaptativa):** Em vez de otimizar parâmetros fixos antes da implantação, o agente aprende uma política $\pi(a | o)$ através da interação sequencial com o ambiente, guiado por um sinal de recompensa. Na sua extensão multiagente (MARL), cada robô ajusta o seu comportamento em função das observações locais, permitindo em princípio a adaptação a condições não previstas no desenho. O custo desta flexibilidade é uma otimização substancialmente mais difícil: a não-estacionaridade introduzida pela aprendizagem simultânea de todos os agentes, a atribuição de crédito em recompensas coletivas e a sensibilidade ao desenho do sinal de recompensa (Secção 4.3) são desafios centrais que esta tese aborda empiricamente.

Esta distinção é crucial para o *benchmarking* proposto, pois permite avaliar se o custo computacional e a complexidade de treino da aprendizagem automática são justificados por um ganho real em adaptabilidade face às soluções otimizadas tradicionais.

2.4. Abordagem 1: Otimização Bio-inspirada

Esta abordagem utiliza algoritmos de busca meta-heurística para navegar no vasto espaço de parâmetros θ de um controlador — frequentemente representado por uma rede neuronal (*Neuroevolution*) ou uma máquina de estados — com o intuito de maximizar uma função de desempenho global $J(\theta)$.

2.4.1. Fundamentos Matemáticos dos Algoritmos Evolutivos

O processo evolutivo opera sobre uma **população** $\Pi_g = \{\theta_1, \theta_2, \dots, \theta_K\}$ de K controladores candidatos, denominados **genomas**, onde cada genoma $\theta_k \in \mathbb{R}^{|\theta|}$ é o vetor concatenado de todos os pesos e *biases* de uma rede neuronal. A topologia arquitetural da rede é fixada *a priori*; apenas os seus parâmetros numéricos são sujeitos a otimização — esta abordagem denomina-se **Neuroevolução**. Ao contrário do RL, a Neuroevolução não necessita de gradientes calculáveis, tornando-a aplicável a qualquer função de recompensa não-diferenciável.

Função de *Fitness*: O critério de qualidade de cada genoma, $J(\theta_k)$, foi deliberadamente desenhado para que a **tarefa** domine a seleção, relegando o *shaping* para o papel de gradiente de desempate. A formulação final combina as recolhas com um *shaping* de **navegação terminal** (*homíng*):

$$J(\theta_k) = \underbrace{\bar{f}_k \cdot \lambda_{task}}_{\text{tarefa}} + \underbrace{\beta \cdot \bar{h}_k}_{\text{shaping de homíng}} \quad (2.1)$$

onde $\bar{f}_k = \frac{1}{E} \sum_{e=1}^E f_{k,e}$ é o número médio de recolhas (a tarefa pura) e $\bar{h}_k \in [0, 1]$ é o **homíng**: a fração média de aproximação ao ninho *no fim do episódio*, medida por agente como $\text{clip}((\Phi_0 - \Phi_T)/\Phi_0, 0, 1)$, em que Φ é o potencial de navegação até ao ninho (geodésico nos cenários com paredes) avaliado no primeiro e no último instante do episódio. O peso $\lambda_{task} = 10000$ e a amplitude $\beta = 5000$ garantem que uma única recolha vale o dobro do *shaping* máximo: assim que um genoma recolhe, a tarefa domina a seleção.

A escolha do *homíng* terminal como *shaping* — em vez do retorno acumulado $\bar{R}_k$ — é a decisão de desenho central desta função, e resultou de uma iteração experimental documentada no Capítulo 6. Numa primeira formulação, o desempate era o retorno acumulado comprimido por uma $\tanh$; contudo, o retorno acumulado é *farmável*: os termos de progresso e exploração rendem a cada passo, pelo que genomas que vagueiam indefinidamente pelo mapa maximizam-no sem nunca entrar no ninho (observaram-se retornos brutos $\approx 88\,000$ com zero recolhas) — parar no ninho, a pré-condição da tarefa, corta precisamente o rendimento por passo. O *homíng* elimina esta patologia por construção: depende apenas dos estados *extremos* do episódio (posição inicial vs. final) e não do caminho percorrido, pelo que vaguear não o aumenta; a única forma de o maximizar é *terminar* junto ao ninho. A média sobre $E = 4$ episódios com *seeds* fixas reduz a variância estocástica da avaliação, estabilizando o sinal de seleção entre gerações.

Mutação Gaussiana *Element-wise*: Em vez de perturbar a totalidade dos $|\theta|$ pesos em simultâneo — o que degradaria irrecuperavelmente a diversidade populacional em redes de grande dimensão —, implementa-se uma mutação esparsa controlada por uma máscara binária $\mathbf{m} \in \{0, 1\}^{|\theta|}$:

$$\theta_j^{(\text{filho})} = \theta_j^{(\text{pai})} + m_j \cdot \varepsilon_j, \quad m_j \sim \text{Bernoulli}(p_{mut}), \quad \varepsilon_j \sim \mathcal{N}(0, \sigma^2) \quad (2.2)$$

com $p_{mut} = 0.1$. Apenas 10% dos pesos são perturbados por operação, preservando a estrutura do controlador pai enquanto introduz variações localizadas. Esta formulação é equivalente ao *Evolution Strategies* de Salimans et al. (2017) com máscara esparsa.

Sigma Decay — Anilamento Adaptativo da Exploração: O desvio padrão σ controla a magnitude das perturbações. Para implementar a transição gradual de exploração agressiva para refinamento local, σ decai geometricamente a cada geração g :

$$\sigma_{g+1} = \max(\sigma_{\min}, \sigma_g \cdot \rho) \quad (2.3)$$

com $\sigma_0 = 0.1$, $\rho = 0.999$ e $\sigma_{\min} = 0.03$. Este decaimento assegura que nas fases iniciais o algoritmo explora amplamente a superfície $J(\theta)$, convergindo progressivamente para refinamento local do melhor ótimo encontrado. O patamar mínimo $\sigma_{\min}$ é deliberadamente conservador: deixar σ colapsar para valores residuais fazia o campeão sobre-ajustar-se às *seeds* de avaliação (*overfitting* de seleção), degradando a generalização medida na avaliação determinística.

Elitismo e Seleção por Ordenação: Após avaliar todos os K genomas, estes são ordenados por $J(\theta_k)$ em ordem decrescente. Os $e = \max(3, \lfloor 0.2 \cdot K \rfloor)$ melhores genomas (elites) são preservados intactos na nova geração, garantindo que o melhor comportamento descoberto nunca se perde por mutações desfavoráveis. Os restantes $K - e$ genomas são gerados como mutações de cópias aleatórias dos elites. Para $K = 30$, preservam-se $e = 6$ genomas elite por geração.

Otimização por Enxame de Partículas (PSO): O PSO modela cada controlador candidato como uma “partícula” que navega num espaço de busca multidimensional. A inovação crucial deste método reside na partilha de informação social: cada partícula ajusta a sua velocidade e posição baseada na sua melhor experiência pessoal (*pBest*) e na melhor experiência coletiva descoberta por todo o enxame (*gBest*). Hassen e Amin (2025) empregam o PSO para a otimização global do percurso, delegando a reação a obstáculos imprevistos num controlador reativo separado (*Dynamic Window Approach*) — arranjo que ilustra a característica estrutural do paradigma: a convergência do enxame produz um controlador *estático*, de elevada qualidade para o problema otimizado, mas cuja adaptação ao imprevisto tem de ser acrescentada por fora [8].

2.5. Abordagem 2: Multi-Agent Reinforcement Learning (MARL)

O MARL representa a fronteira da Aprendizagem Automática aplicada a sistemas coletivos, focando-se na aprendizagem de políticas adaptativas (π) através da interação sequencial.

2.5.1. Processos de Decisão de Markov (MDPs) e Dec-POMDP

Fundamentos do Processo de Decisão de Markov (MDP): A base matemática da aprendizagem por reforço é o **Processo de Decisão de Markov (MDP)**, definido pela tupla $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \gamma \rangle$:

- $\mathcal{S}$: espaço de estados do ambiente (contínuo em robótica);

- $\mathcal{A}$: espaço de ações ($\mathbf{a}_i \in [-1, 1]^3$ nesta tese — deslocamento contínuo nos três eixos egocêntricos do robô: Frontal, Direito e Superior, escalado por um passo máximo de 0.2m);
- $\mathcal{P}(s' | s, a)$: dinâmica de transição — probabilidade de atingir s' ao executar a em s ;
- $\mathcal{R}(s, a, s') \in \mathbb{R}$: sinal de recompensa escalar;
- $\gamma \in [0, 1]$: fator de desconto temporal ($\gamma = 0.99$ nesta implementação).

Funções de Valor e Equação de Bellman: O objetivo do agente é encontrar uma política $\pi^*(a | s)$ que maximize o **retorno esperado descontado** $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$. A qualidade de uma política é quantificada por duas funções fundamentais.

A **Função de Valor de Estado** $V^\pi(s)$ mede o retorno esperado ao seguir π a partir de s :

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s \right] \quad (2.4)$$

A **Função Q** (ou Função de Valor de Ação) $Q^\pi(s, a)$ mede o retorno esperado ao executar a em s e depois seguir π :

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s, a_0 = a \right] \quad (2.5)$$

Estas funções satisfazem a **Equação de Bellman**, que permite a sua estimação recursiva:

$$V^\pi(s) = \sum_a \pi(a | s) \sum_{s'} \mathcal{P}(s' | s, a) [\mathcal{R}(s, a, s') + \gamma V^\pi(s')] \quad (2.6)$$

A política ótima verifica a **Equação de Bellman de Optimalidade**:

$$Q^*(s, a) = \mathbb{E}_{s'} \left[\mathcal{R}(s, a, s') + \gamma \max_{a'} Q^*(s', a') \right] \quad (2.7)$$

Extensão para Dec-POMDP em Enxames: Em sistemas multiagente, o estado global $s \in \mathcal{S}$ é inacessível a qualquer robô individual. O problema estende-se para o **Processo de Decisão de Markov Parcialmente Observável Descentralizado (Dec-POMDP)**, definido pela tupla $\langle \mathcal{N}, \mathcal{S}, \{\mathcal{A}_i\}_{i \in \mathcal{N}}, \mathcal{P}, \mathcal{R}, \Omega, \{\mathcal{O}_i\}_{i \in \mathcal{N}}, \gamma \rangle$, onde $\mathcal{N} = \{1, \dots, N\}$ é o conjunto de agentes, Ω é o espaço de observações conjuntas, e $\mathcal{O}_i(s) \in \mathbb{R}^{111}$ é a observação local e parcial do agente i [11], [12], [18]. Cada robô mantém uma política descentralizada $\pi_i(a_i | o_i)$ e o objetivo coletivo é maximizar o retorno conjunto:

$$J(\pi_1, \dots, \pi_N) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t \mathcal{R}(s_t, \mathbf{a}_t) \right], \quad \mathbf{a}_t = (a_1^t, \dots, a_N^t) \quad (2.8)$$

2.5.2. Otimização de Políticas: Do Actor-Critic ao PPO e SAC

Arquitetura Actor-Critic: Tanto o PPO como o SAC assentam na arquitetura **Actor-Critic**, onde duas redes são treinadas em conjunto:

- **Ator** (π_θ): mapeia a observação o_i para uma distribuição sobre ações. Em execução descentralizada, **apenas o ator é necessário** a bordo de cada robô.

- **Crítico** (V_ϕ ou Q_ψ): rede auxiliar que estima o retorno esperado, utilizada **exclusivamente durante o treino** para calcular gradientes de atualização do ator.

PPO — *Clipped Surrogate Objective*: O PPO (*Proximal Policy Optimization*) [19] é um algoritmo *on-policy*: recolhe *rollouts* com a política atual e atualiza os parâmetros através de um objetivo *clipped* que previne passos de gradiente destabilizadores. Seja a **razão de importância**:

$$r_t(\theta) = \frac{\pi_\theta(a_t | o_t)}{\pi_{\theta_{\text{old}}}(a_t | o_t)} \quad (2.9)$$

O objetivo PPO *clipped* é:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon) \hat{A}_t \right) \right] \quad (2.10)$$

com $\varepsilon = 0.2$. A estimativa da vantagem $\hat{A}_t$ é calculada via **Estimação de Vantagem Generalizada (GAE)** [20]:

$$\hat{A}_t = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t) \quad (2.11)$$

O objetivo final inclui ainda um erro de regressão do crítico e um bônus de entropia:

$$L^{\text{PPO}}(\theta, \phi) = L^{\text{CLIP}}(\theta) - c_1 \mathbb{E}_t [(V_\phi(s_t) - V_t^{\text{targ}})^2] + c_2 \mathbb{E}_t [H(\pi_\theta(\cdot | o_t))] \quad (2.12)$$

SAC — Maximização de Entropia: O SAC (*Soft Actor-Critic*) [21] é um algoritmo *off-policy* que acrescenta ao objetivo de RL o termo de entropia de Shannon $H(\pi) = -\mathbb{E}[\log \pi(a | s)]$, promovendo a **exploração máxima** consistente com o desempenho na tarefa:

$$J(\pi) = \mathbb{E}_{\rho_\pi} \left[\sum_{t=0}^{\infty} \gamma^t \left(r_t + \alpha H(\pi(\cdot | s_t)) \right) \right] \quad (2.13)$$

onde $\alpha > 0$ é o **coeficiente de temperatura**, que controla o equilíbrio entre exploração e recompensa ($\alpha = 0.1$ nesta implementação). A **Função de Valor Suave** incorpora diretamente o termo de entropia:

$$V^*(s) = \mathbb{E}_{a \sim \pi} [Q^*(s, a) - \alpha \log \pi(a | s)] \quad (2.14)$$

O ator é atualizado minimizando a divergência KL relativamente à distribuição de Boltzmann $\propto \exp(Q/\alpha)$:

$$J_\pi(\phi) = \mathbb{E}_{s_t \sim \mathcal{D}} [\mathbb{E}_{a_t \sim \pi_\phi} [\alpha \log \pi_\phi(a_t | o_t) - Q_\psi(s_t, a_t)]] \quad (2.15)$$

O SAC mantém dois críticos Q_{ψ_1}, Q_{ψ_2} cujo mínimo é usado no alvo (*double Q-trick*), reduzindo o viés de sobre-estimação. As transições (s_t, a_t, r_t, s_{t+1}) são armazenadas num **Replay Buffer** de capacidade $|\mathcal{D}| = 500\,000$, permitindo a reutilização *off-policy* de experiências passadas.

O Desafio da Não-Estacionaridade: A principal barreira teórica na aplicação de MARL a exames é a não-estacionaridade do ambiente. Num sistema multiagente, as transições

de estado para um agente i dependem das ações de todos os outros agentes. Como todos os robôs estão a aprender e a alterar as suas políticas simultaneamente, a dinâmica do ambiente é instável, o que viola a propriedade fundamental de Markov e dificulta a convergência dos algoritmos tradicionais de agente único.

Arquitetura CTDE e Escalabilidade com GNNs: Para mitigar a instabilidade, a literatura recorre frequentemente à arquitetura de **Treinamento Centralizado com Execução Descentralizada** (CTDE) [22]. Nesta abordagem, um componente “Crítico” tem acesso ao estado global durante a fase de treino para estabilizar os sinais de recompensa, enquanto o “Ator” (controlador do robô) utiliza apenas observações locais durante a fase de execução [15]. Importa notar que a implementação desta tese adota uma variante mais estrita — treino e execução descentralizados, com o crítico a partir da mesma observação local do ator (Secção 7.3) —, pelo que o CTDE é aqui apresentado como contexto da literatura, e não como a arquitetura utilizada.

Contudo, para garantir que o enxame é verdadeiramente escalável, a investigação moderna tem integrado **Graph Neural Networks (GNNs)** e mecanismos de atenção espacial.

2.5.3. Fundamentos de Graph Neural Networks e Mecanismos de Atenção

Representação em Grafo do Enxame: O enxame de N robôs é modelado como um grafo dinâmico $G_t = (V_t, E_t)$, onde $V_t = \{1, \dots, N\}$ é o conjunto de nós (robôs). Em vez de um corte rígido por raio de comunicação, adota-se um **grafo completamente ligado** ($E_t = \{(i, j) : i \neq j\}$) cujas arestas são ponderadas dinamicamente pelos coeficientes de atenção α_{ij} (Equação 2.19). A topologia efetiva é assim aprendida e reflete, a cada passo, a relevância relativa de cada vizinho para a decisão do agente — arestas irrelevantes recebem peso próximo de zero, emulando o efeito de um corte de proximidade sem o impor *a priori*.

Propagação de Mensagens (*Message Passing*): O paradigma geral das GNNs é o framework MPNN (*Message Passing Neural Network*). A representação $\mathbf{h}_i^{(l)}$ do nó i na camada l é atualizada por:

$$\mathbf{h}_i^{(l+1)} = \text{UPDATE}\left(\mathbf{h}_i^{(l)}, \text{AGG}\left(\left\{\mathbf{m}_{j \rightarrow i}^{(l)} : j \in \mathcal{N}(i)\right\}\right)\right) \quad (2.16)$$

onde $\mathbf{m}_{j \rightarrow i}^{(l)} = \text{MSG}(\mathbf{h}_i^{(l)}, \mathbf{h}_j^{(l)})$ é a mensagem do vizinho j , e $\text{AGG}(\cdot)$ é uma função invariante à permutação (soma, média ou máximo). Esta invariância é fundamental: a operação é independente da ordem dos vizinhos, garantindo que o enxame é tratado como um conjunto não-ordenado.

Mecanismo de Atenção Escalada: A arquitetura implementada (GNNAgent3D) usa o mecanismo de **atenção de produto interno escalado** [23], distinto da formulação aditiva original das *Graph Attention Networks* [24]. Para o agente i , codificam-se as suas características locais $\mathbf{h}_i^{\text{env}} = f_{\text{enc}}(\mathbf{o}_i^{\text{env}}) \in \mathbb{R}^d$ e as dos $N-1$ vizinhos $\mathbf{h}_j^{\text{nbr}} = f_{\text{nbr}}(\mathbf{n}_{j,i}) \in \mathbb{R}^d$,

formando a matriz de contexto:

$$\mathbf{H} = [\mathbf{h}_i^{env}; \mathbf{h}_1^{nbr}; \dots; \mathbf{h}_{N-1}^{nbr}] \in \mathbb{R}^{N \times d} \quad (2.17)$$

As projeções de consulta, chave e valor são calculadas pelas matrizes treináveis $\mathbf{W}^Q$, $\mathbf{W}^K$, $\mathbf{W}^V \in \mathbb{R}^{d \times d}$:

$$\mathbf{Q} = \mathbf{h}_i^{env} \mathbf{W}^Q \in \mathbb{R}^{1 \times d}, \quad \mathbf{K} = \mathbf{H} \mathbf{W}^K \in \mathbb{R}^{N \times d}, \quad \mathbf{V} = \mathbf{H} \mathbf{W}^V \in \mathbb{R}^{N \times d} \quad (2.18)$$

Os **coeficientes de atenção** α_{ij} medem a importância que o agente i atribui ao vizinho j :

$$\alpha_i = \text{softmax}\left(\frac{\mathbf{Q} \mathbf{K}^\top}{\sqrt{d}}\right) \in \mathbb{R}^{1 \times N} \quad (2.19)$$

A divisão por $\sqrt{d}$ previne a saturação do *softmax* em espaços de alta dimensão. O **pool de mensagens agregado** é:

$$\mathbf{m}_i = \alpha_i \mathbf{V} = \sum_{j=0}^{N-1} \alpha_{ij} \mathbf{h}_j \in \mathbb{R}^d \quad (2.20)$$

A representação final concatena o vetor ego com o pool de mensagens e gera a ação contínua:

$$\mathbf{z}_i = [\mathbf{h}_i^{env}; \mathbf{m}_i] \in \mathbb{R}^{2d}, \quad \mathbf{a}_i = f_{actor}(\mathbf{z}_i) \in [-1, 1]^3 \quad (2.21)$$

Ao modelar o enxame como um grafo dinâmico $G = (V, E)$, onde os nós são robôs e as arestas são ligações de comunicação, é possível aprender políticas invariantes ao número de agentes. A razão é estrutural: como Chen et al. (2024) demonstram — em redes multi-salto sem fios, e não em robótica, mas o argumento é o mesmo — o número de parâmetros treináveis de uma política sobre grafo é *independente do número de agentes*, pelo que a política aprendida se aplica a grafos maiores sem retreino [15]. Em robótica, Zhang et al. (2022) confirmam empiricamente esta invariância na exploração multi-robô [25]. É esta propriedade — o **Zero-Shot Transfer** — que esta dissertação testa em enxames de $N = 10$ a $N = 100$, confrontando-a com as limitações de rigidez das abordagens de dimensão fixa.

[Página intencionalmente deixada em branco.]

CAPÍTULO 3

Estado da Arte

3.1. Introdução ao Capítulo

Este capítulo apresenta uma Revisão Sistemática da Literatura (SLR) rigorosa, centrada nos paradigmas de controlo de enxames discutidos no capítulo anterior. Numa área em constante evolução como a Robótica de Enxame, onde as publicações científicas transitaram das conceptualizações iniciais da década de 1990 para aplicações críticas e complexas em 2025, torna-se imperativo adotar um método de análise que seja simultaneamente metódico e reproduzível. O objetivo primordial desta revisão não é apenas sumarizar o conhecimento existente, mas sim analisar de que forma a Aprendizagem Automática — especificamente o *Multi-Agent Reinforcement Learning* (MARL) — e os métodos de Otimização Bio-inspirada (como o PSO e AE) têm convergido ou divergido na resolução de problemas de coordenação descentralizada [5].

A motivação para esta análise sistemática reside na observação de que a comunidade científica tem, historicamente, operado em silos: a literatura de MARL foca-se predominantemente na estabilidade do treino sob condições de não-estacionaridade, enquanto a literatura de otimização prioriza a robustez de controladores estáticos. Esta divisão tem dificultado a realização de confrontos diretos que validem qual o paradigma mais resiliente perante o chamado “Deployment Gap” (fosso de implantação). Conforme discutido por Majid et al. (2024), a falta de um referencial comum de desempenho impede uma compreensão clara de quando a adaptabilidade *online* compensa a complexidade computacional do treino em larga escala [7].

Desta forma, a revisão foca-se em identificar e mapear as lacunas de *benchmarking* que justificam a necessidade desta dissertação. Serão examinadas as tendências de escalabilidade por via de *Graph Neural Networks* (GNNs), o desenho automático de controladores de enxame do lado bio-inspirado, e as abordagens de hibridização — tanto as que aco- plam MARL a pesquisa heurística, como o *Large Neighborhood Search* no planeamento de trajetórias multiagente [26], quanto as que combinam efetivamente aprendizagem por reforço com evolução colaborativa de populações [27]. O capítulo encerra sustentando a metodologia adotada no Capítulo 4 como resposta às limitações encontradas na literatura [14].

3.2. Metodologia da Revisão Sistemática

Para garantir o rigor científico, a reprodutibilidade e a transparência no processo de seleção da informação, adotou-se uma metodologia baseada no protocolo PRISMA (*Preferred Reporting Items for Systematic Reviews and Meta-Analyses*, versão 2020). O protocolo foi

definido *antes* de executar qualquer pesquisa e está integralmente disponível no repositório da dissertação (`docs/PROTOCOLO_SLR.md`, Apêndice D), a par do registo completo da triagem; qualquer leitor pode, com esses ficheiros, reproduzir as pesquisas e reconstruir cada uma das decisões de inclusão e exclusão aqui reportadas.

3.2.1. Pergunta e Critérios de Elegibilidade

A revisão responde à pergunta que motiva a dissertação: *como se comparam, no controlo descentralizado de enxames robóticos, os paradigmas de MARL e de otimização bio-inspirada quanto a desempenho de tarefa, escalabilidade e robustez — e que lacunas de benchmarking comparativo persistem?*

Foram incluídos estudos que, cumulativamente: (I1) abordam o controlo, navegação ou coordenação de **sistemas multi-robô** (dois ou mais agentes, excluindo o robô isolado); (I2) aplicam **MARL/RL profundo** ou **otimização bio-inspirada** (PSO, ACO, algoritmos evolutivos, neuroevolução); (I3) apresentam **validação empírica quantitativa**; (I4) foram publicados entre **2020 e 2026** com revisão por pares; e (I5) têm texto integral acessível. Os critérios de exclusão, aplicados com registo do motivo em cada registo, foram: robô único (E1), ausência de validação experimental — incluindo revisões e artigos conceptuais (E2), ausência de dados quantitativos (E3), falta de revisão por pares ou integridade da publicação em causa (E4), duplicação (E5), método fora do âmbito — multi-robô mas controlado por teoria de jogos, *Dynamic Window Approach* ou campos potenciais clássicos, isto é, nem MARL nem bio-inspirado (E6), e tarefa fora do âmbito — alocação e escalonamento industrial, SLAM, comunicações ou manipulação (E7), uma vez que a revisão incide sobre **navegação, exploração, *foraging*, formação e coordenação de movimento**.

3.2.2. Protocolo de Pesquisa e Bases de Dados

As pesquisas foram executadas a **13 de julho de 2026** sobre o **Scopus** e o **IEEE Xplore**, com acesso institucional. A ACM Digital Library foi dispensada por decisão explícita: o Scopus indexa a maior parte das suas atas relevantes para esta pergunta e a sobreposição observada entre as duas bases utilizadas já era substancial (203 duplicados em 883 registos, $\approx 23\%$), pelo que o retorno marginal de uma terceira base seria reduzido. O Google Scholar não foi usado como fonte primária por não ser reproduzível — os seus resultados variam entre utilizadores e sessões — nem exportável em bloco.

A *string* de pesquisa exige o conceito de enxame no **título** e o paradigma de controlo em título, resumo ou palavras-chave:

```
TITLE ( "swarm robot*"OR "robot swarm*"OR "multi-robot")
AND TITLE-ABS-KEY ( "reinforcement learning"OR "MARL"OR "neuroevolution"
                    OR "evolutionary algorithm"OR "particle swarm"
                    OR "bio-inspired optimization")
AND TITLE-ABS-KEY ( control OR navigation OR coordination OR foraging
                    OR "path planning") AND PUBYEAR > 2019
```


A calibração desta *string* é ela própria um resultado metodológico que importa reportar. Numa primeira versão, o conceito de enxame era procurado também no resumo e nas palavras-chave, o que devolveu **7 628 registos**: a expressão “*multi-agent system*” num resumo captura toda a literatura de sistemas multiagente sem componente robótica. Restringir o conceito de enxame ao título — decisão de âmbito, e não de conveniência — concentra a pesquisa nos trabalhos cujo *objeto de estudo* é o coletivo de robôs, devolvendo 456 registos no Scopus. Ambas as contagens ficam registadas no repositório, pela mesma razão que se regista tudo o resto: uma revisão só é sistemática se as suas escolhas forem auditáveis.

3.2.3. Processo de Seleção (Fluxo PRISMA)

As duas pesquisas devolveram **883 registos** (456 no Scopus, 427 no IEEE Xplore). A deduplicação automática por DOI e por título normalizado removeu **203 duplicados**, restando **680 registos únicos**.

Na **triagem** por título e resumo foram excluídos **201 registos**. O motivo dominante foi, com igual peso (75 registos cada), o robô único (E1) e o método fora do âmbito (E6). O primeiro é um artefacto instrutivo da própria *string*: a expressão *particle swarm* designa um *método* de otimização, pelo que a pesquisa recolhe abundante literatura de PSO aplicado a um único manipulador ou a um robô móvel isolado — trabalhos legítimos, mas sem enxame. Excluíram-se ainda 42 revisões e artigos conceptuais (E2), quatro registos por integridade ou ausência de revisão por pares (E4) — entre os quais um artigo formalmente retirado pelo IEEE por violação dos princípios de publicação — e quatro duplicados que a deteção automática não apanhara (E5), por terem DOI numa base e não na outra.

Os restantes **479 estudos** passaram à fase de **elegibilidade**. Este número, incomportável para leitura integral no âmbito de uma dissertação individual, é em si mesmo um dado sobre a maturidade do campo. Aplicaram-se então dois critérios adicionais, ambos registados no protocolo: a exclusão de tarefas fora do âmbito da pergunta (E7: alocação e escalonamento, SLAM, comunicações, manipulação, plataformas de simulação), que retirou 79 estudos; e a restrição do corpo a trabalhos que tratem efetivamente de *enxames* e estejam publicados em **venues de referência** — ou que, independentemente do venue, constituam o *núcleo* da pergunta desta revisão, isto é, que apliquem *novelty search*, neuroevolução ou *foraging* evolutivo a enxames, ou que comparem *diretamente* os dois paradigmas (E8, 342 estudos excluídos).

A lista de venues de referência cobre deliberadamente *ambos* os campos em confronto: robótica (IEEE T-RO, RA-L, ICRA, IROS, *Autonomous Robots*), MARL e IA (AAMAS, IEEE T-Cybernetics, TNNLS) e computação evolutiva e inteligência de enxame (GECCO, IEEE CEC, *Swarm Intelligence*, *Swarm and Evolutionary Computation*). Esta simetria não é acessória: uma lista restrita aos venues de robótica e MARL — a tentação natural de quem parte da literatura de aprendizagem — teria excluído sistematicamente a produção

de qualidade do paradigma bio-inspirado, enviesando a revisão *contra* um dos dois objetos que ela existe para comparar.

O processo fixou **58 estudos** para análise qualitativa e extração de dados (Figura 3.1; a lista completa consta do Apêndice B).

3.2.4. Fronteira entre o Corpo da Revisão e as Fontes de Fundamentação

Nem todas as referências citadas nesta dissertação provêm do corpo da revisão, e a distinção deve ser explícita para que o leitor não procure reconciliar o n do fluxograma com o número de entradas na bibliografia. Distinguem-se três categorias. O **corpo da revisão** (os 58 estudos do Apêndice B) responde à pergunta da Secção 3.2 e é o único que sustenta as contagens aqui reportadas. As **fontes primárias dos métodos** utilizados — PPO [19], SAC [21], estratégias evolutivas [9], redes de atenção sobre grafo [24], procura por novidade [28], Dec-POMDP [18] — descrevem as ferramentas empregues e são citadas nos Capítulos 2 e 5, não por resultarem desta pesquisa. Finalmente, as **revisões e artigos de enquadramento** [7], [13], [14], excluídos do corpo por não apresentarem validação experimental própria (critério E2), são usados para situar o problema — e um deles, precisamente por catalogar apenas *baselines* de aprendizagem, serve de evidência do isolamento metodológico que aqui se documenta [13].

Um caso merece nota, por ser instrutivo quanto aos limites da própria *string* de pesquisa. O trabalho de Chen et al. (2024), que fundamenta a escolha arquitetural desta tese, não integra o corpo por não ser robótico: caracteriza a invariância à dimensão do grafo em *redes multi-salto sem fios* [15]. É citado pelo mecanismo que estabelece — o número de parâmetros de uma política sobre grafo não depende do número de agentes — e não por um resultado em enxames; a evidência robótica equivalente vem de dentro do corpo [25]. Do mesmo modo, a pesquisa não recupera trabalhos cujo título use *swarm navigation* sem o termo *robot* [29], uma consequência assumida da decisão de exigir o conceito de enxame no título.

3.2.5. Ameaças à Validade da Revisão

Três limitações devem ser declaradas. Primeiro, a revisão foi conduzida por um **único revisor**, sem dupla triagem independente, o que não permite calcular concordância entre avaliadores — uma restrição inerente a uma dissertação individual. Segundo, a triagem dos 680 registos foi **assistida por um modelo de linguagem**, que propôs para cada registo uma decisão e o respetivo motivo a partir do título e do resumo, cabendo ao autor a validação; o registo integral das decisões, publicado no repositório, permite a auditoria de qualquer uma delas. Terceiro, a restrição a duas bases, a publicações em inglês e ao recorte temporal 2020–2026 introduz um possível viés de cobertura, deliberado quanto ao recorte (o foco é o estado da arte posterior à adoção de GNNs) mas assumido quanto às bases.

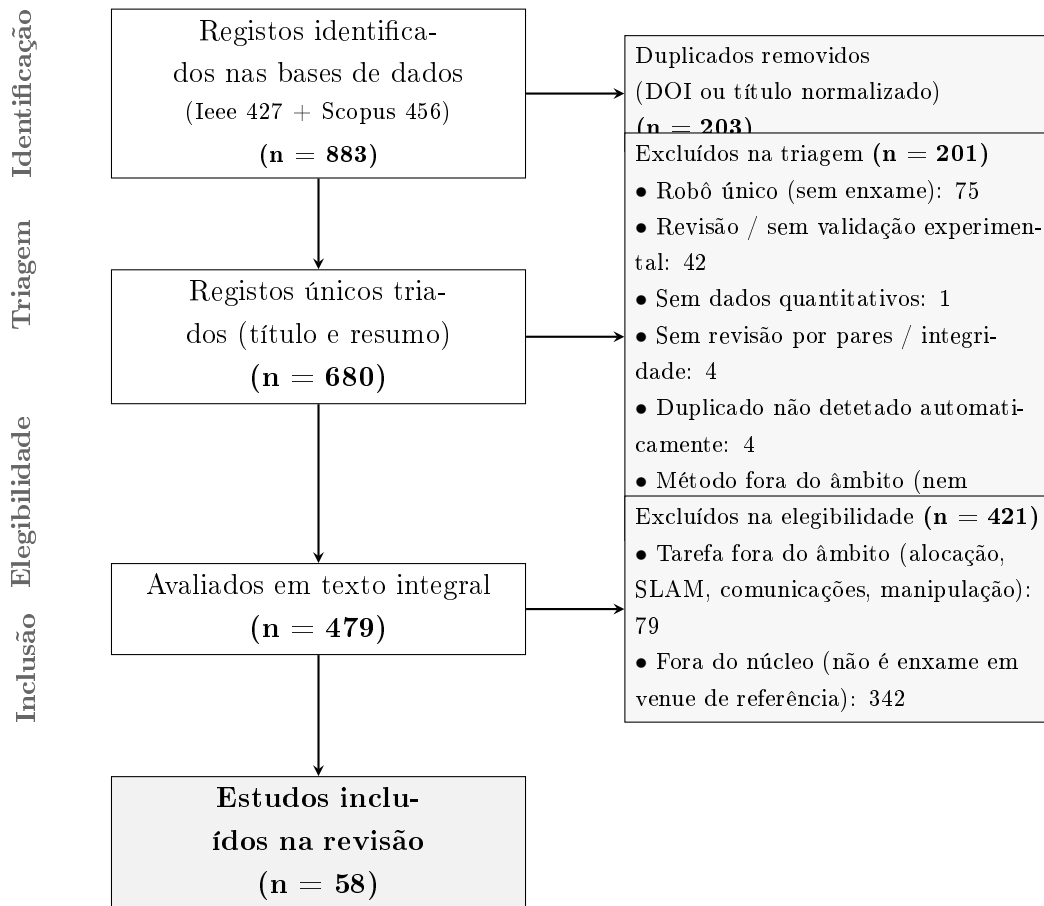


FIGURA 3.1. Fluxograma PRISMA 2020 da revisão conduzida. Os números são gerados automaticamente a partir do registo de triagem (docs/slr/screening.csv no repositório, Apêndice D), onde cada decisão está associada ao respetivo critério de exclusão.

3.3. Caracterização do Corpo de Literatura

Antes de analisar os subgrupos, importa caracterizar quantitativamente os 58 estudos incluídos, pois a sua distribuição é, por si só, o argumento central deste capítulo.

Classificando cada estudo pelo paradigma de controlo que emprega, obtém-se um equilíbrio quase perfeito: **21 estudos assentam exclusivamente em MARL** (aprendizagem por reforço profunda, nas suas variantes MADDPG, MAPPO, PPO, DQN ou actor-critic), **23 assentam exclusivamente em otimização bio-inspirada** (PSO, colónias de formigas, algoritmos evolutivos, neuroevolução) e apenas **4 combinam explicitamente os dois** numa arquitetura híbrida; os restantes recorrem a formas de aprendizagem que não se enquadram em nenhuma das duas famílias (aprendizagem por imitação, por demonstração, ou modelos de campo médio). Quanto aos eixos de avaliação, **30 dos 58** abordam a escalabilidade ou a generalização a dimensões de enxame não vistas, e **14** testam explicitamente robustez a falhas, ruído ou ataques.

O dado decisivo, porém, é outro. Dos 58 estudos, **apenas dois** colocam os dois paradigmas em confronto direto — e um deles fá-lo por hibridização (combinando colónias

de formigas com *Q-learning*), não por comparação. Existe, portanto, na literatura recolhida, **um único estudo que compara empiricamente MARL com otimização bio-inspirada na navegação de enxames** [30], e fá-lo numa única tarefa, sem avaliar a transferência para dimensões de enxame não vistas (*Zero-Shot*) nem a robustez a falhas.

Esta assimetria — dois campos igualmente produtivos (21 contra 23 estudos), que quase nunca se medem um contra o outro — é a lacuna que a presente dissertação ocupa, e deixa de ser aqui uma afirmação de princípio para passar a ser uma contagem verificável no registo de triagem publicado (Apêndice B).

3.4. Análise dos Melhores Resultados

3.4.1. Subgrupo A: Otimização Bio-inspirada e Desenho Automático de Enxames

O trabalho mais consequente deste subgrupo provém da linha de *desenho automático* de enxames desenvolvida no IRIDIA. Hasselmann, Ligot e Birattari (2023) geram, por *novelty search*, um repertório de comportamentos que é depois montado modularmente em controladores de enxame, mostrando que a procura por novidade — ao abandonar o objetivo explícito — produz repertórios comportamentais que um objetivo único não descobriria [31]. A mesma equipa demonstra que o desenho automático *off-line* transfere para plataformas físicas distintas daquela para que foi otimizado [32] e, num desenvolvimento notável, que missões compostas podem ser especificadas por *demonstração* em vez de por função-objetivo, recorrendo a aprendizagem por reforço inversa [33]. A relevância direta para esta dissertação é considerável: confirma que a procura por novidade é um mecanismo estabelecido no desenho de enxames (Secção 6.10), e que a dificuldade de *especificar a função de aptidão* — que aqui se revelou determinante (QI5) — é reconhecida na literatura ao ponto de motivar métodos que a dispensam.

Na tarefa concreta de *foraging*, que constitui o cenário desta dissertação, o subgrupo é ativo e recente: aplicam-se **NEAT** para evoluir estratégias adaptativas de recolha em ambientes com obstáculos [34], [35] e otimização multiobjetivo para arbitrar entre recolha e consumo energético [36]. Uma linha complementar — a **aprendizagem social** — trata a evolução como um processo *online* e distribuído, em que os controladores se propagam entre robôs durante a própria missão, sob restrições realistas de largura de banda [37], [38].

3.4.2. Subgrupo B: MARL, Escalabilidade e Arquiteturas de Grafo

No subgrupo de aprendizagem, a preocupação dominante é a mesma que motiva a escolha arquitetural desta tese: como manter uma política válida quando a dimensão do enxame muda. As respostas convergem para duas famílias. A primeira substitui a política de entrada fixa por **redes neuronais de grafos**: Zhang et al. (2022) propõem GNNs hierárquicas por saltos para exploração multi-robô, obtendo políticas que operam sobre vizinhanças de dimensão variável [25]. O mecanismo subjacente é geral e foi caracterizado com clareza fora da robótica, em redes multi-salto sem fios, por Chen et al. (2024): numa

política sobre grafo, o número de parâmetros treináveis não depende do número de agentes, pelo que a política transfere para grafos maiores sem retreino [15] — é esta a propriedade que o controlador de grafo desta dissertação explora, e que a avaliação *Zero-Shot* do Capítulo 6 testa em enxames de $N = 10$ a $N = 100$. A segunda recorre a aproximações de **campo médio**, em que o agente reage à distribuição estatística dos vizinhos em vez de a cada um individualmente, tornando o custo de coordenação independente de N [39]. Li et al. (2024) atacam o problema pelo lado da tarefa, mantendo a formação de padrões coerente perante *variação da escala do enxame* [40].

Mais próximo da formulação adotada nesta dissertação, Yigit et al. (2026) propõem um sistema de navegação de enxames com execução descentralizada, modelado como um *Multi-Agent Markov Decision Process* (MAMDP) e treinado com **PPO** sob *parameter sharing* — uma política única, partilhada por todos os agentes, que decidem a partir da observação local e sem comunicação explícita [29]. Os autores reportam convergência rápida e elevadas taxas de sucesso numa arena bidimensional com obstáculos circulares imóveis, recorrendo a uma função de recompensa multicomponente (progresso em direção ao alvo, penalização de colisões, bónus de sucesso coletivo e amortecimento de velocidade junto ao objetivo) da mesma família da aqui empregue. O trabalho valida, assim, o paradigma PPO descentralizado adotado nesta dissertação como *baseline*, e as suas limitações delimitam com precisão o espaço que esta tese ocupa.

Três dessas limitações são pertinentes. Primeira, a **representação da vizinhança**: a observação de cada agente resume-se a oito componentes — a sua posição, a do objetivo, e as posições relativas do vizinho *mais próximo* e do obstáculo *mais próximo*. O coletivo é, pois, percecionado através de um único vizinho, o que impede o agente de captar a estrutura do enxame (densidade, distribuição, formação) e contrasta com a atenção sobre todos os vizinhos empregue no controlador de grafo aqui proposto. Segunda, a **escalabilidade não é avaliada**: a dimensão do enxame é mantida fixa, não havendo qualquer teste de transferência para N não visto em treino. Terceira, a **avaliação** confronta a política aprendida apenas com políticas aleatórias, sem *baseline* de outro paradigma, sem repetição em execuções independentes e sem inferência estatística. Os próprios autores remetem para trabalho futuro a inclusão de obstáculos dinâmicos, agentes heterogêneos e limitações de comunicação — aspetos que a metodologia desta dissertação contempla.

3.4.3. O Estudo Comparativo Isolado

Merece registo autónomo o único estudo do corpo que confronta diretamente os dois paradigmas na navegação de enxames [30]. É significativo que provenha do mesmo grupo (Universidade de Miskolc) que havia aplicado RL profundo à navegação de enxames [41]: a comparação surge, portanto, de investigadores que chegaram à otimização bio-inspirada *a partir* da aprendizagem, e não de uma tradição comparativa estabelecida. A sua existência isolada — um estudo em 58, restrito a uma tarefa de navegação (ainda que avaliada em ambientes de complexidade crescente) e sem transferência para dimensões de enxame não

vistas, sem robustez a falhas e sem significância estatística — confirma, mais do que refuta, o diagnóstico de isolamento metodológico entre as duas comunidades.

3.5. O Desafio do *Deployment Gap* em Robótica de Enxame

Uma política de controlo que convirja para taxas de sucesso próximas de 100% em simulação não oferece, por si só, qualquer garantia de desempenho num robô físico. O fosso entre os dois domínios — designado *reality gap* ou *Deployment Gap* — é apontado por Kegeleirs et al. (2025) como o principal entrave à transição da robótica de enxame do laboratório para aplicações reais [14]. A causa é estrutural: o treino por reforço otimiza a política para a distribuição exata de transições $\mathcal{P}(s' | s, a)$ do simulador, pelo que qualquer discrepância sistemática entre essa distribuição e a do mundo real é explorada como se fosse uma regularidade legítima do ambiente, conduzindo a comportamentos que falham — por vezes catastroficamente — na plataforma física.

3.5.1. Fontes do Fosso de Realidade

As discrepâncias que alimentam o *Deployment Gap* podem agrupar-se em três classes:

- **Dinâmica de atuação e física não modelada:** um simulador cinemático — como o adotado nesta dissertação, em que a ação é um deslocamento egocêntrico — ignora a inércia, a aceleração finita dos motores, a fricção assimétrica entre rodas, a folga mecânica e o escorregamento. Num chassis *differential-drive* real (e-puck, Khepera IV), um comando de velocidade não se traduz instantaneamente na deslocação pretendida, e pequenos enviesamentos de calibração entre motores acumulam-se em desvios de trajetória que a política treinada nunca observou.
- **Ruído e enviesamento sensorial:** a observação simulada (bússola de *homíng*, LiDAR, posição relativa de vizinhos) é exata ou corrompida apenas por ruído idealizado. Os sensores reais introduzem ruído não-gaussiano, latência, ângulos mortos, reflexões espúrias no LiDAR e erros de odometria que derivam ao longo do tempo. Uma política que dependa de leituras precisas da direção ou da distância degrada-se à medida que estas se tornam ruidosas.
- **Comunicação e coordenação:** o sinal de recrutamento estigmérgico entre vizinhos é, no simulador, transmitido de forma instantânea e fiável. Em hardware real, a comunicação sofre de latência, perda de pacotes, alcance limitado e colisões de mensagens — e a perceção da posição dos vizinhos depende ela própria de sensores imperfeitos.

3.5.2. Agravamento à Escala do Enxame

O *Deployment Gap* é particularmente severo em sistemas multiagente. Ao contrário de um único robô, num enxame os pequenos erros de cada agente **compõem-se e interagem**: um desvio de trajetória individual altera a configuração espacial percebida pelos vizinhos, que ajustam o seu comportamento, propagando a perturbação pelo coletivo. Comportamentos coordenados que emergem de um equilíbrio delicado em simulação (a

sincronização de três agentes na porta cooperativa, o cerco a 360 do alvo móvel) são precisamente os mais frágeis perante este efeito de amplificação, pois dependem de uma coordenação fina que o ruído real tende a dissolver.

3.5.3. Estratégias de Mitigação

A literatura propõe várias famílias de técnicas para estreitar o fosso. A **aleatorização de domínio** (*domain randomization*) é a mais difundida: durante o treino, randomizam-se sistematicamente os parâmetros físicos e sensoriais do simulador (massas, fricções, ganhos de motor, níveis de ruído), forçando a política a ser robusta a uma *distribuição* de dinâmicas em vez de a uma única, de modo que o mundo real surja como apenas mais uma amostra dessa distribuição [42]. Complementarmente, a **identificação de sistema** (*system identification*) calibra os parâmetros do simulador a partir de medições reais, reduzindo o enviesamento à partida; e o **ajuste fino** (*fine-tuning*) no robô físico, ou em simuladores de fidelidade crescente, adapta a política às condições efetivas de operação.

3.5.4. Posicionamento desta Dissertação

Embora a validação *sim-to-real* permaneça fora do âmbito experimental deste trabalho (Secção 7.3), várias opções de desenho foram tomadas no sentido de *antecipar* o fosso, e não de o ignorar. A observação é mantida estritamente **local e parcial** (LiDAR limitado a 8 m, sem acesso ao estado global), replicando as restrições sensoriais de plataformas como o e-puck; introduz-se **observabilidade parcial genuína** ao nível dos obstáculos (detetáveis apenas localmente); a robustez é explicitamente testada através da **injeção de falhas** de 10% dos agentes a meio do episódio (Secção 6.12); e o ambiente inclui **perturbações dinâmicas** (obstáculos e alvo móveis). A elevada resiliência observada perante falhas sugere que a partilha de parâmetros confere ao enxame uma redundância funcional favorável à transferência. A validação empírica em hardware, recorrendo às técnicas de *domain randomization* e *system identification* acima descritas, constitui a extensão natural deste trabalho (Secção 7.5).

3.6. Lacunas Identificadas e Oportunidade de Melhoria

A revisão permite substituir por evidência quantificada aquilo que, na literatura, é habitualmente afirmado por intuição: a principal barreira ao avanço da robótica de enxame não é a escassez de algoritmos — os 58 estudos incluídos, produzidos em sete anos, atestam o contrário — mas a **ausência de rigor comparativo entre paradigmas**. As duas comunidades operam em silos metodológicos quase estanques, e a Secção 3.3 mede a espessura da parede que os separa: 21 estudos de MARL, 23 de otimização bio-inspirada, **um único** a compará-los.

O silo manifesta-se de forma simétrica em cada campo. A comunidade de aprendizagem afere os seus algoritmos contra outros algoritmos de aprendizagem — a própria *suite* de referência da área contempla exclusivamente métodos de MARL, sem qualquer *baseline* evolutiva [13] — privilegiando a estabilidade do treino e a superação da não-estacionaridade. A comunidade de otimização, por seu turno, mede-se em otimalidade

de percurso e eficiência energética, tipicamente em mapas conhecidos, sem confrontar a rigidez dos seus controladores com a adaptabilidade que a aprendizagem promete [7]. Nenhuma das duas responde à pergunta que um engenheiro de enxames efetivamente enfrenta: *qual dos paradigmas escolher, para este cenário, com este orçamento de cómputo e esta variabilidade de dimensão do enxame?*

Desta fragmentação decorrem três consequências que esta dissertação procura corrigir:

- **Ausência de *benchmarking* cruzado sob protocolo idêntico.** Sem cenários, métricas, orçamentos de treino e protocolos de avaliação comuns, os resultados publicados pelos dois campos são incomensuráveis. Esta tese avalia os três controladores nos mesmos sete cenários, com o mesmo número de execuções independentes, avaliação determinística emparelhada e inferência não-paramétrica ao nível do *run*.
- **Escalabilidade afirmada, raramente medida.** Ainda que 30 dos 58 estudos invoquem a escalabilidade, a avaliação por transferência *Zero-Shot* para dimensões de enxame não vistas em treino permanece rara; as soluções que a asseguram fazem-no pela *representação* — grafos [15], [25] ou campo médio [39] — e não pelo algoritmo de otimização. É esta separação entre representação e otimizador que a presente dissertação isola experimentalmente (QI2).
- **O desenho do sinal de treino como variável esquecida.** A literatura de desenho automático de enxames reconhece a dificuldade de especificar a função-objetivo, ao ponto de desenvolver métodos que a contornam — procura por novidade [31] ou aprendizagem por demonstração [33]. Mas o efeito do *desenho da aptidão* sobre o sucesso ou o colapso do paradigma evolutivo não é, tipicamente, tratado como objeto experimental. Esta tese trata-o (QI5), e conclui que dele depende a diferença entre um controlador que colapsa e um que iguala os métodos de gradiente.

A oportunidade que se abre é, assim, dupla: fornecer o *benchmark* cruzado que falta, e fazê-lo sobre uma arquitetura — o grafo dinâmico $G = (V, E)$ com atenção [15], [24] — que confere invariância à permutação e escalabilidade, permitindo comparar não apenas algoritmos, mas os *eixos* em que cada paradigma é preferível.

3.7. Artigos-Chave e Ligação ao Problema

Seis trabalhos delimitam com precisão o espaço que esta dissertação ocupa — três provenientes dos 58 estudos incluídos (Iskandar; Hasselmann; Kegeleirs) e três de enquadramento complementar, externos ao corpus e assinalados como tal: Zhang, identificado pela pesquisa mas excluído na fase de elegibilidade (E8 — fora do núcleo da revisão); Chen, doutro domínio de aplicação; e Yigit, não recuperado pelas cadeias de pesquisa (o título não contém nenhum dos termos de exame robótico exigidos no título). A lacuna quantificada adiante (“um em 58”) refere-se exclusivamente ao corpus da revisão:

- (1) **Iskandar et al. (2024)** — o *único* estudo do corpo a comparar diretamente aprendizagem por reforço com PSO na navegação de enxames [30]. Confirma que a comparação é possível e pertinente, mas fá-la numa só tarefa (em ambientes de complexidade crescente), sem transferência para dimensões de enxame não vistas, sem robustez a falhas e sem tratamento estatístico: exatamente as três dimensões que esta tese acrescenta.
- (2) **Hasselmann, Ligot e Birattari (2023)** — gera repertórios comportamentais por *novelty search* para o desenho automático de enxames [31]. Estabelece a procura por novidade como mecanismo legítimo no desenho de enxames, fundamentando a hibridização testada na QI6.
- (3) **Zhang et al. (2022)** — GNNs hierárquicas para exploração multi-robô, operando sobre vizinhanças de dimensão variável [25]; é a evidência *robótica* de que a invariância à dimensão do enxame é uma propriedade da *representação*, e não do otimizador (QI2). (Recuperado pela pesquisa mas excluído na elegibilidade — E8, tarefa de exploração fora do núcleo da revisão; é citado pelo mecanismo de representação.)
- (4) **Chen et al. (2024)** — estabelece o mecanismo que sustenta essa invariância: numa política sobre grafo, o número de parâmetros treináveis é *independente do número de agentes*, pelo que a política aprendida se aplica diretamente a grafos maiores sem retreino [15]. O trabalho é conduzido em redes multi-salto sem fios, e não em robótica — é citado aqui pelo mecanismo, não pelo domínio.
- (5) **Kegeleirs et al. (2024)** — mede a **transferibilidade** do desenho automático *off-line* de enxames, da simulação para o real e entre plataformas distintas [32]; delimita o *Deployment Gap* que esta dissertação antecipa no desenho mas não valida empiricamente (Secção 7.3).
- (6) **Yigit et al. (2026)** — PPO descentralizado com *parameter sharing* em navegação de enxame [29], da mesma família da *baseline* aqui adotada (identificado fora da pesquisa sistemática, cujas cadeias exigem termos de enxame robótico no título); a sua avaliação — contra políticas aleatórias, com vizinhança reduzida ao agente mais próximo, sem variação da dimensão do enxame e sem inferência estatística — ilustra o déficit de rigor comparativo que motiva este trabalho.

3.8. Conclusão Parcial

A revisão sistemática confirma que os dois paradigmas em confronto estão, ambos, em desenvolvimento ativo: 21 dos 58 estudos incluídos assentam em MARL e 23 em otimização bio-inspirada. O enquadramento complementar mostra, por sua vez, que a escalabilidade a dimensões de enxame não vistas é hoje resolvida pela *representação* — grafos com atenção [15], [25] ou aproximações de campo médio [39] — e que o paradigma evolutivo dispõe de instrumentos maduros para tarefas decetivas, da procura por novidade [31] à aprendizagem social distribuída [37].

O que a revisão *não* encontra é um confronto rigoroso entre os dois. Um único estudo em 58 compara empiricamente as duas famílias na navegação de enxames [30], e fá-lo sem transferência para dimensões de enxame não vistas, sem testes de robustez a falhas e sem inferência estatística. É este vazio — medido, e não apenas alegado — que o presente trabalho preenche: um *benchmark* cruzado sob protocolo idêntico, que permite determinar quando o custo computacional da aprendizagem compensa a robustez da otimização, e que mostra, no processo, que a fronteira entre os dois paradigmas depende menos do algoritmo escolhido do que do desenho do sinal que o guia (Capítulo 6).

3.9. Síntese Comparativa da Literatura

A Tabela 3.1 sintetiza os estudos — do corpo da revisão e do seu enquadramento complementar, com as entradas externas assinaladas — que melhor delimitam a lacuna, organizados pelo paradigma que empregam.

TABELA 3.1. Síntese dos estudos mais relevantes do corpo da revisão ($n = 58$) e do seu enquadramento complementar (entradas externas ao corpus assinaladas), com as lacunas que motivam esta dissertação.

Referência	Paradigma	Contributo principal	Limitação / lacuna
Iskandar et al. (2024) [30]	Comparativo (RL vs. PSO)	Único estudo do corpo a confrontar os dois paradigmas na navegação de enxames.	Uma tarefa (vários ambientes); sem <i>Zero-Shot</i> , sem testes de falha, sem inferência estatística.
Hasselmann et al. (2023) [31]	Bio-inspirado (<i>novelty search</i>)	Repertórios de comportamentos gerados por procura por novidade para desenho automático de enxames.	Não confronta o desempenho com controladores treinados por gradiente.
Ordáz-Rivas et al. (2026) [36]	Bio-inspirado (multiobjetivo)	Otimização multiobjetivo de <i>foraging</i> com enxames (recolha vs. energia).	Sem <i>baseline</i> de aprendizagem; sem cenários decetivos.
Bredeche & Fontbonne (2022) [37]	Bio-inspirado (evolução <i>online</i>)	Aprendizagem social distribuída: os controladores propagam-se entre robôs durante a missão.	Foco no mecanismo evolutivo; sem comparação com MARL.
Chen et al. (2024) [15]	MARL (GNN) (<i>fora do corpus: redes sem fios</i>)	Estabelece o mecanismo da invariância: n^o de parâmetros independente do n^o de agentes $\Rightarrow$ transferência sem retreino.	Domínio não robótico (redes multi-salto); sem obstáculos físicos nem <i>baseline</i> evolutiva.
Zhang et al. (2022) [25]	MARL (GNN) (<i>fora do corpus: excluído na elegibilidade, E8</i>)	GNNs hierárquicas para exploração com vizinhanças de dimensão variável.	Avaliado só contra métodos de aprendizagem; escalabilidade não isolada do otimizador.
Tang et al. (2023) [39]	MARL (campo médio)	Custo de coordenação independente de N por aproximação de campo médio.	Sem comparação com otimização bio-inspirada; sem testes de robustez a falhas.
Kegeleirs et al. (2024) [32]	Bio-inspirado (<i>sim-to-real</i>)	Quantifica a transferibilidade do desenho automático entre simulação e plataformas reais.	Não avalia desempenho de tarefa face a MARL.
Yigit et al. (2026) [29]	MARL (PPO) (<i>fora do corpus: não recuperado pelas cadeias</i>)	PPO descentralizado com <i>parameter sharing</i> em navegação de enxame; valida o <i>baseline</i> aqui adotado.	Vizinhança reduzida ao agente mais próximo; N fixo (escalabilidade não avaliada); compara-se apenas com políticas aleatórias.

A leitura da tabela expõe o padrão: cada estudo é sólido *dentro* do seu paradigma e mede-se contra os seus pares — os trabalhos de MARL comparam-se com MARL,

os evolutivos com evolutivos. A coluna das limitações repete, com variações, a mesma frase: *não confronta o outro paradigma*. É precisamente esse confronto, sob protocolo comum e nas três dimensões que a literatura trata isoladamente (desempenho de tarefa, escalabilidade e robustez), que constitui o objeto do capítulo seguinte.

Desenvolvimento do Ambiente de Simulação

4.1. Arquitetura de Software e Stack Tecnológica

Para assegurar a viabilidade técnica e a reprodutibilidade dos resultados, a implementação do sistema foi desenvolvida com base nas tecnologias padrão da indústria para Inteligência Artificial. A arquitetura adotada privilegia a modularidade, a flexibilidade e o desempenho computacional, permitindo a substituição independente de qualquer componente sem comprometer a integridade do *pipeline* experimental.

4.1.1. Ambiente de Simulação e Interação

O desenvolvimento foi centralizado na linguagem **Python 3.x**, escolhida pela sua vasta comunidade científica e pelo ecossistema de bibliotecas especializadas em aprendizagem automática. A interface do simulador segue o estilo da **API *parallel* do PettingZoo** — um padrão aberto para ambientes de agentes múltiplos —, devolvendo a cada passo dicionários de observações, recompensas e sinais de terminação indexados por agente. Esta convenção abstrai a física interna e facilita a troca entre algoritmos sem reescrever o código base do ambiente [13].

A observação de cada agente é estritamente **local e parcial**, garantindo que o sistema opera num regime verdadeiramente descentralizado. Não existe qualquer componente com acesso ao estado global durante a fase de execução, replicando fielmente as limitações sensoriais de plataformas físicas como o Khepera IV ou o e-puck.

4.1.2. Implementação dos Algoritmos

A construção das redes neuronais e o treino dos agentes MARL apoiaram-se em *frameworks* de computação tensorial de alto desempenho:

- **PyTorch:** Utilizado para a implementação da *Graph Neural Network* (GNN) do algoritmo evolutivo, devido ao seu suporte nativo para grafos dinâmicos e à facilidade de depuração de operações tensoriais personalizadas.
- **Stable-Baselines3:** Biblioteca de referência para a implementação dos algoritmos PPO e SAC, fornecendo implementações estáveis e bem validadas pela comunidade científica, com suporte a ambientes vetorizados paralelos.
- **Paralelização Multiprocesso (CPU):** A aceleração do treino assenta na execução paralela de múltiplos ambientes de simulação em processos independentes — `SubprocVecEnv` para o PPO e o SAC e `multiprocessing.Pool` para a avaliação da população evolutiva —, distribuindo a carga pelos vários núcleos de CPU dos servidores de cálculo. O simulador é implementado em Python/NumPy e

executa em CPU; as redes envolvidas (a MLP [256, 256] da *Stable-Baselines3* e a *GNNAgent3D* de $\approx 8k$ parâmetros) são suficientemente compactas para não exigirem GPU dedicada, sendo o *throughput* de treino dominado pela simulação física do ambiente e não pela passagem pelas redes.

4.1.3. Arquitetura Geral do Simulador

A Figura 4.1 ilustra o fluxo de dados em ciclo fechado do simulador, onde cada agente iterativamente recolhe observações e executa ações sob a validação de um motor de física contínuo.

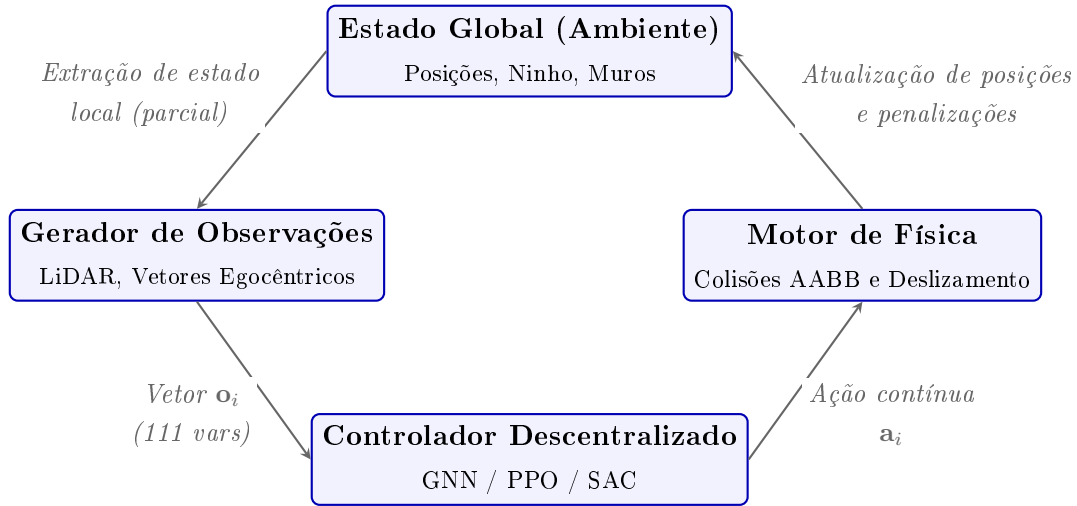


FIGURA 4.1. Diagrama da arquitetura do simulador e fluxo de interação agente-ambiente.

4.2. O Ambiente de Simulação

O simulador foi desenvolvido de raiz em **Python**, projetado para colmatar o *Deployment Gap* (fosso de implantação) através da modelação de alta fidelidade da física dos agentes, das restrições sensoriais e das perturbações dinâmicas do ambiente [14]. Todos os algoritmos são avaliados no mesmo ambiente, garantindo a paridade da comparação.

4.2.1. Modelo Físico e Hiperparâmetros

A arena de simulação consiste numa área circular de raio $r_{arena} = 15\text{m}$, onde $N = 20$ agentes homogêneos operam em simultâneo. A Tabela 4.1 sintetiza os hiperparâmetros físicos e arquiteturais aplicados.

TABELA 4.1. Hiperparâmetros reais utilizados na simulação e no treino dos modelos.

Categoria	Hiperparâmetro	Valor Real
Ambiente	Raio da Arena (r_{arena})	15.0 m
	Número de Agentes (N)	20
	Número de Obstáculos Dinâmicos	100
	Raio dos Obstáculos (r_{obs})	0.2 m
	Velocidade dos Obstáculos (v_{obs})	0.02 m/s
	Raio do Ninho (r_{nest})	1.5 m
	Velocidade do Ninho (v_{nest})	0.015 m/s
	Limite do Temporizador de Fome ($hunger$)	250 passos (só Sandbox)
	Cooperação Mínima para Alimentar (k_{min})	3 ag. (1 nos labirintos)
	Alcance do LiDAR	8.0 m
	Fator de Progresso (PBRs)	10.0
	Resolução do Campo Geodésico	0.4 m
PPO	Taxa de Aprendizagem (lr)	10^{-4}
	Passos por Rollout (n_{steps})	64 (por CPU)
	Batch Size	512
	Número de Épocas (n_{epochs})	4
	Arquitetura da Rede	MLP [256, 256]
	Número de CPUs Paralelos	16
SAC	Buffer de Replay	500,000 transições
	Coefficiente de Entropia (α)	0.1
	Taxa de Aprendizagem (lr)	10^{-4}
	Arquitetura da Rede	MLP [256, 256]
Evolutivo (AE)	Tamanho da População (K)	30 genomas
	Taxa de Mutação (p_{mut})	0.1
	Desvio Padrão Inicial (σ)	0.1 ($\times 0.999/\text{ger.}$; $\sigma_{\min} = 0.03$)
	Episódios de Avaliação por Genoma (E)	4
	Dimensão Oculta da Rede (h)	32 ($\approx 8k$ pesos)

4.2.2. Espaço de Observação

As observações de cada agente são estritamente locais, parciais e expressas num referencial **egocêntrico** (relativo à orientação atual do próprio robô). O vetor de observação tem dimensão $d_{obs} = 16 + (N - 1) \times 5 = 111$ (para $N = 20$ agentes) e está estruturado em quatro componentes:

$$\mathbf{o}_i = [\mathbf{v}_{nest,i}, d_{nest,i}, \mathbf{l}_i, \mathbf{v}_{porta,i}, d_{porta,i}, \mathbf{n}_{1,i}, \dots, \mathbf{n}_{19,i}] \in \mathbb{R}^{111} \quad (4.1)$$

- (1) **Direção e Distância ao Ninho (4 valores):** Projeção do vetor direcional unitário do ninho nos eixos egocêntricos Frontal (F), Direito (R) e Superior (U) do

robô, mais a distância normalizada ($d/2r_{arena}$). Esta informação de **homing** está sempre disponível, mesmo quando existem muros entre o agente e o ninho. Este pressuposto é padrão em robótica de enxame — assume-se um sentido global de orientação para a base (análogo à bússola magnética dos pombos, à orientação solar das formigas ou ao GPS de drones), mas **nunca** um mapa prévio dos obstáculos. A parcialidade da observação é assim mantida ao nível dos obstáculos (detetáveis apenas localmente pelo LiDAR), e não ao nível do objetivo. Esta opção corrige uma limitação de uma versão anterior, em que a direção era anulada na ausência de linha de visão, o que cegava os agentes em todos os cenários com paredes e os impedia de aprender a contornar.

- (2) **LiDAR Horizontal (8 valores, vetor $\mathbf{l}_i$)**: Oito raios medidos em plano horizontal, uniformemente distribuídos a 360° , com alcance máximo de 8 m. O valor devolvido é $1 - (d_{hit}/d_{max})$, onde 1.0 significa impacto iminente. O alcance foi dimensionado para que o contorno dos grandes obstáculos dos labirintos (p.ex. as pernas do Muro em U, a ≈ 8 m da rota direta) seja detetável a tempo de iniciar o desvio, preservando ainda assim a observabilidade parcial: o agente não vê o mapa completo nem a direção geodésica.
- (3) **Direção e Distância à Porta/Entrada (4 valores, $\mathbf{v}_{porta,i}$ e $d_{porta,i}$)**: Direção egocêntrica e distância normalizada à porta-objetivo do cenário, preenchidas **apenas** nos cenários com uma porta física definida (Porta Cooperativa e Porta Cooperativa com Alternativa); nos restantes são nulas. A porta é um sub-objetivo explícito do cenário — tal como o ninho —, pelo que a indicar não constitui revelar o trajeto. Distingue-se assim de fornecer *waypoints* das passagens dos labirintos, o que equivaleria a um planeador externo e quebraria a observabilidade parcial.
- (4) **Vetor de Vizinhos ($19 \times 5 = 95$ valores, vetores $\mathbf{n}_{j,i}$)**: Para os restantes 19 agentes, envia-se o vetor de direção local (3 valores), distância normalizada (1 valor) e um sinal binário de comunicação (1 valor). Este último é ativado (1.0) quando o vizinho atinge o ninho e se encontra a repousar, servindo como mecanismo de recrutamento estigmérgico.

4.2.3. Espaço de Ação e Dinâmica do Passo de Simulação

Cada agente emite, a cada passo, uma ação contínua $\mathbf{a}_i = (a_F, a_R, a_U) \in [-1, 1]^3$ interpretada como um **deslocamento no referencial egocêntrico** do robô. O vetor é convertido para o referencial global através da base ortonormal $\{F, R, U\}$ (Frontal, Direito, Superior) e escalado por um passo máximo $\delta_{max} = 0.2$ m:

$$\Delta \mathbf{p}_i = 0.2 \cdot (a_F \mathbf{F}_i + a_R \mathbf{R}_i + a_U \mathbf{U}_i) \quad (4.2)$$

A orientação $\mathbf{F}_i$ é atualizada para a direção do movimento sempre que $\|\Delta \mathbf{p}_i\| > 0.02$, garantindo coerência entre a pose visual e a trajetória.

O passo de simulação (**step**) processa o enxame por uma ordem determinística que assegura a consistência física:

- (1) **Perturbações dinâmicas:** movimento dos obstáculos e do alvo móvel; injeção de falhas de agentes (Secção 6.12) quando aplicável.
- (2) **Aplicação da ação com *wall-sliding*:** cada deslocamento é projetado ortogonalmente à normal de qualquer muro intersetado, permitindo deslizamento em vez de paragem abrupta.
- (3) **Resolução de colisões:** reposicionamento por penetração contra obstáculos (distância euclidiana), muros (AABB) e fronteira circular da arena.
- (4) **Separação física:** repulsão geométrica de pares de agentes sobrepostos (a penalização *anti-clustering* associada está desativada na formulação final da recompensa, Secção 4.3; a resolução física mantém-se sempre ativa).
- (5) **Lógica de tarefa:** verificação de ocupação do ninho (k_{min} agentes, consoante o cenário), cerco do alvo móvel (360°) ou abertura da porta cooperativa.
- (6) **Atribuição de recompensa e terminação:** cálculo dos termos de $\mathcal{R}$ (Secção 4.3) e verificação do horizonte temporal ($t \geq t_{max}$).

4.2.4. Cenários de Teste

Para garantir uma avaliação exaustiva e progressiva das capacidades de controlo descentralizado, foram desenvolvidos sete cenários de dificuldade crescente (Figura 4.2):

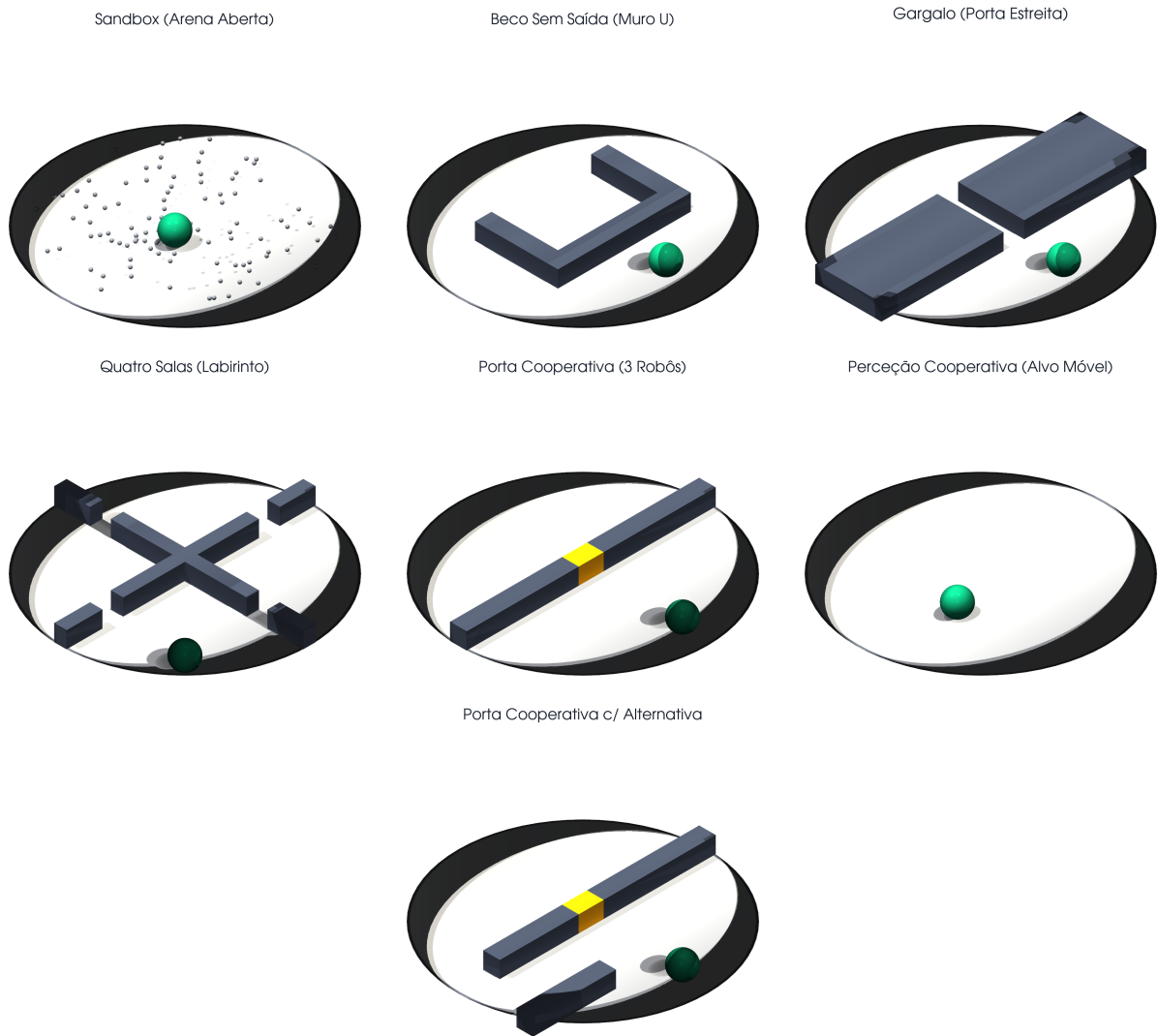


FIGURA 4.2. Renderizações 3D dos sete cenários de teste, geradas a partir da geometria real do simulador (`scripts/render_maps.py`). Em cima, da esquerda para a direita: Sandbox, Beco Sem Saída (Muro em U) e Gargalo; ao centro: Quatro Salas, Porta Cooperativa (com a porta destacada a âmbar) e Percepção Cooperativa; em baixo: Porta Cooperativa com Alternativa (*bypass*). A esfera verde assinala o ninho (ou o alvo móvel, na Percepção Cooperativa).

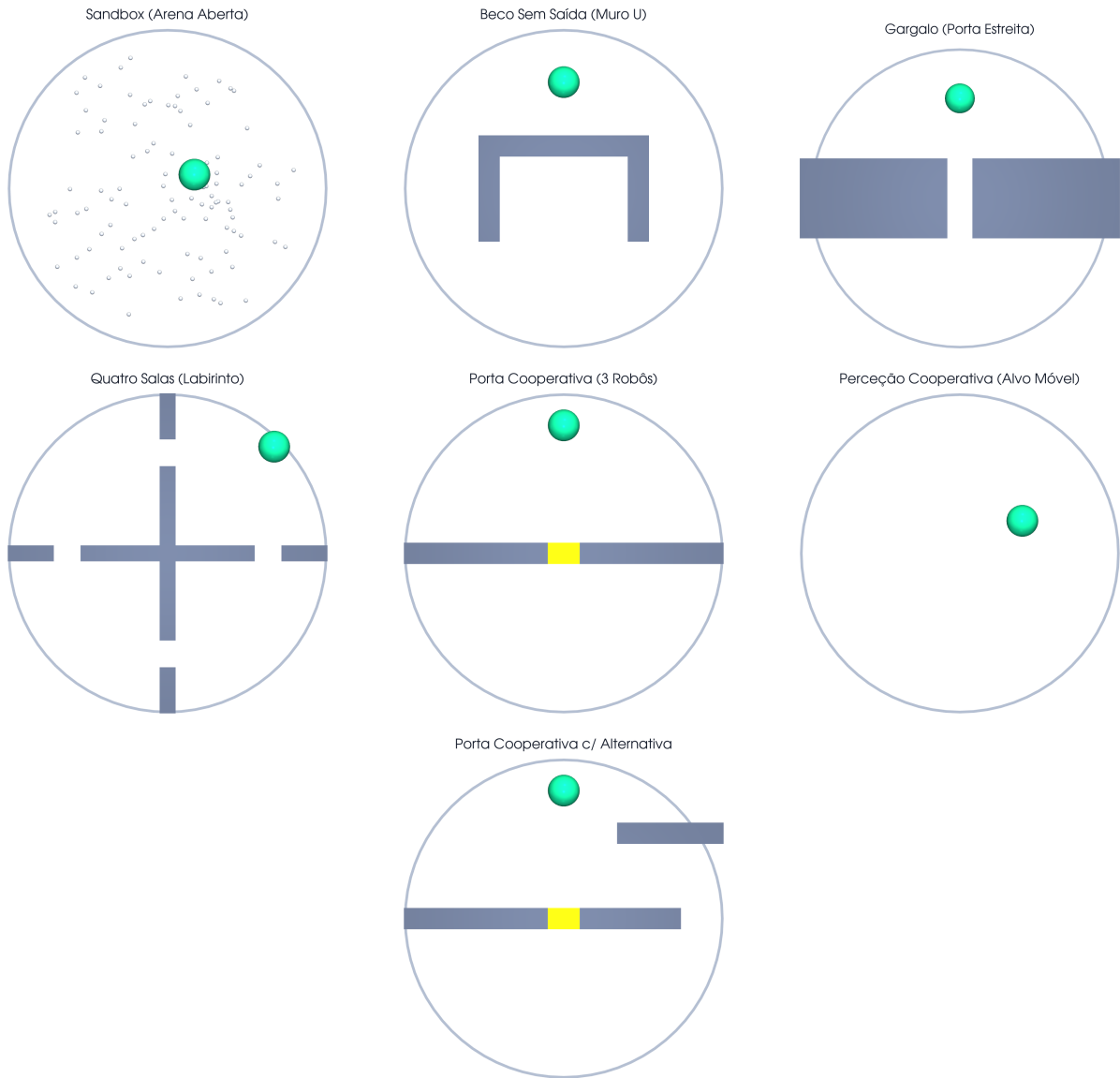


FIGURA 4.3. Vista de topo (planta) dos sete cenários, evidenciando a geometria das paredes e a largura das passagens. Mesma ordem da Figura 4.2: em cima, Sandbox, Muro em U e Gargalo; ao centro, Quatro Salas, Porta Cooperativa e Percepção Cooperativa; em baixo, Porta Cooperativa com Alternativa.

- (1) **Foraging Aberto (Sandbox):** Cenário base sem barreiras fixas. Contém os agentes, obstáculos dinâmicos e um ninho móvel. Serve para aferir a capacidade pura de cooperação e gestão de energia sem entropia espacial acrescida.
- (2) **Beco Sem Saída (Muro em U):** Um grande obstáculo em forma de "U" bloqueia a linha direta ao ninho. A parede superior tem 14m e as pernas laterais têm 10m. Os agentes começam do lado de fora da "taça" e têm de explorar lateralmente as aberturas de 7m. Avalia a exploração prolongada em detrimento da navegação gananciosa.

- (3) **Gargalo (Porta Estreita):** A arena é cortada a meio por uma parede contínua, deixando apenas uma passagem de 2.5 m de largura. Obriga a um alinhamento e gestão do fluxo do enxame, punindo gravemente o congestionamento desenfreado.
- (4) **Quatro Salas (Labirinto):** Duas grandes paredes cruzam a arena vertical e horizontalmente, criando quatro salas isoladas ligadas apenas por aberturas de 2.5 m de largura. Os agentes começam no extremo oposto ao ninho, exigindo memória temporal e navegação estruturada de longo termo.
- (5) **Porta Cooperativa:** Similar ao Gargalo, mas a passagem de 3 m de largura encontra-se bloqueada por um portão físico. A porta só abre (deslizando para fora do caminho, com realce visual no *dashboard*) quando no mínimo 3 agentes ocupam a zona de pressão em simultâneo. Por exigir uma sincronização mais demorada, este cenário dispõe de um horizonte temporal alargado de 800 passos (face aos 500 dos restantes), dando margem para que o enxame coordene a abertura e ainda alcance o ninho. Foca-se em comportamentos de sincronização sem comunicação explícita.
- (6) **Perceção Cooperativa (Alvo Móvel):** A noção de "ninho físico" desaparece. O alvo de interesse viaja livremente pelo mapa ao dobro da velocidade normal (0.03 m/s). Só é considerado capturado quando 3 agentes se aproximam a menos de 4 m de distância e distribuem-se num raio de 360° em redor do mesmo, testando manobras de cerco e perseguição.
- (7) **Porta Cooperativa com Alternativa (Bypass):** Variante da Porta Cooperativa desenhada como problema de *recompensa enganadora (deceptive)*. A barreira horizontal mantém a porta central de 3 m (que continua a exigir 3 agentes na zona de pressão), mas o segmento direito da barreira termina antes da fronteira da arena, deixando uma abertura permanentemente livre de 4 m junto à parede leste — um percurso alternativo que dispensa cooperação. Uma parede defletora, colocada mais a norte no mesmo lado, obriga quem usa a alternativa a afastar-se temporariamente do ninho, tornando o percurso alternativo cerca de duas vezes mais longo do que a passagem pela porta. A rota mais curta atrai, assim, o enxame para a porta (que exige coordenação), enquanto a descoberta da alternativa exige contrariar o gradiente de aproximação ao objetivo — a propriedade que faz deste o cenário de referência para a QI6 (procura por novidade, Secção 1.4). Pelo horizonte de exploração acrescido, o limite temporal é alargado para 1000 passos.

4.3. Design da Função de Recompensa

A função de recompensa é o instrumento de desenho experimental mais sensível do sistema: as primeiras campanhas de treino acumularam termos secundários (penalizações de dispersão, de colisão entre agentes e de contacto com obstáculos) que, na prática, diluíam o sinal da tarefa e abriam portas a comportamentos degenerados. A formulação final adotou deliberadamente uma **recompensa simples**, com o número mínimo de termos

e dois focos — *explorar o mapa* e *encontrar o ninho* — em que a recompensa de tarefa domina claramente todos os sinais de *shaping*. As penalizações secundárias foram removidas *apenas do sinal de treino*: a resolução física correspondente mantém-se no motor (os agentes continuam a não se sobrepor nem a atravessar paredes, Secção 4.4.2), pelo que a simplificação não altera a dinâmica do mundo, apenas o que é ensinado.

$$\mathcal{R}(s, \mathbf{a}) = R_{\text{tarefa}} + R_{\text{progresso}} + R_{\text{exploração}} - R_{\text{controlo}} \quad (4.3)$$

Os valores exatos utilizados na campanha experimental final são os seguintes:

- **Tarefa** (R_{tarefa}): Recompensa dominante de +300 por recolha, atribuída (e *repetível* ao longo do episódio) a cada agente que participa numa recolha bem sucedida. O número de agentes exigidos em simultâneo no ninho ($k_{\min}$) é definido **por cenário**: $k_{\min} = 3$ nos cenários em que a cooperação é o objeto de estudo (Sandbox, Porta Cooperativa, Perceção Cooperativa e *Bypass*), mas $k_{\min} = 1$ nos labirintos de navegação pura (Muro em U, Gargalo e Quatro Salas) — exigir três agentes simultâneos nesses cenários transformava um problema de navegação numa cooperação artificial, registando zero recolhas mesmo quando um ou dois agentes alcançavam o ninho. Nos cenários com porta cooperativa acresce um bónus único de +100 a cada agente que participa na abertura da porta.
- **Progresso** ($R_{\text{progresso}}$): Baseado no método *Potential-Based Reward Shaping* (PBRS) [43], atribui $10.0 \cdot (\Phi_{t-1} - \Phi_t)$, onde Φ é o **potencial de navegação** até ao ninho. Nos cenários com paredes (Muro U, Gargalo, Quatro Salas e Porta Cooperativa) este potencial é a **distância geodésica** — o comprimento do caminho mais curto que contorna os obstáculos, calculado por uma pesquisa de custo uniforme (Dijkstra, 8-conexa) sobre uma grelha de discretização (0.4 m, com as paredes dilatadas pelo raio do robô) — e não a distância euclidiana. Esta escolha elimina na raiz um mínimo local clássico: com a distância euclidiana, contornar uma parede *afasta* o agente do ninho e é penalizado, prendendo o enxame contra os obstáculos; com a distância geodésica, qualquer passo ao longo do caminho viável é recompensado. Por se tratar de PBRS, não altera a política ótima do cenário [43]. Decisivamente, o campo geodésico é um **sinal de treino** (a geometria é conhecida pelo simulador apenas durante o treino) e **não** uma observação: em execução o agente dispõe somente da bússola de *homing* euclidiana e do LiDAR local (Secção 4.2.2), preservando a observabilidade parcial. Distingue-se assim, claramente, de fornecer ao agente um mapa dos obstáculos ou *waypoints* de um planeador A*.
- **Exploração** ($R_{\text{exploração}}$): Bónus de exploração *count-based* que atribui +0.5 na primeira vez que um agente visita cada célula de uma grelha de discretização espacial (2×2 m), em linha com a necessidade de mecanismos densos de exploração identificada por Xiao et al. (2026) [44]. O valor é **pequeno de propósito**: incentiva a cobertura do mapa em regiões onde o gradiente do potencial é pouco

informativo, mas é dominado por larga margem pela recompensa de tarefa (+300, repetível). Iterações anteriores com um bônus maior demonstraram o risco desta calibração: a política aprendia a *vaguear* para colher exploração em vez de convergir para a recolha — um caso de *reward hacking* que motivou a simplificação da recompensa aqui descrita, e cujo análogo evolutivo (*fitness exploitation*) se analisa na Secção 6.14.

- **Controlo Energético (R_{controlo}):** Subtração de -0.05 a cada passo de simulação por agente não inativo. Pressiona temporalmente o enxame para finalizar o forrageamento o mais célere possível; nos cenários de objetivo fixo é a única fonte de pressão temporal.
- **Fome (apenas no Sandbox):** Caso o temporizador de *hunger* de um agente ultrapasse 250 passos, o agente “morre”, sofre um revés de -50.0 e a sua posição é restabelecida no ponto de partida. Esta reposição está **ativa apenas no forrageamento contínuo** (cenário Sandbox); nos cenários de objetivo fixo foi desativada, pois a combinação do teletransporte com a re-sincronização do potencial tornava o termo de progresso *farmável* (aproximar-se, ser repostado longe sem cobrança do recuo, repetir), induzindo políticas degeneradas que acumulavam recompensa de *shaping* sem nunca cumprir a tarefa.
- **Penalizações secundárias (desativadas na formulação final):** As versões preliminares incluíam penalizações de *anti-clustering* (proximidade entre agentes), de colisão agente-agente e de contacto com obstáculos. Na formulação final, os respetivos pesos foram colocados a zero: a separação física e o *push-out* continuam a ser *resolvidos* pelo motor (Secção 4.4.2), mas deixaram de gerar sinal de recompensa, eliminando três termos cuja calibração relativa baralhava o sinal da tarefa. Definiu-se como salvaguarda experimental que, caso surgissem comportamentos degenerados (agregação excessiva ou agentes “colados” às paredes), os termos seriam reintroduzidos gradualmente — o que não se revelou necessário.

4.4. Desafios de Engenharia e Otimização Computacional

A construção de um simulador de raiz capaz de suportar o treino de algoritmos de aprendizagem por reforço profundo exige um rigoroso esforço de engenharia de software. O ambiente desenvolvido (`swarm_env_3d.py` e os visualizadores 3D associados) foi desenhado para mitigar estrangulamentos computacionais comuns em arquiteturas *Multi-Agent*, focando-se na otimização matemática e abstração arquitetural.

4.4.1. Motor de Visualização Interativa (Dashboard)

Para permitir a inspeção do comportamento emergente do enxame sem comprometer o desempenho do treino, o sistema adota um padrão de arquitetura que desacopla rigorosamente a lógica de simulação física do motor de renderização gráfica. A inspeção do comportamento é feita por um visualizador 3D implementado com o motor **Ursina** (assente

em Panda3D), que projeta a telemetria do motor contínuo interno num espaço tridimensional navegável, com representação do ninho, obstáculos móveis, muros e indicadores de proximidade. O controlo experimental — lançamento de treinos, seleção de cenários e geração de gráficos — é centralizado num *dashboard* interativo construído em **NiceGUI** (aplicação *web* local). Esta separação permite que o simulador opere num modo *headless* (sem sobrecarga gráfica) durante sessões de treino intensivo — processando milhares de transições por segundo —, enquanto oferece um modo interativo para demonstração e análise qualitativa das políticas aprendidas.

4.4.2. Otimização Matemática da Física e Detecção de Colisões

A viabilidade do RL depende de ciclos de simulação contínuos em ordem de milissegundos. Simular colisões dinâmicas para $N = 20$ agentes e uma centena de obstáculos não-estáticos de forma ineficiente levaria a um aumento quadrático no custo computacional.

- **Detecção Geométrica de Colisões:** O motor resolve as colisões por testes geométricos diretos — distância Euclidiana para os obstáculos esféricos e caixas envolventes alinhadas aos eixos (*Axis-Aligned Bounding Boxes*, AABB) para os muros retangulares —, recorrendo a operações vetoriais **NumPy** no cálculo de distâncias e profundidades de penetração. Um passo dedicado de **separação física** repele geometricamente qualquer par de agentes sobrepostos, assegurando a ausência de interpenetrações não-físicas no enxame.
- **Wall-Sliding Cinemático:** Quando um robô entra em contacto com uma parede retangular, o motor não o imobiliza mecanicamente (o que penalizaria irreversivelmente a exploração do algoritmo MARL). Em vez disso, recorre à projeção ortogonal do vetor de velocidade ao longo da direção normal da superfície de colisão, reproduzindo matematicamente de forma realista o deslizamento (*wall-sliding*) contínuo de um chassis móvel *differential-drive*.

4.4.3. Mecanismo de Atenção sobre Vizinhos (GNN)

O extrator de relações entre robôs foi implementado de raiz em **PyTorch puro** (sem recurso a bibliotecas de grafos como o PyTorch Geometric), através de um mecanismo de **atenção de produto interno escalado** sobre o conjunto de vizinhos, conforme formalizado nas Equações 2.18–2.20.

- **Representação Densa da Vizinhança:** A cada passo, cada agente i observa os restantes $N - 1$ agentes através de um vetor de *features* de dimensão fixa (direção egocêntrica, distância normalizada e sinal de comunicação por vizinho). Diferentemente de uma abordagem baseada em arestas esparsas com raio de comunicação fixo, todos os vizinhos são apresentados ao módulo de atenção, sendo a **relevância de cada um aprendida** pelos coeficientes α_{ij} — a rede atribui dinamicamente pesos próximos de zero aos vizinhos irrelevantes, dispensando um corte geométrico explícito.

- **Dimensão Fixa e *Parameter Sharing*:** Como a representação tem dimensão fixa ($\mathbb{R}^{111}$), a integração em *batches* de treino é direta, sem necessidade de *padding* dinâmico ou conversão para listas de índices. As projeções *Query-Key-Value* e o *softmax* operam sobre o número de vizinhos presentes, pelo que a mesma rede processa enxames de qualquer dimensão — viabilizando o *Zero-Shot Transfer* para $N \in \{10, 50, 100\}$ sem retreino.

4.4.4. Ambientes Vetorizados e Paralelização Multiprocesso

Por forma a escalar exponencialmente a recolha de experiência (nas instâncias SAC/PPO) e na avaliação paralela da neuroevolução:

- **Vectorized Environments:** O motor de ambiente Gym original foi encapsulado através dos *wrappers* `SubprocVecEnv` (fornecidos pela `*Stable-Baselines3*`), forçando a inicialização de dezenas de cópias do simulador correndo de forma isolada e simultânea através de múltiplos processos de CPU paralelos.
- **Mecanismo de Guilhotina (Early Stopping):** Este mecanismo heurístico otimizou criticamente o tempo de otimização estocástica evolutiva. Se as recompensas operacionais de um indivíduo descenderem subitamente até limiares proibitivos nos passos primários da simulação — fruto de uma política disfuncional gerada por mutação —, a execução contínua é descartada, transitando de imediato os recursos de ciclo de CPU do sistema para o processamento de novos indivíduos concorrentes na mesma geração.

4.5. Estrutura do Projeto e Extensibilidade

Um dos objetivos declarados desta dissertação é entregar uma *implementação de referência* reutilizável (Secção 7.4). Essa ambição obriga a tratar a estrutura do software como resultado, e não como detalhe de implementação: importa dizer, com precisão, o que custa a um terceiro estender este trabalho — e o que *não* foi desenhado para ser barato.

4.5.1. Organização dos Módulos

O repositório separa quatro responsabilidades, com dependências num só sentido (a simulação não conhece o treino, o treino não conhece a análise):

- `src/` (≈ 2300 linhas) — o núcleo. Contém o ambiente (`environment/swarm_env_3d.py`: física, LiDAR, geodésicas, recompensa), os controladores (`agents/`, sob uma interface comum `Agent.get_action`), os treinadores (`training/`: um módulo por paradigma) e o **registo canónico de cenários** (`scenarios.py`).
- `scripts/` (≈ 4100 linhas) — a orquestração e a análise: `run_experiments.py` (campanhas), a bateria de avaliação (`eval_*.py`), os testes estatísticos e a geração de figuras.
- `dashboard/` (≈ 2600 linhas) — observabilidade: acompanhamento do treino, curvas ao vivo, matriz de resultados e ligação ao servidor remoto.
- `tests/` (≈ 800 linhas) — garantias de não-regressão, discutidas adiante.

4.5.2. Pontos de Extensão

Acrescentar um algoritmo é barato. Um novo controlador exige (i) uma classe que implemente `Agent.get_action`, (ii) um módulo de treino em `src/training/` e (iii) *uma entrada* no dicionário `ALGORITHMS` de `run_experiments.py`. A partir daí, o algoritmo é automaticamente incluído nas campanhas, na avaliação determinística, nos testes de significância e nas figuras — porque toda a cadeia a jusante itera sobre esse registo, e não sobre nomes escritos à mão. A comparação MADDPG ou QMIX que a Secção 7.5 propõe é, neste sentido, um exercício de dias e não de semanas.

Acrescentar uma métrica ou um teste é barato. As métricas de avaliação são calculadas num único ponto (`scripts/eval_suite.py`) e propagam-se por escrita em `results/evaluation/eval_summary.csv`, do qual dependem, por leitura, a análise estatística, os gráficos e o painel. Uma métrica nova torna-se visível em toda a cadeia sem alterar nenhum consumidor.

Acrescentar um cenário é barato *à superfície* e caro *no fundo*. Os *metadados* de cenário — lista canónica, rótulos longos e curtos, classificação como labirinto — residem num único ficheiro (`src/scenarios.py`), do qual todos os consumidores importam. Esta centralização não é acidental: resultou da correção de um defeito real, em que o sétimo cenário foi treinado mas nunca avaliado, por a lista de cenários estar duplicada entre os módulos de treino e de avaliação. Com a fonte única, um cenário novo aparece automaticamente nas campanhas, nas tabelas e nas figuras.

A *geometria*, porém, não goza da mesma disciplina: a construção de paredes, as posições de *spawn* e as regras específicas de cada cenário estão distribuídas por doze blocos condicionais (`if classic_scenario == ...`) dentro do ambiente. Acrescentar um cenário implica, portanto, tocar em vários pontos de um ficheiro extenso — um custo que cresce linearmente com o número de cenários e que constitui a dívida técnica mais visível deste projeto. A correção natural é um *registo de cenários* análogo ao dos algoritmos: cada cenário passaria a ser uma classe com uma interface fixa (`build_obstacles`, `spawn_positions`, `required_to_eat`), registada por decorador, e os doze condicionais colapsariam numa única invocação polimórfica. A refatoração é direta e está identificada; não foi executada por a campanha experimental estar em curso e qualquer alteração ao ambiente invalidar a comparabilidade dos resultados já obtidos — precisamente o tipo de compromisso entre pureza arquitetural e integridade experimental que uma dissertação empírica impõe.

4.5.3. Garantias de Não-Regressão

A extensibilidade só é real se o sistema puder ser alterado sem medo. Para esse efeito, as otimizações de desempenho mais arriscadas foram acompanhadas de **testes de equivalência bit-a-bit** contra a implementação de referência: a vetorização do LiDAR (adotada por multiplicar o *throughput* por ≈ 19) é confrontada com o ciclo explícito original em 8 000 cenas aleatórias — incluindo casos-limite como raios paralelos aos eixos e ausência de obstáculos —, exigindo-se igualdade *exata*; a vetorização das observações egocêntricas

($\approx 2,6\times$ no passo de simulação) é verificada do mesmo modo. A mecânica da porta cooperativa — o comportamento mais sutil do simulador — e o *cache* de elites do treinador evolutivo têm testes dedicados, este último assegurando que a otimização não altera a trajetória de treino. Estas verificações são o que permite afirmar que os ganhos de desempenho não alteraram a semântica do ambiente, e o que dará a um terceiro a confiança para modificar o núcleo sem invalidar silenciosamente os resultados publicados.

4.5.4. Reprodutibilidade

A configuração completa — física, recompensa, hiperparâmetros dos três algoritmos — reside num único ficheiro declarativo (`configs/foraging.yaml`), sem constantes dispersas pelo código. As sementes de avaliação são fixas, tornando a avaliação determinística e emparelhada entre algoritmos. Cada campanha regista, por execução, a curva de treino e o modelo campeão, e o repositório inclui ainda o protocolo e o registo integral da revisão sistemática (Capítulo 3). Um terceiro que clone o repositório e execute `run_experiments.py` com a configuração publicada obtém, com as mesmas sementes, os mesmos números — que é o critério mínimo para que uma comparação entre paradigmas possa ser contestada, e portanto levada a sério.

Arquitetura Algorítmica e Controladores

5.1. Algoritmo 1: Multi-Agent Reinforcement Learning (RS2C)

A primeira abordagem investigada fundamenta-se no *framework* **Robust and Scalable Swarm Control (RS2C)**. Este adota um esquema de **execução descentralizada com partilha de parâmetros** (*parameter sharing*) — em que todos os agentes partilham uma única política que atua sobre observações estritamente locais —, formulando a coordenação do enxame como um Processo de Decisão de Markov Parcialmente Observável Cooperativo (Dec-POMDP):

$$\langle \mathcal{N}, \mathcal{S}, \{\mathcal{A}_i\}_{i \in \mathcal{N}}, \mathcal{P}, \mathcal{R}, \Omega, \{\mathcal{O}_i\}_{i \in \mathcal{N}}, \gamma \rangle \quad (5.1)$$

Sob o RS2C investigam-se dois algoritmos de aprendizagem por reforço profundo padrão: **PPO** (*on-policy*) e **SAC** (*off-policy*). Em ambos, os N agentes partilham uma única política (*parameter sharing*) que recebe a observação local de dimensão fixa $\mathbf{o}_i \in \mathbb{R}^{111}$ — a qual já codifica, em formato *dense*, as *features* de toda a vizinhança (Secção 4.2.2). Na implementação atual, esta política partilhada é uma rede totalmente ligada (MLP [256, 256], da *Stable-Baselines3*); o **mecanismo de atenção sobre grafo** formalizado no Capítulo 2 é instanciado no controlador neuroevolutivo (Secção 5.2), sendo a base da invariância à permutação e da capacidade de *Zero-Shot Transfer* aí avaliadas. A integração direta do módulo de atenção numa política treinável por gradiente constitui uma extensão natural deixada para trabalho futuro (Secção 7.3).

5.1.1. Configuração PPO (*On-Policy*)

No modo PPO, os pesos da política MLP partilhada são otimizados através de gradientes calculados sobre *rollouts* de experiência recolhidos com a política atual. O *pipeline* de treino opera da seguinte forma:

- (1) **Recolha de Experiência:** Em cada iteração, $N_{arenas} = 16$ cópias paralelas do simulador (`SubprocVecEnv`) geram transições durante $n_{steps} = 64$ passos cada. Como os $N = 20$ agentes de cada arena são tratados como ambientes independentes (*parameter sharing*), o *rollout buffer* acumula $16 \times 20 \times 64 = 20\,480$ transições por iteração.
- (2) **Cálculo da Vantagem:** As vantagens $\hat{A}_t$ são estimadas via GAE (Equação 2.11) com $\gamma = 0.99$ e $\lambda = 0.95$ sobre o *rollout* completo.
- (3) **Atualização:** O buffer é amostrado em *mini-batches* de 512 transições e o gradiente de L^{PPO} é calculado durante $n_{epochs} = 4$ passagens completas.

- (4) **Parameter Sharing:** Os $N = 20$ robôs partilham o mesmo conjunto de parâmetros θ . Cada transição (o_i^t, a_i^t, r_t) de qualquer agente contribui para o gradiente, aumentando efetivamente o tamanho do *batch* por fator N .

5.1.2. Configuração SAC (*Off-Policy*)

O SAC opera em modo *off-policy*: as experiências acumulam-se assincronamente num **Replay Buffer** de capacidade $|\mathcal{D}| = 500\,000$ transições. Para $N = 20$ agentes, cada passo de simulação contribui 20 transições, preenchendo o buffer rapidamente. O SAC mantém dois críticos independentes Q_{ψ_1} e Q_{ψ_2} (*double Q-trick*) e a cada passo do ambiente uma mini-batch de 256 transições é amostrada aleatoriamente para atualizar os parâmetros. O coeficiente de temperatura $\alpha = 0.1$ é mantido constante, sem adaptação automática.

5.1.3. Representação da Vizinhança e *Parameter Sharing*

A cada passo de simulação, o vetor de observação de cada agente é construído em `swarm_env_3d.py`:

- (1) **Vizinhança Completa:** As *features* de todos os restantes $N - 1$ agentes (direção egocêntrica, distância normalizada e sinal de comunicação) são incluídas no vetor de observação $\mathbf{o}_i$. Não é aplicado um corte por raio de comunicação fixo: a seleção dos vizinhos relevantes é delegada no mecanismo de atenção, que aprende os pesos α_{ij} .
- (2) **Representação *Dense* de Dimensão Fixa:** O vetor $\mathbf{o}_i \in \mathbb{R}^{111}$ incorpora os $N - 1$ vizinhos em formato *dense*, preservando uma dimensão de entrada constante e dispensando estruturas esparsas (*edge-index*) ou *padding* dinâmico.
- (3) **Dimensão de Entrada Fixa (PPO/SAC):** A política MLP partilhada recebe um vetor de dimensão **fixa** $\mathbb{R}^{111}$, definida para $N = 20$ agentes. Por construção, esta abordagem **não é invariante** ao tamanho do enxame: alterar N modifica a dimensão da observação $(16 + (N - 1) \times 5)$ e inviabiliza a aplicação direta da rede. A invariância à permutação e o *Zero-Shot Transfer* para $N \in \{10, 50, 100\}$ são propriedades do mecanismo de atenção (Equações 2.18 e 2.19), cujas operações QKV e *softmax* se definem sobre o número de vizinhos presentes; são, por isso, avaliados no controlador neuroevolutivo (Secção 5.2).

5.2. Algoritmo 2: Otimização Bio-inspirada (Benchmark)

A *baseline* adotada assenta num processo de **Neuroevolução** [9], utilizando um Algoritmo Evolutivo clássico com elitismo (preservação dos 20% melhores) e mutações Gaussianas independentes nos pesos ($p_{mut} = 0.1$, com atenuação *sigma decay*). É neste controlador que se instancia a **rede de grafos com atenção** (GNNAgent3D) formalizada no Capítulo 2: é precisamente esta arquitetura, invariante ao número de agentes, que habilita o *Zero-Shot Transfer*. A otimização evolutiva *offline* ajusta os seus pesos sobre 30 genomas candidatos, sem recurso a gradientes nem a um sinal de recompensa denso por passo.

5.2.1. Ciclo de Treino Evolutivo

O treino evolutivo segue o ciclo *avaliação* $\rightarrow$ *seleção* $\rightarrow$ *mutação*, implementado em `evo_trainer_3d.py`:

- (1) **Avaliação Paralela:** Os $K = 30$ genomas são avaliados em paralelo, com um processo por genoma, via `multiprocessing.Pool` (o número de processos é o mínimo entre o tamanho da população e o número de núcleos disponíveis; nos servidores de cálculo utilizados, com 64 núcleos, os 30 genomas são avaliados simultaneamente). Cada processo executa $E = 4$ episódios independentes (com *seeds* fixas ao longo da evolução) e devolve $J(\theta_k)$ (Equação 2.1).
- (2) **Mecanismo de Guilhotina:** Se a recompensa acumulada após 150 passos for inferior ao limiar $J < -200$, o episódio é terminado prematuramente e o genoma recebe uma penalidade adicional de -1000 . Este mecanismo evita desperdiçar ciclos de CPU em genomas claramente disfuncionais.
- (3) **SeleçãoELITista:** Os $e = 6$ melhores genomas são copiados intactos. Os restantes 24 são gerados por mutação element-wise (Equação 2.2) de cópias aleatórias dos elites.
- (4) **Anilamento de σ :** A cada geração, σ é atualizado pela Equação 2.3, com decaimento de 0.1% por geração e patamar mínimo $\sigma_{\min} = 0.03$.
- (5) **Critério de Paragem:** O treino prossegue até esgotar o orçamento temporal configurado (195 minutos por execução na campanha final), garantindo duração determinística e comparável com os algoritmos MARL.

5.2.2. Procura por Novidade (*Novelty Search*) para Cenários *Deceptive*

O *shaping* de *homing* da Equação 2.1 pressupõe que descer o potencial de navegação conduz à solução. O cenário Porta Cooperativa com Alternativa (Figura 4.3) viola deliberadamente este pressuposto: o campo geodésico trata a porta como transponível, pelo que o gradiente de *homing* conduz o enxame diretamente à porta *fechada* — um ótimo local de que a evolução objetiva só escapa de forma inconsistente, dado que o percurso alternativo exige, primeiro, *subir* o potencial. Para este tipo de paisagem de recompensa enganadora (*deceptive*), implementou-se a **procura por novidade** (*Novelty Search*) de Lehman e Stanley [28], que substitui parcialmente a pressão pelo objetivo por uma pressão pela *diversidade comportamental*:

- **Caracterização comportamental (BC):** cada genoma é resumido pelo vetor $\mathbf{b}(\theta_k) \in \mathbb{R}^2$ — a posição final (x, y) do centroide do enxame, média dos E episódios de avaliação. Este descritor captura *para onde* a política transporta o enxame, permitindo que genomas que exploram regiões novas do mapa (por exemplo, o percurso alternativo) se distingam mesmo sem qualquer recolha.
- **Novidade:** $\nu(\theta_k)$ é a distância euclidiana média de $\mathbf{b}(\theta_k)$ aos seus $k = 10$ vizinhos mais próximos no conjunto formado pela população atual e por um **arquivo** de

comportamentos passados (os 3 mais novos de cada geração; capacidade máxima de 1000, gerida em regime FIFO).

- **Seleção mista:** os genomas são ordenados pelo *score* $s(\theta_k) = (1 - w) \cdot \hat{J}(\theta_k) + w \cdot \hat{v}(\theta_k)$, onde $\hat{\cdot}$ denota normalização *min-max* intra-geração e $w = 0.5$ pondera objetivo e novidade em partes iguais. Com $w = 0$ o mecanismo desliga-se e o algoritmo reduz-se *exatamente* ao ciclo evolutivo base — a implementação é *config-driven*, permitindo comparações controladas.
- **Salvaguarda do objetivo:** a novidade influencia apenas a *seleção* dos progenitores; o campeão gravado e reportado é sempre o melhor genoma segundo o *objetivo* $J(\theta_k)$. Garante-se assim que o modelo final resolve a tarefa, e não apenas que é comportamentalmente “diferente”.

O impacto desta extensão é avaliado no Capítulo 6 em duas frentes: uma comparação emparelhada preliminar no cenário *deceptive* e, sobretudo, duas campanhas *multi-run* com **orçamento igualado** ao da campanha final (7 execuções de 195 minutos por braço) nos dois cenários onde a otimização objetiva se mostrou vulnerável à decepção — a Porta com Alternativa (decepção por desenho) e o Muro em U (decepção espacial, comportamento bimodal) —, respondendo à QI6 (Secção 1.4).

5.3. Plano de Experiências e Tratamento Estatístico

O *pipeline* experimental é segmentado em cinco avaliações prioritárias (Tabela 5.1) por forma a assegurar validação e significância dos resultados reportados na tese.

TABELA 5.1. Plano de testes experimentais e métricas associadas.

Prior.	Experiência	Duração	Objetivo Científico	Métrica
1	Treino longo (Sandbox)	24h	Convergência máxima dos 3 modelos no cenário base.	P_{task} (Treino)
2	Avaliação Determinística (eval_all.py)	5 min	Desempenho sem exploração (20 episódios).	P_{task} (Eval)
3	Benchmark Estatístico (execuções múltiplas)	3–7 dias	Validação de hipóteses estatísticas ($p < 0.05$).	Wilcoxon / δ de Cliff
4	Robustez a Falhas (Remover 10% agentes)	1h + Treino	Medir resiliência à perda súbita de robôs a meio do episódio.	R_{robust}
5	Escalabilidade Zero-Shot ($N \in \{10, 50, 100\}$)	3h	Testar generalização do grafo dinâmico sem retreino.	S_{scale}

A integridade empírica apoia-se na execução paralela de simulações heterogêneas. Quaisquer evidências de supremacia algorítmica são escrutinadas estatisticamente por um

protocolo **não-paramétrico** — as distribuições de recolhas por episódio violam sistematicamente a normalidade (massas em zero, bimodalidade), inviabilizando o teste t clássico como teste principal. Sobre episódios de avaliação **emparelhados** (*seeds* comuns aos algoritmos comparados) aplica-se o teste de **Wilcoxon *signed-rank***; quando o emparelhamento não é possível, o teste de **Mann-Whitney U**. Cada comparação reporta ainda o **δ de Cliff** como tamanho de efeito não-paramétrico (em $[-1, 1]$, onde $|\delta| \geq 0.474$ se convencionou como efeito grande) e, a título complementar, o teste t de Welch. Assume-se o valor convencional de significância ($p < 0.05$). Todo o protocolo está automatizado em `scripts/statistical_tests.py`, que gera as tabelas e figuras do Capítulo 6 diretamente dos CSV de avaliação.

[Página intencionalmente deixada em branco.]

Resultados Experimentais e Discussão

Este capítulo é dedicado à apresentação e análise rigorosa dos dados recolhidos na campanha experimental final. Aqui, confrontamos o paradigma de *Multi-Agent Reinforcement Learning* (PPO e SAC) com o paradigma de Otimização Bio-inspirada (controlador neuroevolutivo com atenção sobre grafo), utilizando tratamento estatístico para responder às questões de investigação (Secção 1.4).

6.1. Metodologia de Avaliação e Notas de Leitura

Os resultados reportados provêm de uma **campanha de treino contínua de sete dias** nos servidores de cálculo, executada com a configuração final do sistema (recompensa simplificada da Secção 4.3, *fitness* de *homing* da Eq. 2.1, LiDAR de 8m). Antes da análise, importa fixar as convenções de leitura:

- **Runs independentes:** Cada combinação (algoritmo, cenário) foi treinada em **7 runs independentes** com sementes distintas, nos **7 cenários** do protocolo ($3 \times 7 \times 7 = 147$ treinos). As curvas reportam a média entre *runs* e a banda sombreada o desvio padrão.
- **Orçamentos de treino por paradigma:** cada *run* do controlador evolutivo dispôs de 195 minutos (população de 30 genomas avaliada em paralelo), e cada *run* do PPO/SAC de 48 minutos (16 ambientes vetorizados). Os orçamentos foram calibrados, com base nas campanhas exploratórias, para levar *cada* paradigma ao seu planalto de convergência — as curvas da Secção 6.5 confirmam a estabilização dentro do orçamento. A comparação mede, portanto, o **desempenho atingível** por cenário, e não a eficiência amostral a cómputo igual; a assimetria de cómputo (favorável ao evolutivo) é retomada na discussão (Secção 6.14).
- **Avaliação determinística por run:** cada um dos 7 modelos finais de cada célula foi avaliado em **20 episódios determinísticos emparelhados** (*seeds* comuns aos três algoritmos), num total de 140 episódios por combinação (algoritmo, cenário). A **unidade estatística é o run**: os testes entre algoritmos usam as médias por *run* ($n = 7$ por grupo, Mann-Whitney U + δ de Cliff), evitando inflacionar n com episódios correlacionados do mesmo modelo.
- **Métrica de tarefa pura (P_{task}):** Para além da recompensa total (que inclui *shaping*), reportam-se a taxa de sucesso (fração de episódios com pelo menos uma recolha) e as recolhas por episódio, obtidas em avaliação determinística, por serem as medidas não-enviesadas e *comparáveis* do cumprimento da missão.

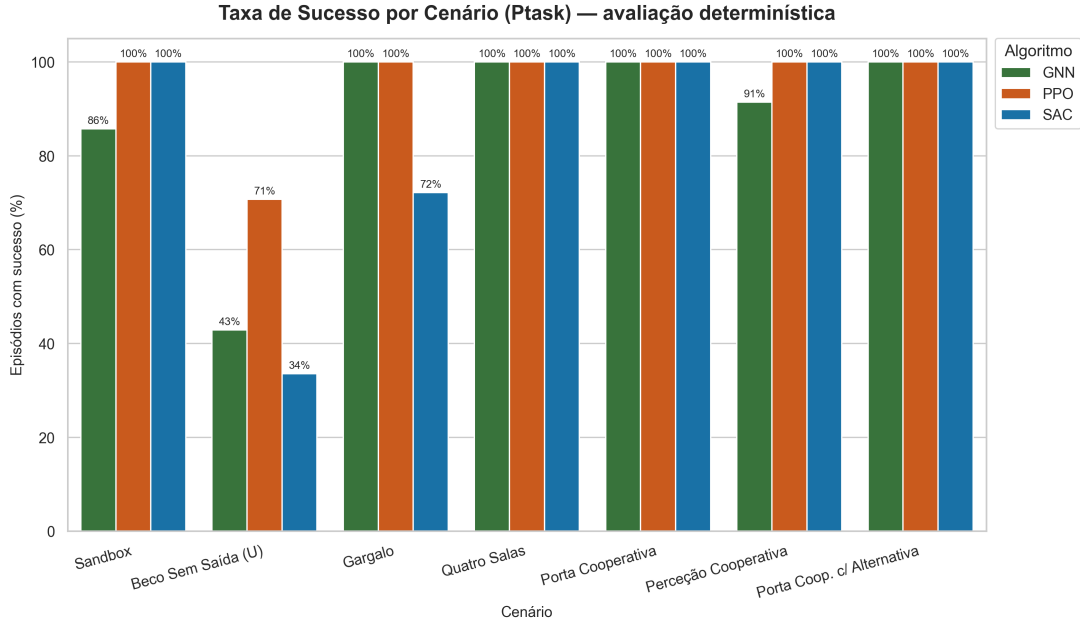
- **Escala não-comparáveis entre paradigmas:** O GNN usa *fitness* evolutiva e o PPO/SAC recompensa episódica; comparações de valor absoluto entre curvas de treino de paradigmas distintos devem ser evitadas, privilegiando as métricas de tarefa da avaliação determinística. O eixo das abcissas das curvas é normalizado para $[0\%, 100\%]$ do orçamento de cada algoritmo.

6.2. Desempenho de Tarefa por Cenário (P_{task})

A métrica central de comparação é o desempenho de **tarefa** em avaliação determinística: a taxa de sucesso (P_{task} , fração de episódios com pelo menos uma recolha) e o número de recolhas por episódio. Ao contrário da recompensa de treino — que inclui *shaping* e mistura escalas entre paradigmas (*fitness* evolutiva vs. recompensa episódica) —, estas métricas são diretamente comparáveis entre os três algoritmos e medem o cumprimento efetivo da missão. Os valores reportados agregam os **140 episódios determinísticos emparelhados** de cada combinação (7 *runs* $\times$ 20 episódios, *seeds* comuns aos três algoritmos).

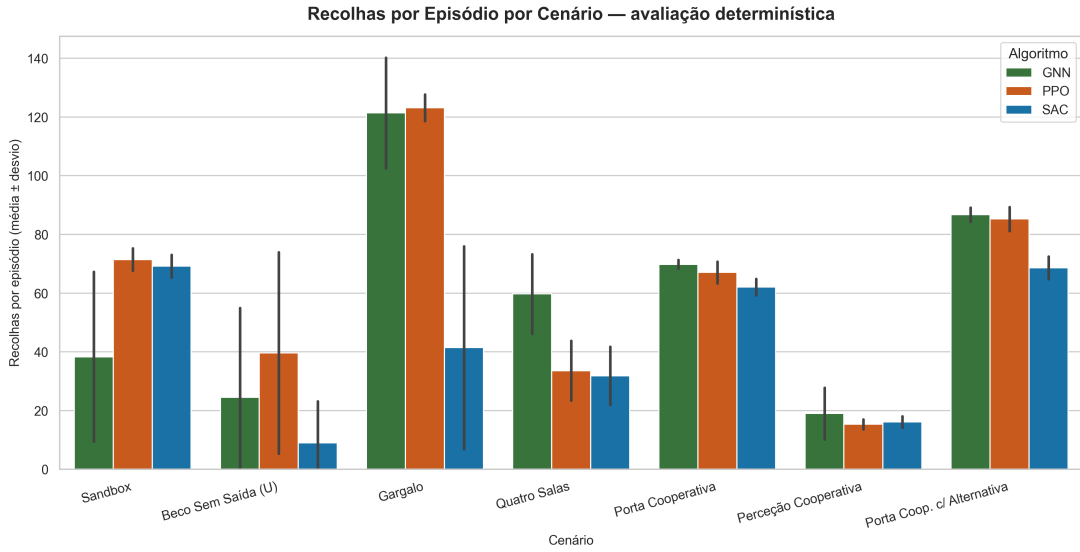
A Tabela 6.2 e as Figuras 6.1–6.2 mostram um quadro maioritariamente resolvido, mas com fragilidades concentradas — e diferentes — por algoritmo: **15 das 21 combinações algoritmo–cenário atingem 100% de sucesso em todos os *runs* e episódios**; as seis restantes correspondem a células em que um ou mais *runs* de treino falham por completo, pelo que a taxa média reportada esconde uma distribuição tudo-ou-nada entre *runs* (analisada na Secção 6.6). A Porta Cooperativa e o cenário *deceptive* com passagem alternativa estão resolvidos pelos três algoritmos. O **PPO** é o mais consistente: 100% de sucesso em seis dos sete cenários, com a menor variância entre *runs* (desvios padrão de 0,4 a 4,0 recolhas/ep fora do Muro U). O **controlador evolutivo (GNN)** resolve os quatro cenários de gargalo — Gargalo, Quatro Salas e as duas Portas Cooperativas — em *todos* os *runs*, destacando-se no Gargalo (121,4 recolhas/ep em média), no Quatro Salas (59,8, cerca de $1,8\times$ o melhor método de gradiente) e na Porta Cooperativa com Alternativa (86,7), mas exhibe *runs* degenerados precisamente nos cenários *sem paredes* (Sandbox: 5/7 *runs* funcionais; Percepção Cooperativa: 6/7). O **SAC** mantém 100% nos cenários cooperativos e no Quatro Salas, mas é o mais frágil nos gargalos físicos: no Gargalo apenas 5/7 *runs* convergem (com magnitude muito dispersa, $41,4 \pm 36,8$) e no Muro U apenas 2/7.

O **Muro U** é o único cenário que nenhum algoritmo resolve de forma fiável: o desempenho é *bimodal* nos três paradigmas — cada *run* ou aprende o desvio ao beco e atinge 100% de sucesso, ou não o aprende de todo (GNN 3/7 *runs*, PPO 4/7, SAC 2/7) — e nenhuma diferença entre algoritmos é estatisticamente significativa (Secção 6.14). A fronteira de dificuldade da campanha final já não separa, portanto, os cenários de gargalo dos de espaço aberto (essa fronteira foi eliminada pela *fitness* de *homing* e pela recompensa simplificada, como analisado adiante): o que resta é a **armadilha do desvio comprido** do Muro U, transversal aos dois paradigmas.



Sucesso = pelo menos 1 recolha no episódio. Métrica de tarefa, comparável entre algoritmos (ao contrário do reward de treino).

FIGURA 6.1. Taxa de sucesso (P_{task}) por cenário, em avaliação determinística (7 runs $\times$ 20 episódios emparelhados por algoritmo).



Média de recolhas (food_collected) por episódio; barras de erro = desvio padrão entre episódios. Mede magnitude do desempenho, não só sucesso/falha.

FIGURA 6.2. Recolhas por episódio (média $\pm$ desvio padrão) por cenário. Mede a magnitude do desempenho, para além do sucesso binário.

6.3. Análise Qualitativa da Navegação: Mapas de Calor

Para fundamentar a opção pelo potencial geodésico no termo de progresso e diagnosticar o comportamento de navegação, recorre-se a dois tipos de mapa de calor. A Figura 6.3 contrasta o potencial euclidiano com o geodésico no Muro U: as curvas de nível euclidianas atravessam a parede (atraindo o agente para o interior do beco), enquanto as geodésicas

a contornam, atribuindo gradiente positivo ao desvio correto. A Figura 6.4 mostra a densidade de ocupação dos robôs — zonas brilhantes coladas a uma parede revelam agentes presos, ao passo que um fluxo que contorna o obstáculo indica navegação resolvida.

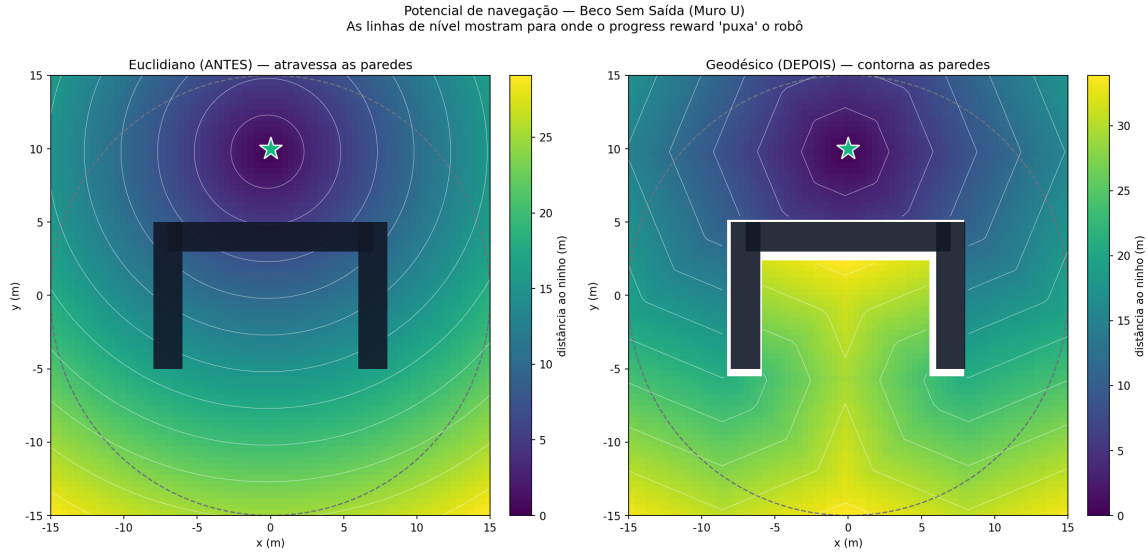


FIGURA 6.3. Potencial de navegação no Muro U: euclidiano (esquerda) vs. geodésico (direita). As curvas de nível indicam a direção para a qual o termo de progresso atrai o agente; só o geodésico contorna a barra do U.

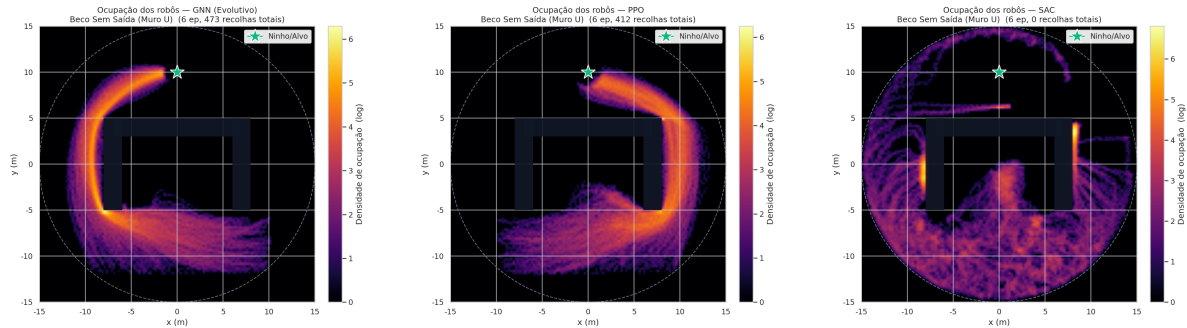


FIGURA 6.4. Densidade de ocupação dos robôs no Muro U (GNN, PPO e SAC, da esquerda para a direita). O fluxo que contorna a barra do U pelos lados evidencia a navegação resolvida pelo potencial geodésico.

6.4. Convergência e Desempenho no Cenário Base (Sandbox)

O cenário Sandbox — arena aberta, sem paredes interiores — isola a capacidade de *foraging* da componente de navegação por obstáculos, servindo de referência de otimalidade. A Figura 6.5 mostra que os dois métodos de gradiente convergem para políticas funcionais e *repetíveis*: as bandas de desvio padrão entre os 7 *runs* são estreitas, e o desempenho final de tarefa é praticamente idêntico (PPO $71,5 \pm 1,0$ recolhas/ep; SAC $69,2 \pm 1,9$), com o **SAC** (*off-policy*) a subir mais cedo, beneficiando da reutilização de experiência do *replay buffer*. O **GNN evolutivo** apresenta aqui o seu comportamento mais irregular

de toda a campanha: quatro dos sete *runs* convergem para políticas competitivas (61,6 a 64,7 recolhas/ep), dois degeneram por completo (< 1 recolha/ep) e um fica num regime intermédio (15,8 recolhas/ep, com sucesso em todos os episódios mas uma fração da magnitude dos melhores), arrastando a média para $38,3 \pm 31,0$. É um resultado à primeira vista paradoxal — o cenário mais simples é o menos fiável para o evolutivo — que se explica pela ausência de gradiente geodésico informativo em campo aberto: sem paredes que canalizem o *homing*, a paisagem de *fitness* tem múltiplos ótimos locais de vaguear, e a fuga a esses ótimos depende da sorte da inicialização (Secção 6.14).

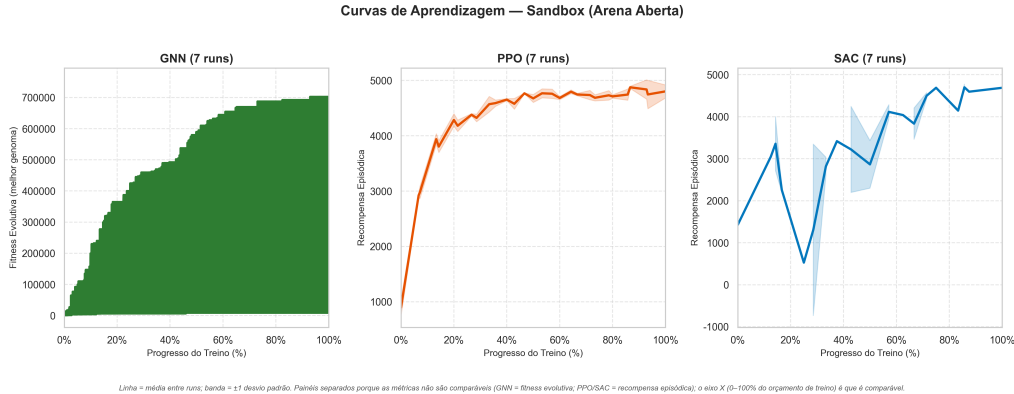


FIGURA 6.5. Curvas de aprendizagem dos três algoritmos no cenário Sandbox. O eixo das abcissas está normalizado (0–100% do orçamento de treino); linha = média de 7 *runs*, banda = ± 1 desvio padrão.

A Tabela 6.1 sintetiza o desempenho de avaliação neste cenário.

TABELA 6.1. Desempenho no cenário Sandbox (avaliação determinística; recolhas/ep em média $\pm$ desvio padrão entre os 7 *runs*; P_{task} sobre os 140 episódios).

Algoritmo	Recolhas/ep	P_{task}	<i>Runs</i> funcionais
GNN (Evolutivo)	$38,31 \pm 31,03$	85,7%	5/7
PPO	$71,46 \pm 0,98$	100%	7/7
SAC	$69,24 \pm 1,88$	100%	7/7

“*Runs* funcionais” = *runs* com 100% de sucesso nos 20 episódios de avaliação.

6.5. Desempenho Comparativo por Cenário (Curvas de Aprendizagem)

As curvas de aprendizagem por cenário (Figuras 6.6–6.11) detalham a dinâmica de treino subjacente aos resultados de tarefa, agora com bandas de variância reais (7 *runs*). Importa uma ressalva de leitura: os eixos verticais não são comparáveis entre paradigmas — o GNN é selecionado por *fitness* (dominada pelo termo de comida $\times 10^4$), ao passo que PPO e SAC reportam recompensa episódica com *shaping* —, pelo que cada painel deve ser lido como a trajetória *interna* de cada algoritmo, e não como uma corrida no mesmo eixo.

Nos cenários de navegação por gargalos (**Gargalo**, Figura 6.7; **Quatro Salas**, Figura 6.8; **Porta Cooperativa**, Figura 6.9) o padrão da campanha final é o oposto do observado nas campanhas exploratórias: a *fitness* do GNN sobe de forma sustentada e traduz-se em recolhas na avaliação — a assinatura do antigo *fitness exploitation* (subida de *shaping* sem tarefa) desapareceu com a *fitness* de *homing*. Entre os métodos de gradiente, o PPO converge de forma quase determinística (bandas estreitas), enquanto o SAC exibe bandas largas no Gargalo, refletindo os *runs* que não convergem. No **Muro U** (Figura 6.6), as bandas largas dos três algoritmos são o retrato da bimodalidade já referida: a média das curvas mistura *runs* resolvidos e falhados. Na **Percepção Cooperativa** (Figura 6.10) os três algoritmos progridem, com o GNN a atingir a maior magnitude média. Finalmente, na **Porta Cooperativa com Alternativa** (Figura 6.11) — o cenário *deceptive* introduzido para a QI6 — os três algoritmos aprendem o desvio pelo corredor alternativo em todos os *runs*, com o GNN e o PPO em vantagem clara de magnitude sobre o SAC (86,7 e 85,3 vs. 68,6 recolhas/ep).

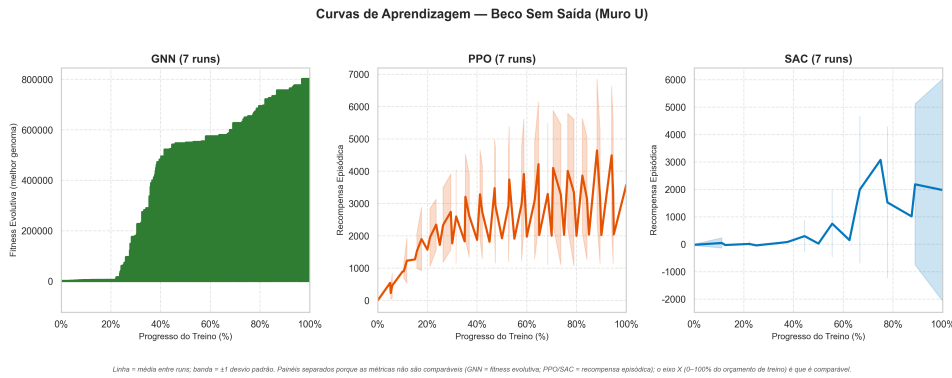


FIGURA 6.6. Curvas de aprendizagem no cenário Beco Sem Saída (Muro em U).

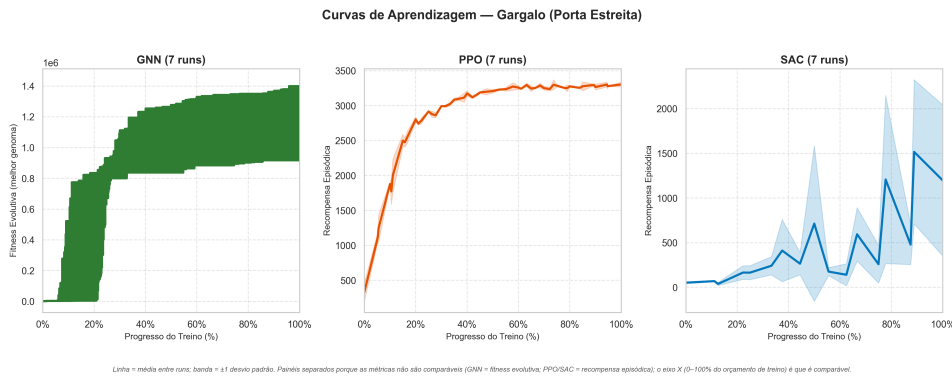


FIGURA 6.7. Curvas de aprendizagem no cenário Gargalo.

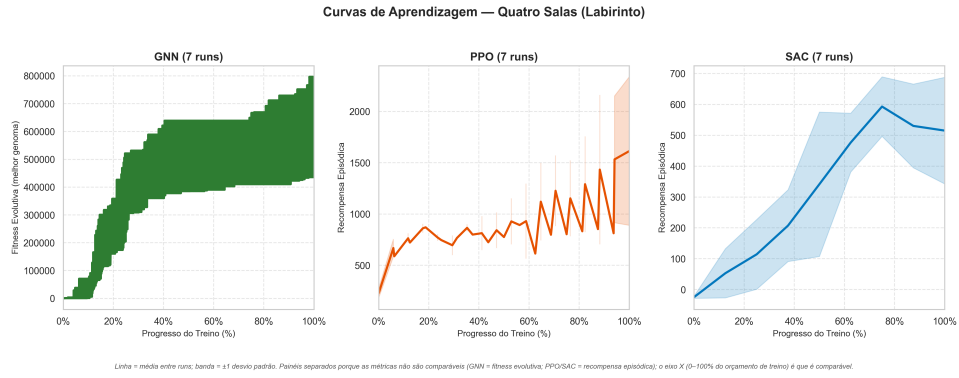


FIGURA 6.8. Curvas de aprendizagem no cenário Quatro Salas (Labirinto).

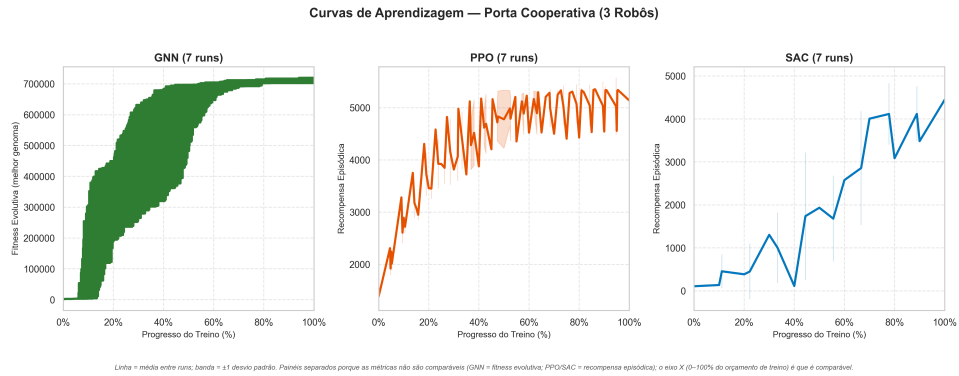


FIGURA 6.9. Curvas de aprendizagem no cenário Porta Cooperativa.

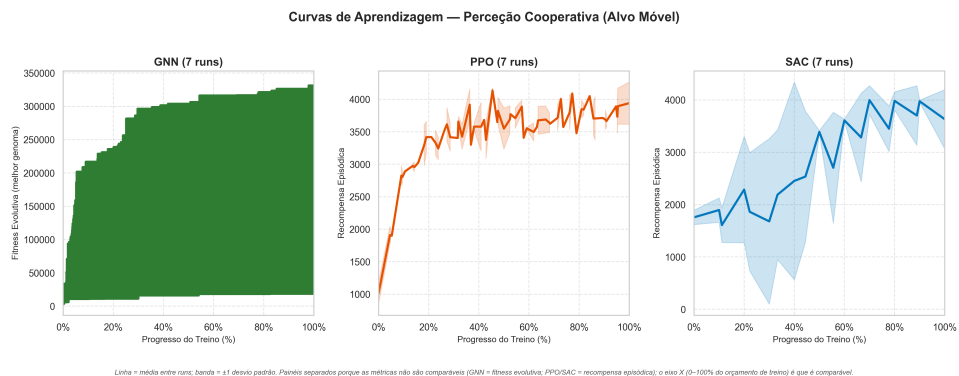


FIGURA 6.10. Curvas de aprendizagem no cenário Percepção Cooperativa (Alvo Móvel).

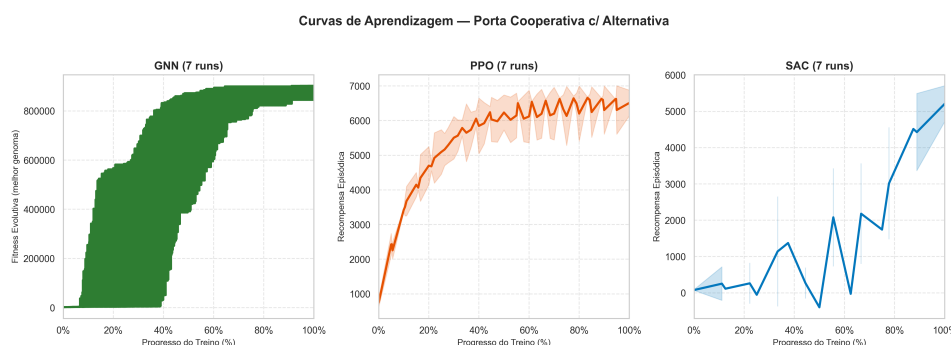
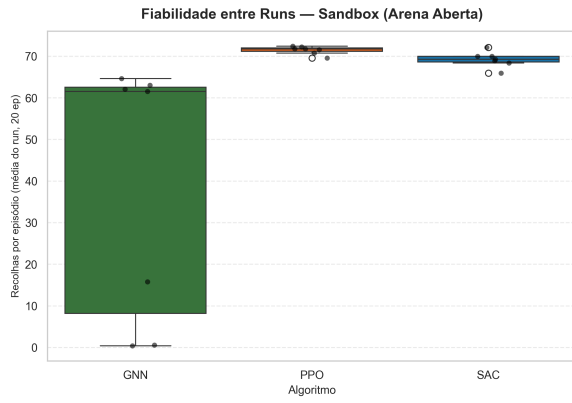


FIGURA 6.11. Curvas de aprendizagem no cenário Porta Cooperativa com Alternativa (*deceptive*).

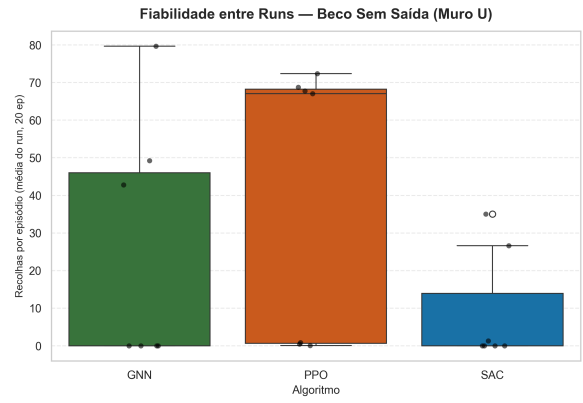
6.6. Fiabilidade e Variância entre *Runs* (Boxplots)

A análise da fiabilidade de treino requer múltiplas execuções independentes por configuração, de modo a distinguir o desempenho sistemático do ruído de inicialização. Os diagramas de caixa das Figuras 6.12–6.15 mostram, para cada cenário, a distribuição das **recolhas por episódio de cada *run*** (média dos 20 episódios de avaliação; 7 pontos por algoritmo) — uma métrica de tarefa, na mesma unidade para os três algoritmos, ao contrário dos *scores* de treino.

A leitura por cenário quantifica o que as secções anteriores anteciparam. Nos cenários **cooperativos** (Porta Cooperativa, Porta com Alternativa) e no **Quatro Salas**, as caixas dos três algoritmos são compactas e disjuntas do zero — treinar é *fiável* — e o GNN posiciona-se sistematicamente acima do SAC (e, no Quatro Salas, muito acima de ambos os métodos de gradiente). No **Gargalo**, PPO e GNN são fiáveis (7/7 *runs*), mas a caixa do SAC estende-se de 0 a 88 recolhas/ep — o treino é uma lotaria. No **Sandbox** inverte-se: PPO/SAC compactos, GNN com dois *runs* degenerados. No **Muro U**, as três caixas tocam o zero: nenhum algoritmo garante a convergência, e a mediana só é positiva no PPO (4/7 *runs* funcionais). Esta análise por *run* é a base dos testes de significância da Secção 6.14.

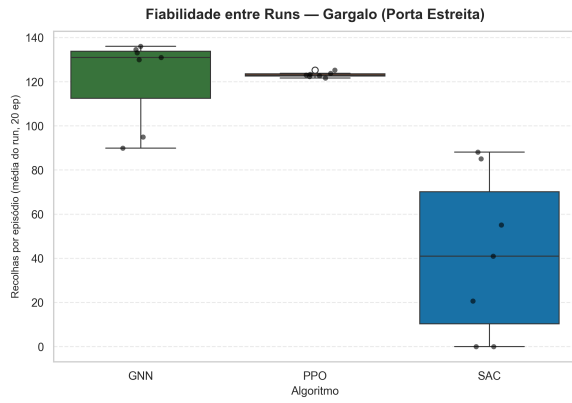


Cada ponto = 1 run independente (7 runs/algorithm; média de 20 episódios determinísticos). Métrica de tarefa: mesma unidade para os três algoritmos.

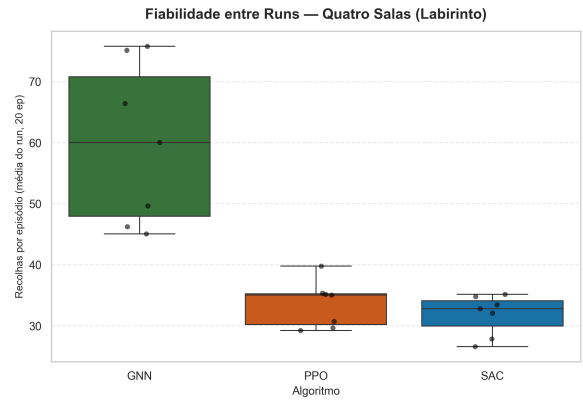


Cada ponto = 1 run independente (7 runs/algorithm; média de 20 episódios determinísticos). Métrica de tarefa: mesma unidade para os três algoritmos.

FIGURA 6.12. Recolhas/ep por *run* (7 *runs*, 20 ep cada): Sandbox (esq.) e Muro U (dir.).

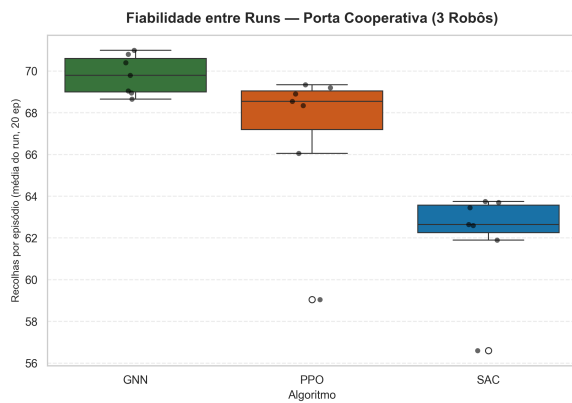


Cada ponto = 1 run independente (7 runs/algorithm; média de 20 episódios determinísticos). Métrica de tarefa: mesma unidade para os três algoritmos.

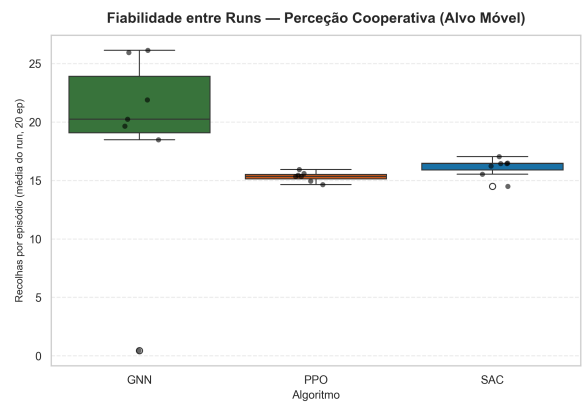


Cada ponto = 1 run independente (7 runs/algorithm; média de 20 episódios determinísticos). Métrica de tarefa: mesma unidade para os três algoritmos.

FIGURA 6.13. Recolhas/ep por *run*: Gargalo (esq.) e Quatro Salas (dir.).

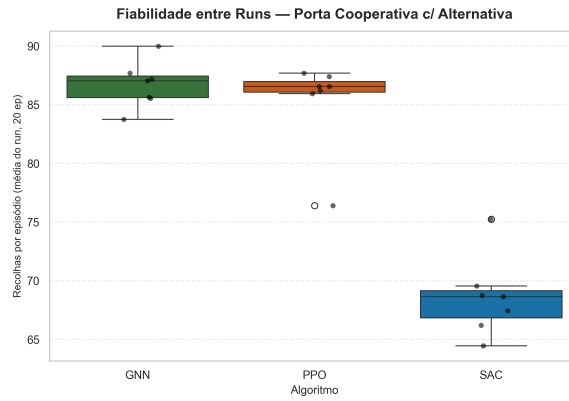


Cada ponto = 1 run independente (7 runs/algorithm; média de 20 episódios determinísticos). Métrica de tarefa: mesma unidade para os três algoritmos.



Cada ponto = 1 run independente (7 runs/algorithm; média de 20 episódios determinísticos). Métrica de tarefa: mesma unidade para os três algoritmos.

FIGURA 6.14. Recolhas/ep por *run*: Porta Cooperativa (esq.) e Percepção Cooperativa (dir.).



Cada ponto = 1 run independente (7 runs/algoritmo; média de 20 episódios determinísticos). Métrica de tarefa: mesma unidade para os três algoritmos.

FIGURA 6.15. Recolhas/ep por *run*: Porta Cooperativa com Alternativa (*deceptive*).

6.7. Visão Global por Algoritmo

Consolidando o desempenho ao longo dos sete cenários (Figuras 6.16–6.18), emergem três perfis bem definidos. O **GNN evolutivo com *fitness de homing*** é o especialista em *navegação estruturada*: converge em *todos* os 28 *runs* dos quatro cenários de gargalo (Gargalo, Quatro Salas e as duas Portas Cooperativas — incluindo, portanto, o cenário *deceptive*, que o *homing* resolve sem necessidade de pressão por novidade), lidera a magnitude de recolhas no Quatro Salas e na Porta Cooperativa e empata estatisticamente com o PPO no Gargalo e na Porta com Alternativa; o seu ponto fraco são os cenários abertos, onde ocasionalmente degenera. O **PPO** é o *generalista fiável*: converge em todos os *runs* de seis cenários com variância mínima, e é o menos afetado pelo Muro U (4/7). O **SAC** é fiável em espaço aberto e nos cooperativos, mas é o mais *sensível aos gargalos físicos*: é o único que falha *runs* no Gargalo e tem a pior taxa no Muro U (2/7). A Figura 6.19 resume a comparação por barras na métrica de tarefa (recolhas/ep na avaliação determinística) — ao contrário das versões preliminares deste gráfico, todas as barras estão na mesma unidade e são diretamente comparáveis.

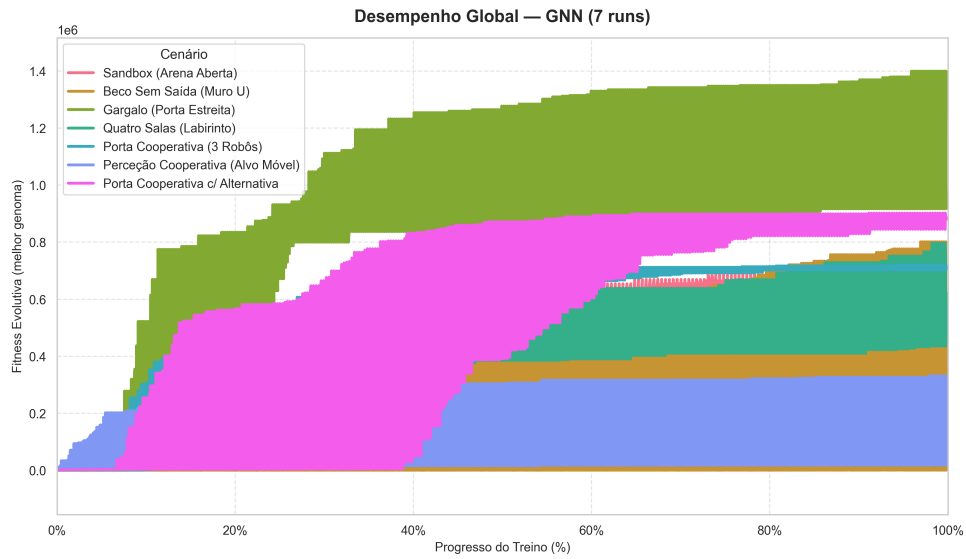


FIGURA 6.16. Desempenho do GNN (evolutivo) nos sete cenários (média $\pm$ desvio padrão de 7 *runs*).

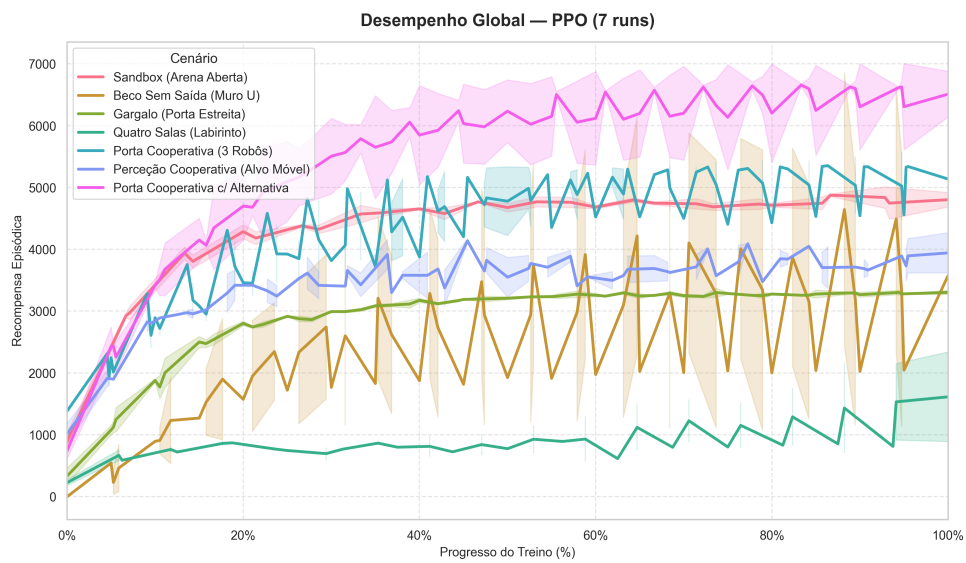


FIGURA 6.17. Desempenho do PPO nos sete cenários (média $\pm$ desvio padrão de 7 *runs*).

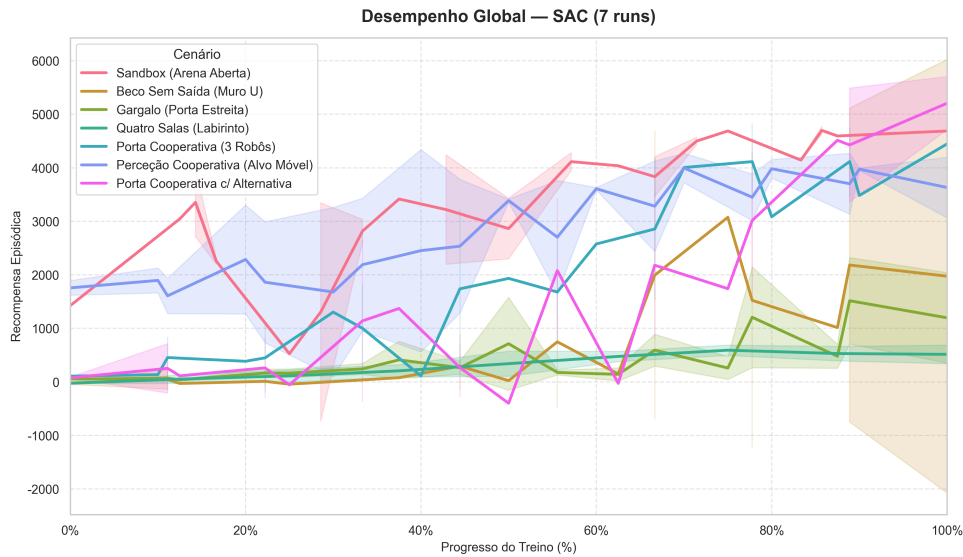
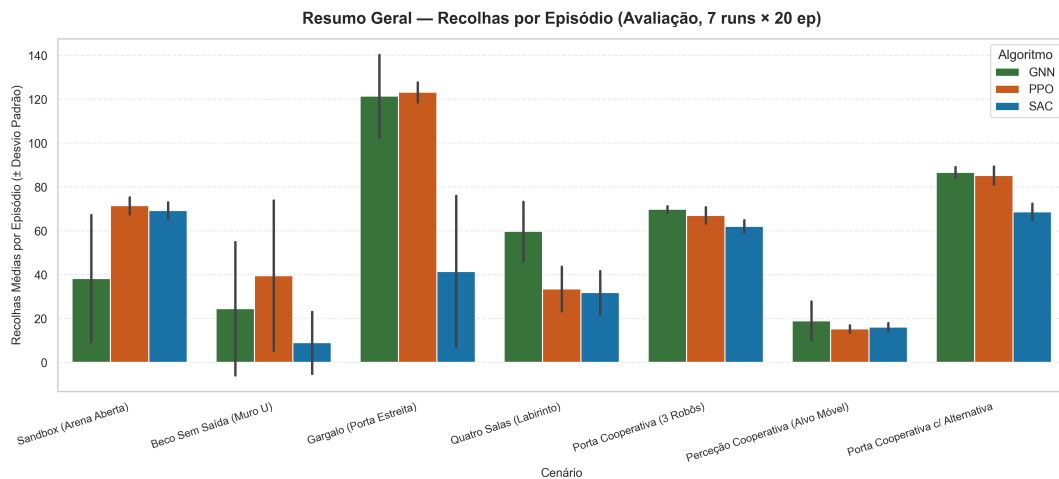


FIGURA 6.18. Desempenho do SAC nos sete cenários (média $\pm$ desvio padrão de 7 *runs*).



Métrica de TAREFA (recolhas/episódio em avaliação determinística, 140 episódios por barra); mesma unidade para os três algoritmos, diretamente comparável.

FIGURA 6.19. Resumo geral: recolhas por episódio (avaliação determinística) por algoritmo e cenário; barras de erro = desvio padrão sobre os 140 episódios.

6.8. Avaliação Determinística (Tarefa Pura)

A Tabela 6.2 reúne a métrica mais honesta do desempenho real na missão: avaliação em modo determinístico (sem ruído de exploração), com 7 *runs* $\times$ 20 episódios emparelhados por configuração. Os valores fundamentam a discussão das secções anteriores — a competitividade do controlador evolutivo nos cenários de navegação, a consistência do PPO e a fragilidade localizada do SAC nos gargalos.

TABELA 6.2. Avaliação determinística ($7 \text{ runs} \times 20$ episódios emparelhados): taxa de sucesso (P_{task} , sobre os 140 episódios) e recolhas por episódio (média $\pm$ desvio padrão entre *runs*). A negrito, a melhor média de recolhas de cada cenário.

Cenário	GNN		PPO		SAC	
	Sucesso	Recolhas	Sucesso	Recolhas	Sucesso	Recolhas
Sandbox	85,7%	$38,3 \pm 31,0$	100%	$71,5 \pm 1,0$	100%	$69,2 \pm 1,9$
Muro U	42,9%	$24,5 \pm 32,6$	70,7%	$39,6 \pm 36,7$	33,6%	$9,0 \pm 15,1$
Gargalo	100%	$121,4 \pm 19,9$	100%	$123,2 \pm 1,2$	72,1%	$41,4 \pm 36,8$
Quatro Salas	100%	$59,8 \pm 13,2$	100%	$33,6 \pm 3,8$	100%	$31,8 \pm 3,3$
Porta Cooperativa	100%	$69,8 \pm 0,9$	100%	$67,1 \pm 3,7$	100%	$62,1 \pm 2,5$
Percepção Coop.	91,4%	$19,0 \pm 8,7$	100%	$15,3 \pm 0,4$	100%	$16,1 \pm 0,8$
Porta c/ Alternativa	100%	$86,7 \pm 2,0$	100%	$85,3 \pm 4,0$	100%	$68,6 \pm 3,4$

6.9. Análise de Cenários Complexos e Comportamento Emergente

Para além das métricas agregadas, a inspeção qualitativa no visualizador 3D (Figuras 6.20–6.21) revela os comportamentos coletivos que sustentam — ou comprometem — o desempenho de tarefa. No **Muro U** e no **Quatro Salas** (Figura 6.20), as políticas bem-sucedidas exibem contorno fluido das paredes, com os agentes a seguirem o gradiente geodésico em vez de colidirem frontalmente com os obstáculos — o efeito direto da substituição do potencial euclidiano discutida na Secção 6.3. A dispersão inicial do *spawn* pelas salas opostas ao ninho mitiga o congestionamento nas passagens estreitas, que era a causa do colapso anterior no Quatro Salas.

Nos cenários cooperativos (Figura 6.21) emergem comportamentos coordenados não programados explicitamente: na **Porta Cooperativa**, observa-se a sincronização de pelo menos três agentes na zona de pressão para abrir a passagem, condição imposta pela mecânica do cenário; na **Percepção Cooperativa**, o enxame organiza-se em torno do alvo móvel, mantendo cobertura à medida que este se desloca. Estes padrões confirmam que os controladores de gradiente aprendem não só a navegar, mas a explorar a estrutura cooperativa da tarefa. (As capturas apresentadas são ilustrativas do comportamento típico observado em execução.)

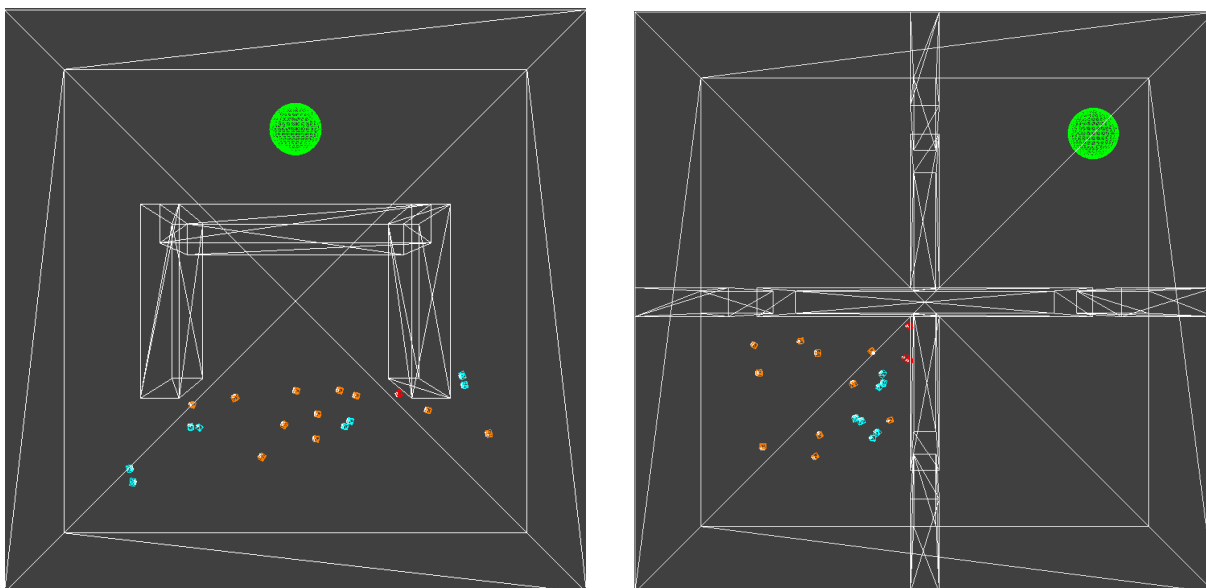


FIGURA 6.20. Comportamento emergente no visualizador 3D: contorno do obstáculo em U (esq.) e navegação no labirinto de Quatro Salas (dir.).

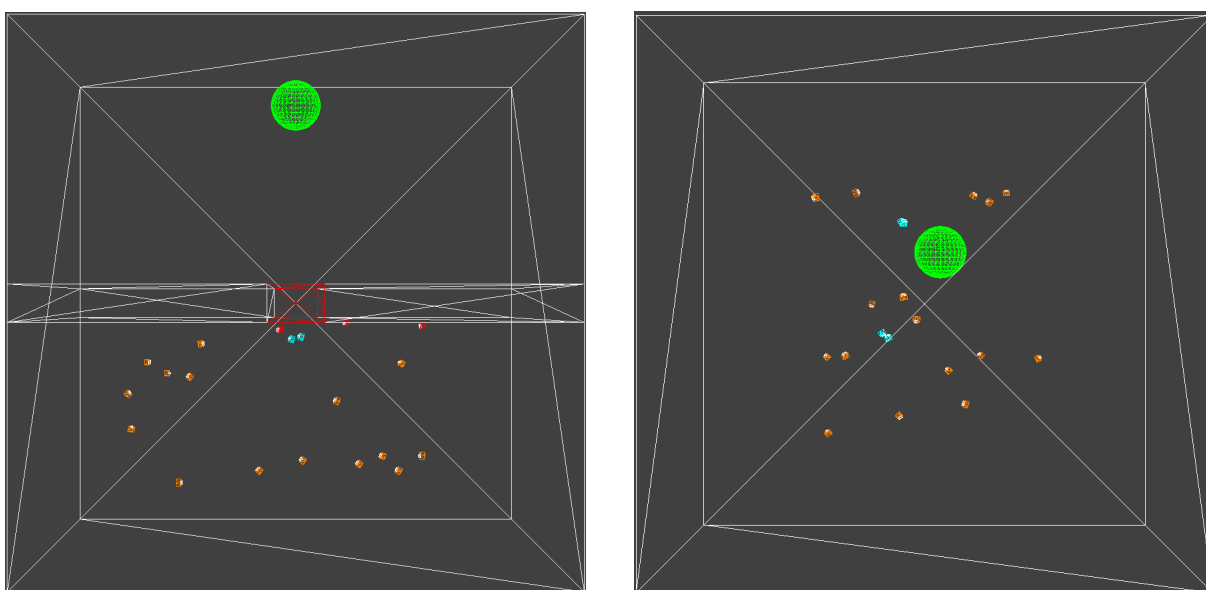


FIGURA 6.21. Comportamento cooperativo: abertura sincronizada da porta (esq.) e cerco a 360° do alvo móvel (dir.).

6.10. Deceção e Procura por Novidade (*Novelty Search*)

Dois cenários da bateria expõem a otimização orientada ao objetivo a paisagens de recompensa enganadoras, por vias distintas. A **Porta Cooperativa com Alternativa** é *deceptive* por desenho: o gradiente de *homing* aponta para a porta bloqueada, e o caminho viável — o corredor alternativo — exige afastamento temporário do ninho. O **Muro em U** exibe *deceção espacial*: o potencial geodésico aponta corretamente para o contorno, mas o agente só observa a bússola euclidiana e o LiDAR local, pelo que a *descoberta* do desvio comprido depende de exploração — e é precisamente aí que a campanha final mostrou

comportamento bimodal transversal aos três algoritmos (Secção 6.6). A QI6 pergunta se a hibridização com *Novelty Search* (Secção 5.2.2) ajuda a escapar a estes ótimos locais. Realizaram-se três experiências: uma comparação emparelhada preliminar e, como teste definitivo, duas campanhas multi-*run* com orçamento igualado.

Comparação emparelhada preliminar (novidade vs. objetivo puro). Numa experiência controlada anterior à campanha final, treinou-se um controlador com *blend* de novidade ($w = 0,5$, 600 minutos) na Porta com Alternativa e comparou-se com o melhor controlador puramente objetivo então disponível (*fitness* de *homing*, 195 minutos), sob o mesmo protocolo de avaliação emparelhado (20 episódios, *seeds* comuns). O controlador com novidade obteve $81,3 \pm 1,9$ recolhas/ep (100% de sucesso) contra 64,5 do objetivo puro — +26%, Wilcoxon $p = 8,7 \times 10^{-5}$, $\delta = +1,00$. Duas ressalvas metodológicas são, contudo, incontornáveis, e ambas motivaram as campanhas seguintes. Primeira, a execução com novidade dispôs de um orçamento $\approx 3\times$ superior, pelo que o resultado media o efeito *conjunto* (novidade + orçamento). Segunda, este teste é o único do capítulo cuja unidade estatística é o *episódio* ($n = 20$, um só modelo por braço) e não o *run*: os episódios de um mesmo modelo não são realizações independentes do processo de treino, pelo que este p mede a consistência do par de modelos comparados e **não é comparável** com os valores reportados nas restantes secções, todos calculados sobre médias por *run*. É por esta razão que a comparação preliminar se apresenta aqui como indício, e não como resultado. Acresce uma limitação de proveniência: os artefactos brutos desta execução preliminar (avaliação por episódio e modelo) não foram retidos — foram entretanto sobrescritos no servidor de treino por campanhas posteriores —, pelo que os valores citados constituem o registo da altura e não são reproduzíveis a partir dos dados arquivados. As campanhas com orçamento igualado que se seguem, essas, têm todos os dados de avaliação versionados e são integralmente reproduzíveis.

Campanhas com orçamento igualado. Para isolar o efeito da novidade, treinaram-se 7 execuções independentes com *blend* de novidade ($w = 0,5$) em cada um dos dois cenários, com o *mesmo* orçamento (195 minutos/*run*), protocolo de avaliação (20 episódios determinísticos) e *seeds* da campanha final — o braço de controlo é, assim, o próprio GNN objetivo da Tabela 6.2. A significância é avaliada ao nível do *run* (Mann–Whitney exato sobre as médias por execução, $n = 7$ por braço), como no resto do capítulo.

No **Muro em U**, o resultado é o mais expressivo da linha de novidade: a hibridização elevou a taxa de convergência de 3/7 para 7/7 **runs a 100% de sucesso**, com $69,8 \pm 5,9$ recolhas/ep contra $24,5 \pm 32,6$ do objetivo puro ($p = 0,026$, $\delta = +0,71$). A pressão por novidade não só venceu a bimodalidade que nenhum dos três algoritmos base superou, como o fez com a variância mais baixa registada neste cenário — a configuração com novidade é a única, de todas as estudadas nesta dissertação, que resolve o Muro em U de forma fiável.

Na **Porta com Alternativa**, o quadro inverte-se: ambos os braços convergem em 7/7 *runs*, mas o objetivo puro é significativamente superior em magnitude — $86,7 \pm 2,0$

contra $63,0 \pm 21,9$ recolhas/ep com novidade ($p = 0,0006$, $\delta = -1,00$: todas as execuções objetivas superam todas as execuções com novidade). O ganho de +26% da comparação preliminar fica assim desmascarado como artefacto do orçamento triplo; com orçamento igualado, desviar metade da pressão de seleção para a diversidade comportamental *custa* desempenho num cenário cujo gradiente, apesar de enganador à partida, é vencível pela otimização objetiva com a profundidade de treino da campanha final.

Leitura conjunta. A pressão por novidade comporta-se como um instrumento *direcionado*, não como um melhoramento universal: onde a paisagem de *fitness* deixa a otimização objetiva inconsistente (Muro em U, bimodal), a novidade compra a descoberta do desvio e paga-se largamente; onde o gradiente da tarefa já basta (Porta com Alternativa, 7/7), o mesmo mecanismo desperdiça metade da pressão seletiva em diversidade redundante e degrada o resultado. O peso fixo $w = 0,5$ é, aliás, a explicação provável de ambos os efeitos — uma ponderação adaptativa (por exemplo, decrescente com o progresso do objetivo) é a extensão natural desta linha (Secção 7.5).

6.11. Análise de Escalabilidade (Zero-Shot Transfer)

Para além do desempenho de tarefa, é na **escalabilidade** que reside o contributo distintivo do controlador evolutivo — e a justificação arquitetural da abordagem por atenção sobre grafo. Avaliou-se a transferência, *sem qualquer retreino*, da política aprendida com $N = 20$ agentes para enxames de $N \in \{10, 50, 100\}$ — e, ao contrário do que é prática comum na literatura (onde a escalabilidade, quando é medida, se limita a arenas abertas), a bateria cobre **os sete cenários**, incluindo os labirintos e as tarefas cooperativas. Em cada célula (cenário $\times$ dimensão) correram-se 20 episódios determinísticos com o modelo campeão da campanha final.

O resultado é inequívoco: o controlador de grafo mantém **100% de sucesso nas 28 combinações**, de metade a cinco vezes o enxame de treino, em todos os cenários (Tabela 6.3). As políticas MLP do PPO e do SAC, com dimensão de entrada fixa ($\mathbb{R}^{111}$, calibrada para $N = 20$), são por construção incompatíveis com outras dimensões de enxame (Tabela 6.4); no tamanho de treino, os três algoritmos são comparáveis (Sandbox, $N = 20$: GNN 65,8, PPO 71,7, SAC 71,3 recolhas/ep). Este contraste corrobora diretamente a hipótese de que o mecanismo de atenção sobre a vizinhança confere invariância à cardinalidade do enxame, sendo a propriedade que habilita o controlo escalável.

TABELA 6.3. Escalabilidade Zero-Shot do controlador de grafo nos sete cenários (20 episódios determinísticos por célula; modelo campeão da campanha final, treinado com $N = 20$): recolhas *por agente* e retenção da eficiência per capita em $N = 100$ face a $N = 20$. Taxa de sucesso: 100% em todas as células.

Cenário	$N = 10$	$N = 20$	$N = 50$	$N = 100$	Retenção
Sandbox	3,74	3,29	1,95	1,27	39%
Perceção Cooperativa	1,64	1,30	0,83	0,59	45%
Gargalo	6,92	6,93	5,96	4,04	58%
Quatro Salas	3,60	3,86	3,25	2,54	66%
Beco Sem Saída (U)	3,83	4,01	3,81	3,14	78%
Porta Cooperativa	3,40	3,54	3,41	3,12	88%
Porta c/ Alternativa	4,31	4,50	4,44	4,06	90%

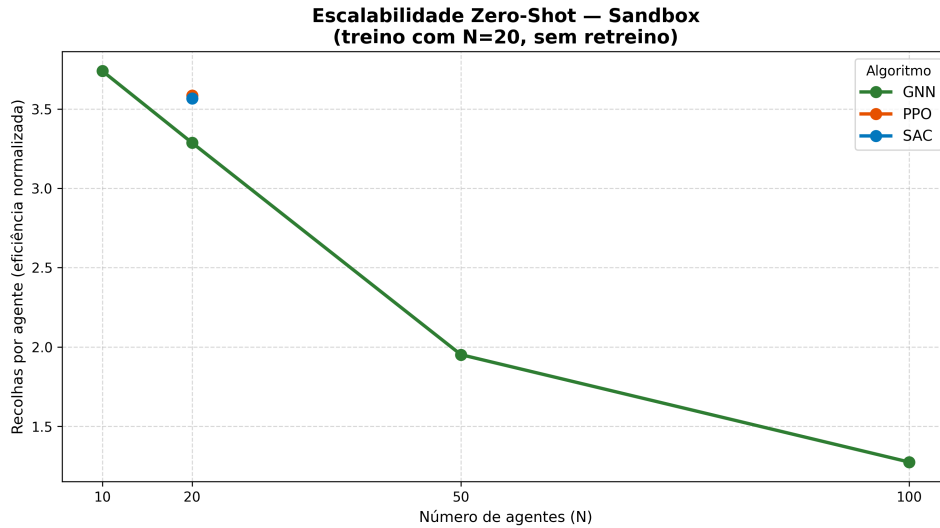
O padrão da coluna de retenção merece leitura atenta, porque desarma a objeção mais natural a este resultado — a de que um enxame cinco vezes maior colapsaria por congestionamento nas passagens estreitas. Observa-se, em larga medida, o *contrário*: a retenção mais alta pertence aos cenários com estrutura física (Porta com Alternativa 90%, Porta Cooperativa 88%, Muro em U 78%) e a mais baixa aos cenários abertos (Sandbox 39%, Perceção Cooperativa 45%), onde $N = 100$ agentes partilham um recurso finito cuja densidade por agente se reduz a um quinto — diluição de recurso, não falha de coordenação. Nesses cenários estruturados, o fluxo canalizado pelas paredes e a natureza repetível da recolha no ninho mantêm o rendimento individual: com 100 agentes, a porta cooperativa e o desvio do Muro em U continuam a ser atravessados de forma ordeira.

A leitura honesta exige, porém, registar os dois casos intermédios: o Quatro Salas retém 66% e o **Gargalo** 58% — a retenção mais baixa de todos os cenários com paredes. É precisamente o que se esperaria de uma saturação parcial do fluxo: o Gargalo tem uma *única* passagem de 2,5 m, e é o cenário em que o congestionamento tem mais margem para morder. Mas mesmo aí não há colapso: a taxa de sucesso mantém-se em 100% e as recolhas *totais* continuam a crescer com o enxame (138,6 em $N = 20$ para 403,6 em $N = 100$). A conclusão que a coluna suporta é, pois, mais matizada do que a ausência total de congestionamento: a estrutura física *atenua* a diluição em vez de a agravar, e onde a geometria impõe um estrangulamento único observa-se um custo de fluxo mensurável — mas nunca a rutura da coordenação que a objeção antecipava.

TABELA 6.4. O contraste arquitetural no cenário Sandbox (20 episódios/dimensão): taxa de sucesso e recolhas totais por episódio ao transferir a política treinada com $N = 20$, sem retreino.

Algoritmo	$N = 10$	$N = 20$ (treino)	$N = 50$	$N = 100$
GNN (Evolutivo)	100% / 37,4	100% / 65,8	100% / 97,5	100% / 127,3
PPO	N/A [†]	100% / 71,7	N/A [†]	N/A [†]
SAC	N/A [†]	100% / 71,3	N/A [†]	N/A [†]

Cada célula: taxa de sucesso / recolhas por episódio. [†] A política MLP do PPO/SAC tem dimensão de entrada fixa ($\mathbb{R}^{111}$, para $N = 20$), sendo incompatível com outras dimensões de enxame. Só a arquitetura de atenção sobre grafo (controlador evolutivo) suporta a transferência *Zero-Shot*. As recolhas totais do GNN crescem monotonicamente com N ($37,4 \rightarrow 127,3$) sem qualquer colapso.



PPO, SAC: MLP de entrada fixa — incompatível com $N=20$ (só a GNN faz transferência Zero-Shot).

FIGURA 6.22. Degradação (ou instabilidade) do desempenho com o aumento do número de agentes, sem retreino.

6.12. Resiliência e Robustez a Falhas de Agentes

A métrica R_{robust} avalia a resiliência do enxame perante a perda súbita de 10% dos agentes a meio do episódio (os agentes afetados ficam inertes, deixando de agir e de contar para os requisitos cooperativos). A Figura 6.23 compara as recolhas com e sem falha, em avaliação emparelhada (20 episódios por célula, 3 algoritmos $\times$ 7 cenários) sobre os modelos da campanha final.

O resultado evidencia uma resiliência elevada e *transversal aos paradigmas*: a retenção de recolhas situa-se entre **92% e 106%** em todas as 21 combinações algoritmo–cenário com desempenho de base, incluindo a Porta Cooperativa — notável por exigir três agentes simultâneos para abrir a passagem: o enxame redistribui-se e mantém a função apesar da baixa. O controlador evolutivo, medido pela primeira vez nos labirintos nesta campanha, retém 92–97%. (Os valores pontualmente acima de 100% correspondem a episódios em

que a perda de agentes reduziu o congestionamento nas passagens estreitas.) A ausência de degradação catastrófica sugere que o *parameter sharing* e a observação local conferem ao enxame uma redundância funcional implícita: nenhum agente é insubstituível, pelo que a falha parcial é absorvida pela reorganização do coletivo.

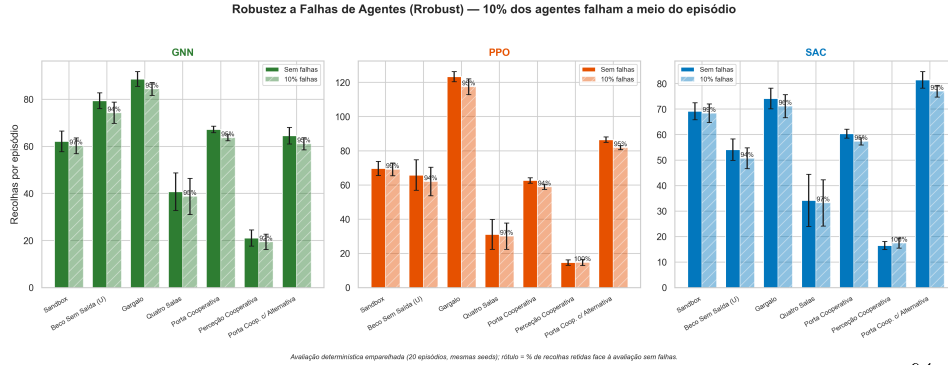


FIGURA 6.23. Resiliência do enxame perante a perda súbita de 10% dos agentes a meio do episódio (R_{robust}).

6.13. Desempenho Computacional do Simulador

A viabilidade da campanha experimental depende do *throughput* do simulador, medido em execução de referência *single-thread* (uma arena, $N = 20$ agentes, 3 000 passos com ações aleatórias na máquina de desenvolvimento; protocolo automatizado em `scripts/benchmark_sim.py`). A implementação original sustentava cerca de **139 passos de simulação por segundo** ($\approx 2\,770$ atualizações de agente por segundo; $\approx 3,6$ s por episódio de 500 passos); após a vetorização descrita abaixo, a mesma medição sustenta **≈ 420 passos/s** ($\approx 8\,400$ agente-passos/s; $\approx 1,2$ s por episódio) — um ganho de $\approx 3,0\times$, consistente com a aceleração de $2,58\times$ medida isoladamente no custo do passo (Tabela 6.5).

O treino dos métodos de gradiente escapa a este limite por **paralelização**: PPO e SAC executam 16 clones independentes da arena em processos paralelos (`SubprocVecEnv`), recolhendo experiência concorrentemente nos vários núcleos dos servidores de cálculo; o controlador evolutivo, por sua vez, avalia os 30 genomas da população em paralelo. Adicionalmente, antes da campanha final o próprio simulador foi **vetorizado** com NumPy: o varrimento LiDAR em lote (todos os agentes $\times$ todos os raios numa única operação matricial, aceleração de $19,5\times$ nessa componente) e o cálculo em lote das observações egocêntricas ($2,58\times$ no custo total do passo de simulação). Ambas as otimizações foram validadas por testes de equivalência *bit-exata* contra a implementação de referência (zero discrepâncias em dezenas de milhares de comparações), garantindo que aceleram a experimentação sem alterar a tarefa — no caso do controlador evolutivo, o ganho no custo do passo de simulação ($2,58\times$) traduz-se num aumento aproximadamente proporcional do número de gerações avaliadas no mesmo orçamento de treino.

TABELA 6.5. Desempenho computacional do simulador (*throughput* de referência, $N = 20$).

Configuração	<i>Throughput</i>
Single-thread, pré-vetorização (1 arena)	≈ 139 passos/s ($\approx 2\,770$ ag.-passo/s)
Single-thread, pós-vetorização (1 arena)	≈ 420 passos/s ($\approx 8\,400$ ag.-passo/s)
Episódio completo (500 passos), pós-vetorização	$\approx 1,2$ s
Treino vetorizado (SubprocVecEnv, 16 arenas)	paralelização $\approx n.^o$ de núcleos

6.14. Discussão Global e Validação da Hipótese

A hipótese central desta investigação postula que a *inteligência adaptativa* (MARL) supera a *robustez estática* bio-inspirada em cenários de *stress* dinâmico. Os resultados da campanha final exigem uma resposta diferenciada, e não um veredicto único.

Desempenho de tarefa. Para quantificar as diferenças entre algoritmos aplicou-se o teste não-paramétrico de Mann-Whitney U sobre as **médias de recolhas por *run*** ($n = 7$ por grupo — a unidade estatística independente é o *run* de treino, não o episódio), com o δ de Cliff como tamanho de efeito; a Tabela 6.6 reporta os resultados.

TABELA 6.6. Significância das diferenças em recolhas por episódio (Mann-Whitney U sobre as médias por *run*, $n = 7$ por algoritmo; δ de Cliff; $\alpha = 0,05$).

Cenário	Par	média A	média B	p	δ	Signif.
Sandbox	GNN vs PPO	38,31	71,46	0,0006	-1,00	Sim
Sandbox	GNN vs SAC	38,31	69,24	0,0021	-1,00	Sim
Sandbox	PPO vs SAC	71,46	69,24	0,0297	+0,71	Sim
Beco Sem Saída (U)	GNN vs PPO	24,54	39,62	0,1552	-0,47	não
Beco Sem Saída (U)	GNN vs SAC	24,54	8,99	0,5714	+0,18	não
Beco Sem Saída (U)	PPO vs SAC	39,62	8,99	0,0526	+0,63	não
Gargalo	GNN vs PPO	121,39	123,17	0,2086	+0,43	não
Gargalo	GNN vs SAC	121,39	41,44	0,0006	+1,00	Sim
Gargalo	PPO vs SAC	123,17	41,44	0,0006	+1,00	Sim
Quatro Salas	GNN vs PPO	59,76	33,58	0,0006	+1,00	Sim
Quatro Salas	GNN vs SAC	59,76	31,82	0,0006	+1,00	Sim
Quatro Salas	PPO vs SAC	33,58	31,82	0,3374	+0,33	não
Porta Cooperativa	GNN vs PPO	69,81	67,06	0,0262	+0,71	Sim
Porta Cooperativa	GNN vs SAC	69,81	62,09	0,0006	+1,00	Sim
Porta Cooperativa	PPO vs SAC	67,06	62,09	0,0175	+0,76	Sim
Perceção Cooperativa	GNN vs PPO	18,98	15,33	0,0297	+0,71	Sim
Perceção Cooperativa	GNN vs SAC	18,98	16,11	0,0297	+0,71	Sim
Perceção Cooperativa	PPO vs SAC	15,33	16,11	0,0547	-0,63	não
Porta c/ Alternativa	GNN vs PPO	86,70	85,25	0,8477	+0,08	não
Porta c/ Alternativa	GNN vs SAC	86,70	68,61	0,0006	+1,00	Sim
Porta c/ Alternativa	PPO vs SAC	85,25	68,61	0,0021	+1,00	Sim

A leitura da tabela desfaz a narrativa simples de um paradigma dominante. O controlador **evolutivo é significativamente superior a ambos os métodos de gradiente** em três cenários (Quatro Salas, Porta Cooperativa e Perceção Cooperativa, com $\delta \geq +0,71$) e superior ao SAC em mais dois (Gargalo e Porta com Alternativa, $\delta = +1,00$), empatando com o PPO nesses dois. Os métodos de gradiente são superiores no **Sandbox** (onde os *runs* degenerados do GNN pesam, $\delta = \pm 1,00$). No **Muro U**, nenhuma comparação atinge significância: com desempenho bimodal nos três algoritmos, sete *runs* não bastam para distinguir taxas de convergência de 2/7 a 4/7 — a conclusão honesta é que o cenário permanece por resolver de forma fiável por qualquer dos três algoritmos base. É este impasse que a hibridização com procura por novidade veio a quebrar (7/7 *runs*, Secção 6.10).

Anatomia do (antigo) colapso evolutivo — e da sua cura. Nas campanhas exploratórias, o controlador evolutivo colapsava em todos os cenários de gargalo, e a mecânica desse colapso merece registo porque fundamenta a QI5. A neuroevolução otimiza os pesos

a partir de um único escalar de *fitness* por episódio, sem atribuição de crédito temporal: não dispõe de informação sobre *qual* ação, em *que* instante, contribuiu para o resultado. Com recompensa de tarefa esparsa, a quase totalidade dos genomas recolhe zero unidades, a *fitness* reduz-se ao termo de *shaping* e — na formulação inicial, saturada por uma *tanh* do retorno acumulado — converge para um valor comum a toda a população: um *planalto de fitness* onde a pressão seletiva se anula e a evolução deambula (*fitness exploitation*). A cura não foi mais orçamento, mas **melhor desenho da *fitness***: a substituição do retorno acumulado (farmável por vaguear) pelo *homing* terminal — que só é maximizável *terminando* junto ao ninho (Eq. 2.1) — restaurou o gradiente de seleção nos labirintos. O efeito é visível na campanha final: 28/28 *runs* convergentes nos quatro cenários de gargalo, com magnitudes que igualam ou superam os métodos de gradiente. O “colapso do evolutivo” era, portanto, um artefacto do desenho da *fitness*, e não uma limitação intrínseca do paradigma — com a ressalva de que esta sensibilidade ao desenho é, em si mesma, um custo de engenharia do paradigma evolutivo que os métodos de gradiente, com atribuição de crédito passo-a-passo, não pagam na mesma medida. Esta centralidade do desenho do sinal de treino encontra, aliás, convergência independente no grupo responsável pelo único estudo comparativo da revisão sistemática: no seu trabalho dedicado às estruturas de recompensa, Iskandar conclui igualmente que o desenho do sinal — esparsa, modelado ou aprendido por demonstração — é o fator que mais condiciona o comportamento e a aprendizagem do enxame [45].

Variância entre execuções. A distribuição da variância pelos cenários é, ela própria, informativa. O controlador evolutivo é *quase determinístico* nos labirintos com gradiente geodésico informativo (desvios de 1–2 recolhas/ep na Porta Cooperativa e na Porta com Alternativa), mas bimodal nos cenários abertos (Sandbox 5/7): sem paredes que canalizem o *homing*, a paisagem de *fitness* oferece ótimos locais de vaguear dos quais alguns *runs* não escapam. O SAC exhibe o padrão inverso — estável em espaço aberto, lotaria nos gargalos físicos (Gargalo $41,4 \pm 36,8$, 5/7) —, sugerindo que a exploração por entropia máxima, eficaz em espaço aberto, não gera com fiabilidade as trajetórias longas e coordenadas que atravessam passagens estreitas. O PPO é o mais estável dos três, com um único cenário bimodal (Muro U, 4/7). O Muro U, transversalmente bimodal, isola o ingrediente que falta a todos: nenhum dos mecanismos de exploração estudados (entropia, *clipped surrogate*, mutação gaussiana) garante a descoberta do desvio comprido. Note-se que o obstáculo não está no sinal de treino — o potencial geodésico aponta corretamente para o contorno (Secção 4.3) —, mas na *descoberta*: o agente observa apenas a bússola de *homing* euclidiana e o LiDAR local, pelo que percorrer o desvio exige uma trajetória longa que se afasta, durante dezenas de passos, da única direção que o agente vê como o ninho. É precisamente esta a motivação da linha de *Novelty Search* (Secção 6.10).

Custo computacional e eficiência. A competitividade do evolutivo na tarefa tem um preço de cómputo: cada *run* do GNN consumiu 195 minutos com ≈ 30 núcleos (avaliação paralela da população), contra 48 minutos com 16 núcleos do PPO/SAC — uma razão

de $\approx 8\times$ em núcleos-hora. A comparação de desempenho atingível (Secção 6.1) é válida porque ambos os regimes atingem o planalto; mas em *eficiência amostral e computacional*, os métodos de gradiente mantêm uma vantagem clara, que deve pesar na escolha de paradigma quando o orçamento de treino é o recurso escasso.

Escalabilidade. No eixo da transferência *Zero-Shot* (Secção 6.11), o controlador evolutivo com atenção sobre grafo é o *único* viável, mantendo 100% de sucesso de $N = 10$ a $N = 100$ sem retreino **nos sete cenários** — com a eficiência per capita em $N = 100$ a reter 58–90% nos cenários com paredes (contra 39–45% nos abertos), onde um colapso por congestionamento seria a expectativa natural — enquanto as políticas MLP do PPO/SAC são estruturalmente incompatíveis com $N \neq 20$. A adaptabilidade que a hipótese atribuíra ao MARL manifesta-se, assim, não no algoritmo de otimização, mas na *representação* (grafo com atenção vs. vetor de dimensão fixa).

Robustez. Perante falhas de 10% dos agentes, todos os paradigmas exibem resiliência elevada (retenção de 92–106%, Secção 6.12), sem o colapso que a hipótese poderia antecipar para o controlador estático — o *parameter sharing* confere redundância funcional transversal.

Veredicto. A hipótese confirma-se apenas *parcialmente*, e o quadro final é mais rico do que o antecipado. Com o desenho adequado da *fitness*, o controlador evolutivo com atenção sobre grafo deixou de ser o elo fraco na tarefa: é estatisticamente superior aos métodos de gradiente em três dos sete cenários, indistinguível do melhor deles em mais dois, e é o único a escalar para dimensões de enxame não vistas. Os métodos de gradiente preservam duas vantagens sólidas: a **fiabilidade em espaço aberto** (onde o evolutivo ocasionalmente degenera) e a **eficiência computacional** ($\approx 8\times$ menos núcleos-hora por *run*). O resultado mais valioso para a engenharia de enxames não é eleger um vencedor, mas mapear estes eixos de escolha — estrutura do cenário, variabilidade da dimensão do enxame e orçamento de cómputo — e assinalar o resultado que fecha o quadro: a exploração fiável sob deceção espacial (Muro U), que nenhum dos três algoritmos base resolveu de forma consistente, só cedeu à hibridização com pressão por novidade (7/7 *runs*, Secção 6.10) — um mecanismo que, no entanto, degrada o desempenho onde o gradiente da tarefa já basta, reforçando a tese central de que a escolha (e a dosagem) dos ingredientes de treino deve responder à estrutura do cenário.

[Página intencionalmente deixada em branco.]

Conclusões e Trabalhos Futuros

7.1. Conclusões

Esta dissertação incidiu sobre um dos desafios mais prementes na robótica contemporânea: a definição de paradigmas de controlo que garantam a operabilidade de enxames em ambientes de alta incerteza. Identificou-se uma lacuna crítica na literatura — o isolamento metodológico entre as comunidades de *Multi-Agent Reinforcement Learning* (MARL) e de Otimização Bio-inspirada — e respondeu-se a essa lacuna com um **benchmarking rigoroso e imparcial**: desenvolveu-se um simulador de alta fidelidade, implementou-se o *framework* RS2C (com PPO e SAC) e um controlador neuroevolutivo assente numa rede de grafos com atenção, e compararam-se os três algoritmos nos mesmos sete cenários, com 7 execuções independentes por combinação, sob avaliação determinística emparelhada e tratamento estatístico não-paramétrico.

Os resultados (Capítulo 6) não elegem um vencedor universal, mas desenham um mapa de escolha claro. No cumprimento efetivo da tarefa, e ao contrário do que as campanhas exploratórias sugeriam, o controlador evolutivo com *fitness* de *homing* é **estatisticamente superior** aos métodos de gradiente em três cenários (Quatro Salas, Porta Cooperativa e Perceção Cooperativa) e indistinguível do PPO nos restantes cenários de gargalo, convergindo em todos os *runs* dos quatro cenários de gargalo; o antigo “colapso do evolutivo” revelou-se um artefacto do desenho da *fitness* (*fitness exploitation*), curado pela substituição do retorno acumulado pelo *homing* terminal. Os métodos de gradiente preservam a fiabilidade em espaço aberto (onde o evolutivo ocasionalmente degenera) e uma vantagem de $\approx 8\times$ em núcleos-hora por execução. Na transferência **Zero-Shot**, só a arquitetura de atenção sobre grafo escala de $N = 10$ a $N = 100$ sem retreino, mantendo 100% de sucesso, enquanto as políticas MLP de dimensão fixa do PPO/SAC são estruturalmente incompatíveis com $N \neq 20$ — a adaptabilidade que a hipótese atribuíra ao MARL manifesta-se, afinal, na *representação*, não no algoritmo de otimização. Perante falhas de 10% dos agentes, todos os paradigmas exibem resiliência elevada (retenção de 92–106%), conferida pela partilha de parâmetros e pela observação local. O cenário de deceção espacial (Muro U), bimodal para os três algoritmos base, só foi resolvido de forma consistente pela hibridização da neuroevolução com procura por novidade (7/7 execuções com orçamento igualado) — mecanismo que, simetricamente, degradou o desempenho no cenário *deceptive* que o objetivo puro já resolvia, mostrando que a exploração por novidade é um instrumento direcionado e não um melhoramento universal.

Confirma-se, assim, apenas *parcialmente* a hipótese: a escolha do paradigma deve ser ditada pelos eixos de exigência dominantes da aplicação — estrutura do cenário e

orçamento de cómputo (favoráveis ao gradiente em espaço aberto e a orçamentos curtos) versus navegação estruturada e escalabilidade a dimensões não vistas (favoráveis ao controlador de grafo com *fitness* bem desenhada). Estas conclusões assentam em 7 execuções independentes por configuração, com significância avaliada ao nível do *run*; o alargamento da bateria (Secção 7.5) refinará as estimativas de variância, em particular nos cenários bimodais.

7.2. Resposta às Questões de Investigação

Retomam-se as questões de investigação enunciadas na Secção 1.4, sintetizando as respostas encontradas:

- QI1 — Desempenho de tarefa.:** Não há domínio de paradigma: com a *fitness* de *homing*, o controlador neuroevolutivo é estatisticamente superior aos métodos de gradiente em três dos sete cenários (incluindo os dois cooperativos com magnitude comparável) e indistinguível do PPO nos restantes cenários de gargalo; o PPO é o mais consistente entre *runs* e os métodos de gradiente mantêm vantagem clara nos cenários abertos, onde o evolutivo ocasionalmente degenera. O único cenário por resolver de forma fiável — por qualquer algoritmo — é o de decepção espacial (Muro U) (Capítulo 6).
- QI2 — Escalabilidade.:** Sim, mas apenas para a arquitetura de grafo: o controlador com atenção transfere de $N = 10$ a $N = 100$ agentes sem retreino, com **100% de sucesso nas 28 combinações** cenário $\times$ dimensão — incluindo os labirintos e as tarefas cooperativas, onde a objecção do congestionamento seria mais plausível. A eficiência per capita em $N = 100$ retém 58–90% nos cenários com paredes (máximo nas duas Portas Cooperativas, mínimo no Gargalo, onde a passagem única impõe algum custo de fluxo) e dilui-se nos abertos (39% no Sandbox, 45% na Percepção Cooperativa), por partilha de um recurso finito e não por falha de coordenação. As políticas MLP de dimensão fixa do PPO/SAC são estruturalmente incompatíveis com $N \neq 20$. A capacidade de *Zero-Shot Transfer* revelou-se, portanto, uma propriedade da *representação* (invariância à permutação do grafo), e não do algoritmo de optimização.
- QI3 — Robustez a falhas.:** Todos os paradigmas exibem resiliência elevada à falha súbita de 10% dos agentes, com retenção de desempenho próxima da integral. Esta robustez é conferida pela partilha de parâmetros e pela observação local, que tornam o comportamento coletivo independente da identidade (e da sobrevivência) de agentes individuais.
- QI4 — Critério de escolha.:** Não há um vencedor universal: em exames de dimensão fixa e conhecida, com prioridade na eficiência da tarefa, o MARL por gradiente é preferível; quando a dimensão do exame é variável ou desconhecida à partida, ou a missão exige escalar sem retreino, a arquitetura de grafo com atenção (aqui optimizada por neuroevolução) é a única opção viável entre as estudadas. A robustez a falhas, sendo transversal, não constitui critério discriminante.

QI5 — Desenho da função de *fitness*.: Sim, de forma decisiva: o desempenho da neuroevolução revelou-se mais sensível ao desenho da *fitness* do que ao orçamento de treino. Com a *fitness* inicial — recolha mais o retorno acumulado do episódio, *farmável* por deambulação —, o controlador colapsa nos cenários de navegação complexa; a substituição desse termo pelo *homing* terminal (proximidade ao ninho no *fim* do episódio, medida no potencial geodésico, e por isso não-farmável) desbloqueou os quatro cenários de gargalo, elevando a taxa de sucesso de 0% para 100% na totalidade dos 28 *runs* correspondentes e tornando o controlador evolutivo competitivo — e, em cenários seleccionados, superior — face aos métodos de gradiente (Capítulo 6). O “colapso do evolutivo” é, portanto, um artefacto do desenho da *fitness*, e não uma limitação intrínseca do paradigma. A ressalva é o Muro em U: mesmo com o potencial geodésico a apontar correctamente para o desvio, o desempenho permanece bimodal (3/7 *runs*), tal como sucede nos métodos de gradiente. O desenho da *fitness* é, pois, condição necessária mas não suficiente — resolve a atribuição de crédito, não a descoberta do desvio comprido, que veio a exigir pressão explícita por novidade (QI6).

QI6 — Deceção e procura por novidade.: Sim onde a deceção efetivamente morde — e com custo onde não morde. Nas campanhas com orçamento igualado (7 *runs* de 195 minutos por braço), a hibridização com *Novelty Search* ($w = 0,5$) resolveu o Muro em U de forma consistente — 7/7 *runs* a 100% de sucesso e $69,8 \pm 5,9$ recolhas/ep, contra 3/7 e $24,5 \pm 32,6$ do objetivo puro (Mann–Whitney $p = 0,026$, $\delta = +0,71$) —, tornando-se a única configuração estudada a dominar o cenário que resistia aos três algoritmos base. Simetricamente, na Porta com Alternativa, onde o *homing* já convergia em 7/7 *runs*, a mesma pressão por novidade degradou a magnitude ($63,0 \pm 21,9$ vs. $86,7 \pm 2,0$; $p = 0,0006$, $\delta = -1,00$), desmascarando o +26% da comparação preliminar como artefacto do orçamento triplo. A resposta é, pois, condicional: a procura por novidade melhora o resultado final quando a otimização objetiva é inconsistente (paisagem bimodal) e prejudica-o quando o gradiente da tarefa basta — a exploração tem preço, e a dosagem adaptativa do peso w é a extensão natural (Secção 6.10).

7.3. Limitações do Estudo

Em prol da transparência científica, identificam-se as seguintes limitações da implementação atual:

- **Arquitetura assimétrica entre controladores:** os *baselines* MARL (PPO e SAC) empregam uma política totalmente ligada (MLP) sobre a observação de vizinhança densa, enquanto o mecanismo de atenção sobre grafo é instanciado apenas no controlador neuroevolutivo. Esta opção — ditada pelo uso das implementações de referência da *Stable-Baselines3* — implica que a comparação entre paradigmas combina dois fatores (algoritmo de otimização e arquitetura da rede). A integração do módulo de atenção numa política treinável por gradiente

(*custom policy*) é a extensão prioritária de trabalho futuro, permitindo isolar o efeito do otimizador.

- **Escalabilidade restrita ao controlador de grafo:** como consequência do ponto anterior, o *Zero-Shot Transfer* para $N \neq 20$ só é diretamente avaliável no controlador com atenção; a MLP de entrada fixa do PPO/SAC é, por construção, incompatível com outras dimensões de enxame.
- **Crítico não-centralizado:** a fase de treino utiliza um crítico que parte da mesma observação local do ator (*parameter sharing*), não explorando o estado global. Trata-se de execução e treino descentralizados, e não de uma arquitetura CTDE com crítico centralizado em sentido estrito.
- ***Sim-to-real* não validado:** a totalidade dos resultados é obtida em simulação; a transferência para plataformas físicas (*Deployment Gap*) permanece por validar empiricamente.

7.4. Contributos

Do trabalho realizado resultam os seguintes contributos para a área de Sistemas Multiagente e Robótica de Enxame:

- ***Benchmarking* quantitativo entre paradigmas:** estabeleceu-se uma metodologia de comparação padronizada entre algoritmos de naturezas distintas (aprendizagem por gradiente *vs.* otimização evolutiva), aplicada a 3 controladores $\times$ 7 cenários $\times$ 7 execuções independentes, sob avaliação determinística emparelhada (20 episódios com sementes fixas) e inferência não-paramétrica ao nível do *run* (Mann-Whitney U com δ de Cliff). A unidade estatística é a execução de treino — e não o episódio —, o que evita a inflação de n que é comum na literatura comparada.
- **O desenho da *fitness* como causa (e cura) do colapso evolutivo:** demonstrou-se que o colapso da neuroevolução nos cenários de navegação complexa — reportado como limitação do paradigma — era, neste sistema, um *artefacto do sinal de treino*: com uma *fitness* baseada no retorno acumulado, a população satura num planalto onde a pressão seletiva se anula (*fitness exploitation*). A substituição pelo *homing* terminal, que só é maximizável terminando junto ao ninho, restaurou o gradiente de seleção e levou o controlador a convergir em **28/28 execuções** nos quatro cenários de gargalo (Gargalo, Quatro Salas e as duas Portas Cooperativas), passando de 0% a 100% de sucesso. Este é o principal contributo metodológico do trabalho.
- **Escalabilidade *Zero-Shot* como propriedade da representação:** mostrou-se empiricamente que o controlador com atenção sobre grafo transfere de $N = 10$ para $N = 100$ agentes sem retreino e sem perda de sucesso (100% nas 28 combinações cenário $\times$ dimensão), retendo entre 58% e 90% da eficiência per capita nos cenários estruturados, enquanto as políticas MLP de dimensão fixa são estruturalmente incompatíveis com $N \neq 20$ — respondendo ao desafio de

escalabilidade colocado por Chen et al. (2024) [15] e localizando a adaptabilidade na *arquitetura*, não no algoritmo de otimização.

- **Caracterização condicional da procura por novidade:** quantificou-se, com orçamento de treino igualado, que a pressão por novidade resolve o cenário de decação espacial que resistia aos três algoritmos base (7/7 execuções, contra 3/7 do objetivo puro) mas *degrada* o desempenho onde o gradiente da tarefa já é informativo. A exploração é, assim, um instrumento direcionado e não um melhoramento universal — um resultado que contraria a leitura otimista que uma comparação com orçamentos desiguais sugeriria.
- **Mapa de escolha para engenharia de enxames:** em vez de eleger um vencedor universal, o trabalho mapeia os eixos que devem ditar a escolha do paradigma — estrutura do cenário, variabilidade da dimensão do enxame e orçamento de cómputo ($\approx 8\times$ mais núcleos-hora para o evolutivo) — sintetizados na discussão do Capítulo 6.
- **Implementação de referência extensível:** disponibiliza-se um repositório aberto (Apêndice D) com o simulador, os sete cenários, o *framework* RS2C (PPO/SAC) e o controlador neuroevolutivo com atenção. A arquitetura foi organizada para que a extensão seja barata onde importa (um algoritmo novo exige uma classe, um módulo de treino e uma entrada num registo, passando automaticamente a integrar campanhas, avaliação, estatística e figuras), com as garantias de não-regressão e a dívida técnica remanescente declaradas explicitamente na Secção 4.5 — de modo que quem continue este trabalho saiba, à partida, o que herda.

7.5. Trabalhos Futuros

A partir das limitações identificadas (Secção 7.3) e do estado atual da experimentação, delineiam-se as seguintes linhas de trabalho prioritárias:

- (1) **Alargamento da bateria estatística:** Estender as 7 execuções independentes por configuração da campanha final (Secção 6.6) para 30, com prioridade para os cenários de comportamento bimodal (Muro U, Sandbox-GNN, Gargalo-SAC), onde $n = 7$ limita o poder estatístico para distinguir taxas de convergência — no Muro U, nenhuma diferença entre algoritmos atingiu significância.
- (2) **Política de gradiente com atenção (*custom policy*):** Integrar o mecanismo de atenção sobre grafo numa política treinável por gradiente (PPO/SAC com extrator de *features* personalizado na *Stable-Baselines3*), de modo a *isolar* o efeito do algoritmo de otimização do efeito da arquitetura — removendo a assimetria MLP vs. grafo que atualmente confunde a comparação (Secção 7.3).
- (3) **Novos cenários e tarefas:** Implementar tarefas de cobertura/vigilância de área (para além do *foraging* e do cerco de alvos móveis) e cenários com percursos alternativos, ampliando o espaço de avaliação de comportamentos coletivos.

- (4) **Validação *Sim-to-Real*:** Transferir as políticas para plataformas físicas (e.g. e-puck, Khepera IV) ou para um simulador de física de maior fidelidade, quantificando empiricamente o *Deployment Gap*.
- (5) **Hibridização de paradigmas:** Explorar arquiteturas que combinem a escalabilidade do controlador evolutivo com a eficiência de tarefa dos métodos de gradiente (e.g. inicialização evolutiva seguida de afinação por RL).
- (6) **Ponderação adaptativa da novidade:** O peso fixo $w = 0,5$ explica tanto o ganho como o custo observados na Secção 6.10; uma dosagem adaptativa — por exemplo, w decrescente com o progresso do objetivo, ou reativado quando a população estagna — poderia reter a fuga aos ótimos locais (Muro U) sem sacrificar magnitude onde o gradiente basta (Porta com Alternativa).
- (7) **Aprendizagem por currículo como via alternativa de descoberta:** O grupo responsável pelo único estudo da revisão sistemática que compara os dois paradigmas mitiga o sobreajuste do RL a condições não vistas com aprendizagem por currículo [45]. Aplicá-la aos cenários decetivos desta bateria — arenas progressivamente mais fechadas, culminando no Muro U e na Porta com Alternativa — permitiria confrontar currículo e pressão por novidade, dois mecanismos distintos para o mesmo problema (a *descoberta*), sob o protocolo estatístico e o orçamento igualado aqui estabelecidos.

Referências

- [1] M. Hüttenrauch, A. Šošić e G. Neumann, «Deep Reinforcement Learning for Swarm Systems,» *Journal of Machine Learning Research*, vol. 20, n.º 54, pp. 1–31, 2019.
- [2] M. Tegmark, *Life 3.0: Being Human in the Age of Artificial Intelligence*. Alfred A. Knopf, 2017.
- [3] M. Dorigo, «Optimization, Learning and Natural Algorithms,» tese de doutoramento, Politecnico di Milano, 1992.
- [4] J. Kennedy e R. Eberhart, «Particle swarm optimization,» em *Proceedings of ICNN'95 - International Conference on Neural Networks*, IEEE, vol. 4, 1995, pp. 1942–1948.
- [5] C. C. Ekechi, T. Elfouly, A. Alouani e T. Khattab, «A Survey on UAV Control with Multi-Agent Reinforcement Learning,» *Drones*, vol. 9, n.º 7, p. 484, 2025.
- [6] R. Pina, V. De Silva e C. Artaud, «Towards Self-Adaptive Resilient Swarms Using Multi-Agent Reinforcement Learning,» em *Proceedings of the 13th International Conference on Pattern Recognition Applications and Methods (ICPRAM)*, SciTePress, 2024, pp. 410–417. DOI: [10.5220/0012462800003654](https://doi.org/10.5220/0012462800003654)
- [7] A. Y. Majid, S. Saaybi, V. Francois-Lavet, R. V. Prasad e C. Verhoeven, «Deep Reinforcement Learning Versus Evolution Strategies: A Comparative Survey,» *IEEE Transactions on Neural Networks and Learning Systems*, vol. 35, n.º 9, pp. 11 939–11 957, 2024. DOI: [10.1109/TNNLS.2023.3264540](https://doi.org/10.1109/TNNLS.2023.3264540)
- [8] W. M. Hassen e S. H. Amin, «Dynamic Window Approach for Mobile Robot Path Planning based on Hybrid Swarm Algorithms,» *Journal of Information Hiding and Multimedia Signal Processing*, vol. 16, n.º 2, 2025.
- [9] T. Salimans, J. Ho, X. Chen, S. Sidor e I. Sutskever, «Evolution Strategies as a Scalable Alternative to Reinforcement Learning,» *arXiv preprint arXiv:1703.03864*, 2017.
- [10] S. Somvanshi, M. M. Islam, S. A. Javed, G. Chhetri, K. S. Islam, T. I. Chowdhury, S. B. B. Pollock, A. Dutta e S. Das, «A Review on Influx of Bio-Inspired Algorithms: Critique and Improvement Needs,» *arXiv preprint arXiv:2506.04238*, 2025.
- [11] R. S. Sutton e A. G. Barto, *Reinforcement Learning: An Introduction*, 2ª ed. MIT Press, 2018.
- [12] D. Huh e P. Mohapatra, «Multi-agent Reinforcement Learning: A Comprehensive Survey,» *arXiv preprint arXiv:2312.10256*, 2023.

- [13] M. Bettini, A. Prorok e V. Moens, «BenchMARL: Benchmarking Multi-Agent Reinforcement Learning,» *Journal of Machine Learning Research*, vol. 25, n.^o 217, pp. 1–10, 2024.
- [14] M. Kegeleirs e M. Birattari, «Towards applied swarm robotics: current limitations and enablers,» *Frontiers in Robotics and AI*, vol. 12, 2025. DOI: [10.3389/frobt.2025.1607978](https://doi.org/10.3389/frobt.2025.1607978)
- [15] X. Chen, N. NaderiAlizadeh, A. Ribeiro e S. Saeedi Bidokhti, «Transferable Graphical MARL for Real-Time Estimation in Dynamic Wireless Networks,» *arXiv preprint arXiv:2404.03227*, 2024.
- [16] B. Rivière, W. Hönig, Y. Yue e S.-J. Chung, «GLAS: Global-to-Local Safe Autonomy Synthesis for Multi-Robot Motion Planning with End-to-End Learning,» *IEEE Robotics and Automation Letters*, vol. 5, n.^o 3, pp. 4249–4256, 2020. DOI: [10.1109/LRA.2020.2994035](https://doi.org/10.1109/LRA.2020.2994035)
- [17] E. Bonabeau, M. Dorigo e G. Theraulaz, *Swarm Intelligence: From Natural to Artificial Systems* (Santa Fe Institute Studies in the Sciences of Complexity). Oxford University Press, 1999.
- [18] F. A. Oliehoek e C. Amato, *A Concise Introduction to Decentralized POMDPs*. Springer, 2016.
- [19] J. Schulman, F. Wolski, P. Dhariwal, A. Radford e O. Klimov, «Proximal Policy Optimization Algorithms,» *arXiv preprint arXiv:1707.06347*, 2017.
- [20] J. Schulman, P. Moritz, S. Levine, M. I. Jordan e P. Abbeel, «High-Dimensional Continuous Control Using Generalized Advantage Estimation,» em *International Conference on Learning Representations (ICLR)*, 2016.
- [21] T. Haarnoja, A. Zhou, P. Abbeel e S. Levine, «Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor,» em *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018, pp. 1861–1870.
- [22] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel e I. Mordatch, «Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments,» em *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [23] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser e I. Polosukhin, «Attention is All You Need,» em *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [24] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò e Y. Bengio, «Graph Attention Networks,» em *International Conference on Learning Representations (ICLR)*, 2018.
- [25] H. Zhang, J. Cheng, L. Zhang, Y. Li e W. Zhang, «H2GNN: Hierarchical-Hops Graph Neural Networks for Multi-Robot Exploration in Unknown Environments,» *IEEE Robotics and Automation Letters*, vol. 7, n.^o 2, pp. 3435–3442, 2022. DOI: [10.1109/LRA.2022.3146912](https://doi.org/10.1109/LRA.2022.3146912)

- [26] Y. Wang, T. Duhan, J. Li e G. Sartoretti, «LNS2+RL: Combining Multi-Agent Reinforcement Learning with Large Neighborhood Search in Multi-Agent Path Finding,» em *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, arXiv:2405.17794, 2025.
- [27] P. Li, J. Hao, H. Tang, Y. Zheng e X. Fu, «RACE: Improve Multi-Agent Reinforcement Learning with Representation Asymmetry and Collaborative Evolution,» em *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023, pp. 19 490–19 503.
- [28] J. Lehman e K. O. Stanley, «Abandoning Objectives: Evolution Through the Search for Novelty Alone,» *Evolutionary Computation*, vol. 19, n.º 2, pp. 189–223, 2011. DOI: [10.1162/EVC0_a_00025](https://doi.org/10.1162/EVC0_a_00025)
- [29] O. Yigit, S. E. Ceylan, S. S. Palas, A. Isik, E. Bekar, B. Vatansever e Y. Oniz, «Multi Agent Reinforcement Learning Based Swarm Navigation,» em *2026 8th International Congress on Human-Computer Interaction, Optimization and Robotic Applications (ICHORA)*, IEEE, 2026. DOI: [10.1109/ICHORA69329.2026.11537199](https://doi.org/10.1109/ICHORA69329.2026.11537199)
- [30] A. Iskandar, A. Hammoud e B. Kovács, «Swarm Robotics Navigation Task: A Comparative Study of Reinforcement Learning and Particle Swarm Optimization Methodologies,» *Mekhatronika, Automatizatsiya, Upravlenie*, vol. 25, n.º 9, pp. 471–478, 2024. DOI: [10.17587/mau.25.471-478](https://doi.org/10.17587/mau.25.471-478)
- [31] K. Hasselmann, A. Ligot e M. Birattari, «Automatic modular design of robot swarms based on repertoires of behaviors generated via novelty search,» *Swarm and Evolutionary Computation*, vol. 83, p. 101 395, 2023. DOI: [10.1016/j.swevo.2023.101395](https://doi.org/10.1016/j.swevo.2023.101395)
- [32] M. Kegeleirs, D. G. Ramos, K. Hasselmann, L. Garattoni, G. Francesca e M. Birattari, «Transferability in the Automatic Off-Line Design of Robot Swarms: From Sim-to-Real to Embodiment and Design-Method Transfer Across Different Platforms,» *IEEE Robotics and Automation Letters*, vol. 9, n.º 3, pp. 2758–2765, 2024. DOI: [10.1109/LRA.2024.3360013](https://doi.org/10.1109/LRA.2024.3360013)
- [33] J. Szpirer, D. G. Ramos e M. Birattari, «Automatic design of robot swarms that perform composite missions: an approach based on inverse reinforcement learning,» em *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, 2024. DOI: [10.1109/IROS58592.2024.10801506](https://doi.org/10.1109/IROS58592.2024.10801506)
- [34] T. U. Zaman, P. Biteng e Q. Lu, «Evolving Adaptive Foraging Robot Swarms with NEAT in Environments with Obstacles,» em *8th International Conference on Intelligent Robotics and Control Engineering (IRCE)*, IEEE, 2025. DOI: [10.1109/IRCE66030.2025.11203043](https://doi.org/10.1109/IRCE66030.2025.11203043)
- [35] P. Biteng, T. U. Zaman e Q. Lu, «Training Robot Swarms for Adaptive Foraging in Environments with Obstacles,» em *IEEE International Conference on Mechatronics and Automation (ICMA)*, IEEE, 2025. DOI: [10.1109/ICMA65362.2025.11120654](https://doi.org/10.1109/ICMA65362.2025.11120654)

- [36] E. Ordáz-Rivas, A. Rodríguez-Liñán e L. Torres-Treviño, «Multi-objective optimization in autonomous foraging using swarm robots,» *Swarm and Evolutionary Computation*, vol. 92, p. 102294, 2026. DOI: [10.1016/j.swevo.2026.102294](https://doi.org/10.1016/j.swevo.2026.102294)
- [37] N. Bredeche e N. Fontbonne, «Social learning in swarm robotics,» *Philosophical Transactions of the Royal Society B: Biological Sciences*, vol. 377, n.^o 1843, p. 20200309, 2022. DOI: [10.1098/rstb.2020.0309](https://doi.org/10.1098/rstb.2020.0309)
- [38] N. Fontbonne, O. Dauchot e N. Bredeche, «Distributed On-line Learning in Swarm Robotics with Limited Communication Bandwidth,» em *IEEE Congress on Evolutionary Computation (CEC)*, IEEE, 2020. DOI: [10.1109/CEC48606.2020.9185697](https://doi.org/10.1109/CEC48606.2020.9185697)
- [39] H. Tang, H. Zhang, Z. Shi, X. Chen, W. Ding e X.-P. Zhang, «Autonomous Swarm Robot Coordination via Mean-Field Control Embedding Multi-Agent Reinforcement Learning,» em *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, 2023. DOI: [10.1109/IROS55552.2023.10341749](https://doi.org/10.1109/IROS55552.2023.10341749)
- [40] X. Li, R. Zhou e G. Sun, «Adaptive Shape Formation Against Swarm-Scale Variants in Robot Swarms,» *IEEE Robotics and Automation Letters*, vol. 9, n.^o 8, 2024. DOI: [10.1109/LRA.2024.3414210](https://doi.org/10.1109/LRA.2024.3414210)
- [41] A. Iskandar, H. Rostum e B. Kovács, «Using Deep Reinforcement Learning to Solve a Navigation Problem for a Swarm Robotics System,» em *2023 24th International Carpathian Control Conference (ICCC)*, IEEE, 2023, pp. 185–189.
- [42] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba e P. Abbeel, «Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World,» em *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, IEEE, 2017, pp. 23–30.
- [43] A. Y. Ng, D. Harada e S. Russell, «Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping,» em *Proceedings of the 16th International Conference on Machine Learning (ICML)*, 1999, pp. 278–287.
- [44] W. Xiao, L. Yuan, T. Ran, L. He, J. Zhang e J. Cui, «CSPER: Curiosity-driven Safety-Prioritized Experience Replay for Mapless Navigation using Deep Reinforcement Learning,» *IEEE Transactions on Cognitive and Developmental Systems*, 2026.
- [45] A. Iskandar, «Deep Reinforcement Learning for Swarm Robotic Systems,» tese de doutoramento, University of Miskolc, István Sályi Doctoral School of Mechanical Engineering Sciences, 2025. DOI: [10.14750/ME.2026.007](https://doi.org/10.14750/ME.2026.007)

APÊNDICE A

Configurações e Hiperparâmetros

Para garantir a reprodutibilidade integral das experiências, este apêndice reúne a configuração completa do sistema, tal como definida no ficheiro `configs/foraging.yaml` do repositório (Apêndice D). Os valores não resultaram de uma pesquisa automática de hiperparâmetros (*grid/Optuna*), mas sim de **calibração manual** informada por testes de fumo (*smoke tests*) curtos e pelas escolhas de referência da literatura e das implementações da *Stable-Baselines3*; documentam-se aqui os valores efetivamente utilizados nos treinos reportados no Capítulo 6.

TABELA A.1. Configuração do ambiente de simulação e da física.

Parâmetro	Valor	Descrição
arena_radius	15.0 m	Raio da arena circular
num_agents	20	Número de agentes (N)
num_obstacles	100	Obstáculos dinâmicos
obstacle_radius	0.2 m	Raio dos obstáculos
obstacle_velocity	0.02 m/s	Velocidade dos obstáculos
nest_radius	1.5 m	Raio do ninho
nest_velocity	0.015 m/s	Velocidade do ninho móvel
required_to_eat	3	k_{min} nos cenários cooperativos (forçado a 1 nos labirintos de navegação)
hunger_timer_max	250	Limite de fome (só Sandbox)
lidar_range	8.0 m	Alcance máximo do LiDAR
max_steps	500	Horizonte temporal base
max_steps_cooperative_door	800	Horizonte na Porta Cooperativa
max_steps_cooperative_door_bypass	1000	Horizonte na Porta Coop. c/ Alternativa
progress_reward_factor	10.0	Fator de Progresso (PBRs)
geodesic_cell_size	0.4 m	Resolução do campo geodésico
exploration_bonus	0.5	Bônus por célula nova
exploration_cell_size	2.0 m	Resolução da grelha de exploração
min_spread_distance	1.0 m	Limiar de dispersão (física; sem sinal de recompensa)
spreading_penalty_factor	0.0	Penalização <i>anti-clustering</i> (desativada)
agent_radius	0.15 m	Raio do robô
collision_penalty	0.0	Penalização de colisão entre agentes (desativada)
obstacle_penalty	0.0	Penalização de colisão com obstáculo/parede (desativada)
food_collected	+300.0	Recompensa de tarefa (recolha)
energy_cost	-0.05	Custo energético por passo
guillotine_threshold	-200	Limiar da guilhotina (aos 150 passos)

TABELA A.2. Hiperparâmetros de treino dos três algoritmos.

Algoritmo	Hiperparâmetro	Valor
PPO	learning_rate	10^{-4}
	n_steps (por CPU)	64
	batch_size	512
	n_epochs	4
	net_arch	[256, 256]
	num_cpu (SubprocVecEnv)	16
	γ / λ_{GAE}	0.99 / 0.95
SAC	learning_rate	10^{-4}
	buffer_size	500 000
	ent_coef (α)	0.1 (fixo)
	net_arch	[256, 256]
	num_cpu	16
Evolutivo (AE)	pop_size (K)	30
	mutation_rate (p_{mut})	0.1
	sigma (σ)	0.1 ($\times 0.999$ /geração; $\sigma_{\min} = 0.03$)
	eval_episodes (E)	4 (seeds fixas)
	hidden_dim (h)	32 ($\approx$ 8k pesos)
	Elitismo	20% ($e = 6$ genomas)
	fitness_food_weight	10 000 (peso da recolha em J , Eq. 2.1)
	fitness_shaping_amplitude	5 000 (peso do <i>homing</i> terminal em J)
	novelty_weight (w)	0 (base) / 0,5 (campanhas de <i>Novelty Search</i>)
	novelty_k / arquivo	10 vizinhos / máx. 1000 (FIFO, +3 por geração)

[Página intencionalmente deixada em branco.]

APÊNDICE B

Estudos Incluídos na Revisão Sistemática

Lista integral dos estudos que sobreviveram ao fluxo PRISMA descrito na Secção 3.2 (Figura 3.1). A tabela é gerada automaticamente a partir do registo de triagem (`docs/slr/screening.csv`), onde constam também os 622 registos excluídos, cada um com o respetivo critério — o que permite auditar qualquer decisão desta revisão, e não apenas as inclusões.

Tabela B.1: Estudos incluídos na revisão sistemática ($n = 58$), resultantes do fluxo PRISMA (Figura 3.1).

#	Autores	Ano	Título	Publicação
1	Dai et al.	2020	Multi-Robot Dynamic Task Allocation for Exploration and Destruction	Journal of Intelligent and Robotic Systems: Theory and Applications
2	Das & Jena	2020	Multi-robot path planning using improved particle swarm optimization algorithm through novel evolutionary operators	Applied Soft Computing Journal
3	Fontbonne et al.	2020	Distributed On-line Learning in Swarm Robotics with Limited Communication Bandwidth	2020 IEEE Congress on Evolutionary Computation, CEC 2020 - Conference Proceedings
4	Mısır et al.	2020	Fuzzy-based self organizing aggregation method for swarm robots	BioSystems
5	Konda et al.	2020	Decentralized Function Approximated Q-Learning in Multi-Robot Systems For Predator Avoidance	IEEE Robotics and Automation Letters

#	Autores	Ano	Título	Publicação
6	Behjat et al.	2021	Learning Robot Swarm Tactics over Complex Adversarial Environments	2021 International Symposium on Multi-Robot and Multi-Agent Systems (MRS)
7	Rivière et al.	2021	Neural Tree Expansion for Multi-Robot Planning in Non-Cooperative Environments	IEEE Robotics and Automation Letters
8	Pagnozzi & Birattari	2021	Off-Policy Evaluation of the Performance of a Robot Swarm: Importance Sampling to Assess Potential Modifications to the Finite-State Machine That Controls the Robots	Frontiers in Robotics and AI
9	Zheng et al.	2021	Optimizing Parameters of Self-Organizing Model for Swarm Robots via Evolutionary Algorithms	2021 IEEE Congress on Evolutionary Computation (CEC)
10	Bredecche & Fontbonne	2022	Social learning in swarm robotics	Philosophical Transactions of the Royal Society B: Biological Sciences
11	Baumann & Martinoli	2022	A Noise-Resistant Mixed-Discrete Particle Swarm Optimization Algorithm for the Automatic Design of Robotic Controllers	2022 IEEE Congress on Evolutionary Computation (CEC)
12	Ebert et al.	2022	A Hybrid PSO Algorithm for Multi-robot Target Search and Decision Awareness	2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)
13	Tang et al.	2022	Learning to Coordinate for a Worker-Station Multi-Robot System in Planar Coverage Tasks	IEEE Robotics and Automation Letters

#	Autores	Ano	Título	Publicação
14	Zhang et al.	2022	Moving-Distance-Minimized PSO for Mobile Robot Swarm	IEEE Transactions on Cybernetics
15	Luo et al.	2022	Physarum Polycephalum Inspired Search Strategy for Multi-robot	Lecture Notes on Data Engineering and Communications Technologies
16	Stolfi & Danooy	2022	Optimising autonomous robot swarm parameters for stable formation design	GECCO 2022 - Proceedings of the 2022 Genetic and Evolutionary Computation Conference
17	Liu et al.	2022	Swarm Robots Decentralized Control using Reinforcement Learning Stepwise Training Method in the Encircling Task	2022 Global Conference on Robotics, Artificial Intelligence and Information Technology (GCRAIT)
18	Ayari & Bouamama	2023	Evolutionary Swarm Robotics: A Methodological Approach For Task and Path Planning	2023 IEEE Afro-Mediterranean Conference on Artificial Intelligence (AMCAI)
19	Tan et al.	2023	Deep Reinforcement Learning for Decentralized Multi-Robot Exploration With Macro Actions	IEEE Robotics and Automation Letters
20	Tang et al.	2023	Autonomous Swarm Robot Coordination via Mean-Field Control Embedding Multi-Agent Reinforcement Learning	2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)

#	Autores	Ano	Título	Publicação
21	Hasselmann et al.	2023	Automatic modular design of robot swarms based on repertoires of behaviors generated via novelty search	Swarm and Evolutionary Computation
22	Cui et al.	2023	Scalable Task-Driven Robotic Swarm Control via Collision Avoidance and Learning Mean-Field Control	2023 IEEE International Conference on Robotics and Automation (ICRA)
23	Luo et al.	2023	Multi-robot Cooperative Search for Radioactive Sources Based on Improved Particle Swarm Optimization	2023 2nd International Conference on Robotics, Artificial Intelligence and Intelligent Control (RAIIC)
24	Sebastian et al.	2023	LEMURS: Learning Distributed Multi-Robot Interactions	Proceedings - IEEE International Conference on Robotics and Automation
25	Sharma et al.	2023	D2CoPlan: A Differentiable Decentralized Planner for Multi-Robot Coverage	2023 IEEE International Conference on Robotics and Automation (ICRA)
26	Chen et al.	2024	An Immune Algorithm with Dual-level Ranking Mutation for Large-scale Multi-objective Multi-robot Maritime Patrolling	ICNSC 2024 - 21st International Conference on Networking, Sensing and Control: Artificial Intelligence for the Next Industrial Revolution

#	Autores	Ano	Título	Publicação
27	Ramos & Bioratti	2024	Automatically designing robot swarms in environments populated by other robots: an experiment in robot shepherding	2024 IEEE International Conference on Robotics and Automation (ICRA)
28	Liu et al.	2024	Language-Guided Pattern Formation for Swarm Robotics with Multi-Agent Reinforcement Learning	2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)
29	Iskandar et al.	2024	Swarm Robotics Navigation Task: A Comparative Study of Reinforcement Learning and Particle Swarm Optimization Methodologies	Mekhatronika, Avtomatizatsiya, Upravlenie
30	Szipirer et al.	2024	Automatic design of robot swarms that perform composite missions: an approach based on inverse reinforcement learning	2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)
31	Eguchi et al.	2024	Robot Swarm Control Based on Smoothed Particle Hydrodynamics for Obstacle-Unaware Navigation	2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)
32	Kegeleirs et al.	2024	Transferability in the Automatic Off-Line Design of Robot Swarms: From Sim-to-Real to Embodiment and Design-Method Transfer Across Different Platforms	IEEE Robotics and Automation Letters

#	Autores	Ano	Título	Publicação
33	Makarov et al.	2024	Particle swarm optimization-based method of source seeking by small robotic groups in the presence of noises and communication constraints	2024 9th International Conference on Robotics and Automation Engineering (ICRAE)
34	Peter et al.	2024	Dynamic Subgoal Based Path Formation and Task Allocation: A NeuroFleets Approach to Scalable Swarm Robotics	2024 IEEE International Conference on Systems, Man, and Cybernetics (SMC)
35	He et al.	2024	Multi-spherical robot formation control based on pigeon swarm optimization	2024 3rd International Conference on Robotics, Artificial Intelligence and Intelligent Control (RAIIC)
36	Li et al.	2024	Adaptive Shape Formation Against Swarm-Scale Variants in Robot Swarms	IEEE Robotics and Automation Letters
37	Zhang et al.	2024	Self-Reconfigurable Hierarchical Frameworks for Formation Control of Robot Swarms	IEEE Transactions on Cybernetics
38	Zhou et al.	2024	D3G: Learning Multi-robot Coordination from Demonstrations	2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)
39	Wang et al.	2024	Expert System-Based Multiagent Deep Deterministic Policy Gradient for Swarm Robot Decision Making	IEEE Transactions on Cybernetics

#	Autores	Ano	Título	Publicação
40	A. et al.	2025	Hybrid Ant Colony and Q-Learning Algorithm for Swarm Robots: Path Planning and Collision Avoidance in Unknown Environments	2025 17th International Conference on Electronics, Computers and Artificial Intelligence (ECAI)
41	Calzolari et al.	2025	Reinforcement Learning Driven Multi-Robot Exploration via Explicit Communication and Density-Based Frontier Search	2025 IEEE International Conference on Robotics and Automation (ICRA)
42	Xu et al.	2025	Efficient Multi-Robot Task and Path Planning in Large-Scale Cluttered Environments	IEEE Robotics and Automation Letters
43	Liu et al.	2025	Motion Coordination of Swarm Robots for Mobile Target Search	IEEE Transactions on Automation Science and Engineering
44	Hammoud et al.	2025	DYNAMIC FORAGING IN SWARM ROBOTICS: A HYBRID APPROACH WITH MODULAR DESIGN AND DEEP REINFORCEMENT LEARNING INTELLIGENCE	Informatics and Automation
45	Chiun et al.	2025	MARVEL: Multi-Agent Reinforcement Learning for Constrained Field-of-View Multi-Robot Exploration in Large-Scale Environments	2025 IEEE International Conference on Robotics and Automation (ICRA)
46	Zhu et al.	2025	Autonomous and Adaptive Role Selection for Multi-Robot Collaborative Area Search Based on Deep Reinforcement Learning	IEEE Transactions on Automation Science and Engineering

#	Autores	Ano	Título	Publicação
47	Biteng et al.	2025	Training Robot Swarms for Adaptive Foraging in Environments with Obstacles	2025 IEEE International Conference on Mechatronics and Automation (ICMA)
48	Pagliuca & Vitanza	2025	A Comparative Study of Evolutionary Strategies for Aggregation Tasks in Robot Swarms: Macro- and Micro-Level Behavioral Analysis	IEEE Access
49	Zhu et al.	2025	TaskExp: Enhancing Generalization of Multi-Robot Exploration with Multi-Task Pre-Training	2025 IEEE International Conference on Robotics and Automation (ICRA)
50	An et al.	2025	Scalable Multi-Robot Cooperation for Multi-Goal Tasks Using Reinforcement Learning	IEEE Robotics and Automation Letters
51	Zaman et al.	2025	Evolving Adaptive Foraging Robot Swarms with NEAT in Environments with Obstacles	2025 8th International Conference on Intelligent Robotics and Control Engineering (IRCE)
52	Watson et al.	2025	Predator-Prey Q-Learning Based Collaborative Coverage Path Planning for Swarm Robotics	Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)
53	Xin & Dames	2025	Comparison of stochastic optimization strategies in multi-robot multi-target tracking scenarios	Swarm Intelligence

#	Autores	Ano	Título	Publicação
54	Huang et al.	2025	A Hierarchical Multi Robot Coverage Strategy for Large Maps With Reinforcement Learning and Dense Segmented Siamese Network	IEEE Robotics and Automation Letters
55	Zhao et al.	2025	Enhancing Stability of Multi-Underwater Robot Swarms Based on the Fusion of Social Force and Vicsek Models	International Journal of Pattern Recognition and Artificial Intelligence
56	Vashisth et al.	2026	Scalable Multi-Robot Informative Path Planning for Target Mapping via Deep Reinforcement Learning	IEEE Robotics and Automation Letters
57	Mokhtari et al.	2026	Energy harvesting aware path planning for ambiently-powered multi-robot systems	Robotics and Autonomous Systems
58	Ordáz-Rivas et al.	2026	Multi-objective optimization in autonomous foraging using swarm robots	Swarm and Evolutionary Computation

[Página intencionalmente deixada em branco.]

APÊNDICE C

Excertos de Código Relevantes

Apresentam-se os excertos que materializam os principais contributos técnicos descritos nos Capítulos 2 e seguintes. O código integral está disponível no repositório (Apêndice D).

C.1. Mecanismo de Atenção sobre Vizinhos (QKV)

Implementação em PyTorch puro do *forward-pass* da GNNAgent3D, sem recurso a bibliotecas de grafos. Corresponde às Equações 2.18–2.20.

```
1 # h_env: features do ego; h_all: ego + vizinhos embebidos
2 Q = self.query_proj(h_env_expanded)          # consulta (1 x d)
3 K = self.key_proj(h_all)                     # chaves (N x d)
4 V = self.value_proj(h_all)                   # valores (N x d)
5
6 # Atencao escalada por sqrt(d) -> coeficientes alpha_ij
7 scores = torch.bmm(Q, K.transpose(1, 2)) / (self.hidden_dim ** 0.5)
8 alpha = F.softmax(scores, dim=-1)
9 msg_pool = torch.bmm(alpha, V).squeeze(1)    # agregacao ponderada
10
11 # Concatena ego + mensagem agregada e gera a acao continua
12 combined = torch.cat([h_env, msg_pool], dim=1)
13 action = self.actor(combined)                # Tanh -> [-1, 1]^3
```

LISTING C.1. Atenção de produto interno escalado sobre a vizinhança.

C.2. Física de Deslizamento em Muros (*Wall-Sliding*)

Em vez de imobilizar o agente, o motor projeta o movimento ortogonalmente à normal da face atingida, reproduzindo o deslizamento contínuo de um chassi móvel.

```
1 for wall in self.walls:
2     next_pos = self.agent_positions[idx] + move_global
3     delta = next_pos - wall['pos']
4     half_size = wall['size'] / 2.0
5     penetration = (half_size + self.robot_radius) - np.abs(delta)
6     if np.all(penetration > 0):                # ha colisao
7         min_axis = np.argmin(penetration)      # face atingida
8         normal = np.zeros(3)
9         normal[min_axis] = np.sign(delta[min_axis]) or 1.0
10        # remove a componente do movimento contra a normal (desliza)
11        move_global = move_global - np.dot(move_global, normal) * normal
```

LISTING C.2. Projecao ortogonal do movimento contra a normal do muro.

C.3. Campo Geodésico para o *Reward Shaping*

Potencial de navegação Φ usado no termo de progresso nos cenários com paredes. Uma pesquisa de custo uniforme (Dijkstra, 8-conexa) a partir da célula do ninho calcula o comprimento do caminho mais curto que contorna os obstáculos, eliminando o mínimo local da distância euclidiana. O campo é constante por cenário (paredes e ninho fixos), pelo que é calculado uma única vez e reutilizado.

```
1  # paredes dilatadas por r_robot -> caminho fisicamente percorrivél
2  dist = np.full((n, n), np.inf); dist[ni, nj] = 0.0
3  heap = [(0.0, ni, nj)]
4  while heap:
5      d, i, j = heapq.heappop(heap)
6      if d > dist[i, j]:
7          continue
8      for di, dj, cost in neigh:          # 4 ortogonais (res) + 4 diagonais
9                                          (res*sqrt2)
9      ii, jj = i + di, j + dj
10     if 0 <= ii < n and 0 <= jj < n and not blocked[ii, jj]:
11         nd = d + cost
12         if nd < dist[ii, jj]:
13             dist[ii, jj] = nd
14             heapq.heappush(heap, (nd, ii, jj))
15 # Phi(pos) = dist[celula(pos)]; progresso = 10.0 * (Phi_{t-1} - Phi_t)
```

LISTING C.3. Campo de distancia geodesica ao ninho (Dijkstra sobre grelha).

C.4. Recompensa de Exploração *Count-Based*

Bónus atribuído na primeira visita a cada célula de uma grelha de discretização, complementando o termo de progresso geodésico ao premiar a cobertura de zonas novas onde o gradiente do potencial é pouco informativo.

```
1  cell = (int(pos[0] // self.exploration_cell_size),
2          int(pos[1] // self.exploration_cell_size))
3  if cell not in self.visited_cells[idx]:
4      self.visited_cells[idx].add(cell)
5      rewards[agent] += self.exploration_bonus          # +0.5 por celula nova
```

LISTING C.4. Bonus de exploracao por cobertura de celulas novas.

APÊNDICE D

Recursos Adicionais

A totalidade do código-fonte — simulador, implementações dos três algoritmos, *scripts* de avaliação e geração de figuras, e o *dashboard* de controlo experimental — encontra-se disponível, de forma aberta, no repositório do projeto:

<https://github.com/Pombo2001/swarm-robotics-tese>

O repositório inclui o ficheiro `configs/foraging.yaml` com a configuração reproduzida no Apêndice A, um `README.md` com o procedimento de instalação e de reprodução dos resultados (treino, avaliação determinística, testes estatísticos e geração de gráficos), e os modelos treinados por cenário. As demonstrações qualitativas do comportamento emergente são reproduzíveis localmente através do visualizador 3D (motor Ursina), executado a partir do *dashboard* de controlo.